A PRACTICAL SYSTEMS CURRICULUM

AI Inference Engineering

A bounded, evidence-driven program from Python and Linux operations to model serving, observability, and production inference.

Track 1: 16 weeks of career-entry competence
112 daily lessons | 3-4 focused hours per day

AI Inference Engineering Curriculum¶

1. Mission and Progression¶

This revision is an evidence-driven path from **minimum competency** to **demonstrable projects**, **pursue opportunities**, **gain experience**, **specialize**, and **advance**. The initial career target is **Path A: AI Inference Systems / ML Infrastructure / Model Serving Engineering**. The longer-term target is **Path B: Advanced AI Systems / GPU / Low-Level Inference Engineering**.

The five roadmap PDFs remain source material, but they are not prerequisites to finish in order. The independent audit referenced in the brief was not present as a workspace file, so this revision implements the stated audit findings directly and records each curriculum decision in the change log.

Track 1: 16 weeks, 24 focused hours per week, six study/build days per week, approximately 384 hours. The learner may apply for appropriate Path A roles after the final gate; advanced kernel engineering, engine-source modification, and paper reproduction are not entry requirements.

Track 2: optional sequenced modules. Begin while applying or after gaining practical experience. Each module has a prerequisite, bounded depth, exercises, and a stopping condition so specialization does not become an unbounded prerequisite chain.

Progression checkpoints¶

Stage	Evidence	Decision
Minimum competency	Linux/Python environment, containerized vLLM service, benchmark, model-mechanics notes, and failure report	Start applying to Path A internships, junior, and adjacent ML infrastructure roles
Demonstrable projects	Reproducible end-to-end capstone with metrics, quantization comparison, profiling trace, and runbook	Pursue opportunities and request technical review
Gain experience	Operate or improve a real service; document incidents and capacity decisions	Select a Path B specialization based on actual work or hardware
Specialize	Complete one advanced module with measured artifact	Target GPU, runtime, engine, or distributed-inference roles
Advance	Combine modules into a production or research contribution	Own system design, performance, or low-level implementation work

2. Path A Competency Contract¶

At the end of Track 1, the learner must be able to:

1. Write typed, packaged Python for inference tooling, including async I/O.
2. Use Linux command-line tools, `/proc`, `ps`, `top`, signals, logs, and basic troubleshooting workflows.
3. Build reproducible Docker images and run GPU containers with the NVIDIA Container Toolkit and `nvidia-ctk`.
4. Explain tensors, shapes, embeddings, Q/K/V attention, causal masking, logits, and sampling.
5. Explain why prefill and decode have different compute, memory, and latency behavior.
6. Calculate KV-cache memory and estimate context capacity under a GPU memory budget.
7. Deploy and tune vLLM hands-on; compare Triton and SGLang at awareness level.
8. Measure TTFT, ITL, p50, p95, p99, tokens/sec, concurrency, utilization, and failure rate.
9. Use reproducible benchmark controls, roofline reasoning, and evidence-based bottleneck diagnosis.
10. Deploy AWQ, GPTQ, or FP8 when supported and report the quality/performance Pareto tradeoff.
11. Operate an HTTP streaming service with health/readiness, timeouts, draining, and safe shutdown.
12. Expose Prometheus metrics, troubleshoot operational failures, inject faults, and write a post-mortem.
13. Present and document an end-to-end inference-serving capstone.

Depth levels¶

- **Level 1, Awareness:** define it, identify it in documentation, and explain why it exists.
- **Level 2, Working knowledge:** configure it, test normal behavior, and diagnose common failures.
- **Level 3, Implementation:** implement or integrate a bounded version and test it.
- **Level 4, Systems tuning:** connect behavior to memory, scheduling, hardware, latency, reliability, and cost; profile and improve it.

3. Track 1: Minimum Career-Entry Competency¶

Workload contract¶

Each week is 24 hours across six days: four hours per day. Use one hour for exact reading or retrieval, two hours for implementation and lab work, and one hour for measurement, debugging, documentation, or review. Every benchmark records hardware, driver/runtime, software versions, model, prompt/output lengths, concurrency, warm-up policy, metric definitions, repetitions, and failures. Use at least five measured repetitions after warm-up.

Track 1 Weeks 1-8: granular daily curriculum¶

Every day below is one four-hour block. The mapped resource names the exact entry in Section 7. Do not add courses or chapters because a topic feels interesting; obey the stop condition and move on.

Week 1 - Python tooling, async I/O, Linux orientation¶

Day 1: Python environment / Linux shell CORE: Track 1 MINOR: Linux shell, files, and process signals ¶

> **The Big Picture:** Think of this week as setting up a workshop. Python is the tool you will use to automate inference work, while Linux commands let you see what your programs are actually doing instead of guessing.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Create a reproducible Python 3.12 environment; understand `venv`, `pip`, `PATH`, exit codes, stdout/stderr, shell history, and working directories.
- **Secondary Focus [MINOR - 30% Weight]:** Linux shell, files, and process signals. the shell and process tools are the workshop around the Python program.
- **Mapped Guides & Docs:** Python Packaging User Guide, **Writing your pyproject.toml**, https://packaging.python.org/en/latest/guides/writing-pyproject-toml/. Study project metadata and dependencies; skip publishing.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day01` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Create `track1-client/`, initialize a virtual environment, add `pyproject.toml`, install the package locally, write `hello_inference.py`, run it through the shell, redirect stdout and stderr, and record the commands from a clean terminal.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `track1-client/pyproject.toml`, `hello_inference.py`, and `environment.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Linux shell, files, and process signals: Run `pwd`, `python --version`, `ps`, and `echo $?`; save the output beside the primary artifact.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Python environment / Linux shell` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for python environment / linux shell. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** A new terminal can recreate the environment, install the package, run the script, and explain the difference between stdout, stderr, exit code, and the active interpreter.

- **Tangible Deliverable:** `track1-client/pyproject.toml`, `hello_inference.py`, and `environment.md`.
- **STOP WHEN:** A new terminal can recreate the environment, install the package, run the script, and explain the difference between stdout, stderr, exit code, and the active interpreter.
- **Do Not Study Yet:** Poetry internals, publishing to PyPI, shell implementation, Linux kernel internals, and Python metaprogramming.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 2: Python functions, typing, and structured configuration CORE: Track 1

MINOR: Linux shell, files, and process signals ¶

> **The Big Picture:** Think of this week as setting up a workshop. Python is the tool you will use to automate inference work, while Linux commands let you see what your programs are actually doing instead of guessing.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Use functions, type annotations, `dataclass`, enums, JSON, and explicit error handling to represent an inference request and configuration.
- **Secondary Focus [MINOR - 30% Weight]:** Linux shell, files, and process signals. the shell and process tools are the workshop around the Python program.
- **Mapped Guides & Docs:** Python Packaging User Guide, **Writing your pyproject.toml**, URL above. Study metadata and console scripts; skip build backend internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day02` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Define typed `InferenceRequest`, `GenerationConfig`, and `ServerConfig` objects. Parse a JSON request, validate required fields, reject an invalid token limit, and expose a CLI command named `infer-config`.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `src/inference_client/config.py`, `src/inference_client/cli.py`, and tests for valid and invalid JSON..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Linux shell, files, and process signals: Run `pwd`, `python --version`, `ps`, and `echo $?`; save the output beside the primary artifact.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Python functions, typing, and structured configuration` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for python functions, typing, and structured configuration. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The CLI returns a typed object for valid input, a useful error for invalid input, and tests pass from a clean environment.

- **Tangible Deliverable:** `src/inference_client/config.py`, `src/inference_client/cli.py`, and tests for valid and invalid JSON.
- **STOP WHEN:** The CLI returns a typed object for valid input, a useful error for invalid input, and tests pass from a clean environment.
- **Do Not Study Yet:** Pydantic internals, decorators beyond basic use, metaclasses, static type-checker implementation, and framework dependency injection.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 3: Async tasks, cancellation, and timeouts CORE: Track 1 MINOR: Linux shell, files, and process signals ¶

> **The Big Picture:** Think of this week as setting up a workshop. Python is the tool you will use to automate inference work, while Linux commands let you see what your programs are actually doing instead of guessing.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Understand coroutine, task, await, timeout, cancellation, and cleanup in `asyncio`.
- **Secondary Focus [MINOR - 30% Weight]:** Linux shell, files, and process signals. the shell and process tools are the workshop around the Python program.
- **Mapped Guides & Docs:** Python docs, **Coroutines and Tasks**, https://docs.python.org/3/library/asyncio-task.html. Study tasks, cancellation, and timeouts; skip event-loop implementation.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day03` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Write an async fake inference call that sleeps for a configurable duration. Run three requests concurrently, apply a deadline, cancel one task, and use `try/finally` to record completion state.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `src/inference_client/async_demo.py` and `tests/test_async_demo.py`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Linux shell, files, and process signals: Run `pwd`, `python --version`, `ps`, and `echo $?`; save the output beside the primary artifact.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Async tasks, cancellation, and timeouts` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not confuse a cancelled client with a completed request. Check task state and server logs, then verify cleanup rather than trusting the HTTP status alone.

> **Daily boundary:** Concurrent work overlaps, timeout raises a controlled error, cancellation executes cleanup, and the test proves no task remains pending.

- **Tangible Deliverable:** `src/inference_client/async_demo.py` and `tests/test_async_demo.py`.
- **STOP WHEN:** Concurrent work overlaps, timeout raises a controlled error, cancellation executes cleanup, and the test proves no task remains pending.
- **Do Not Study Yet:** Custom event loops, lock-free async structures, uvloop internals, and coroutine bytecode.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 4: Async HTTP client and streaming chunks CORE: Track 1 MINOR: Linux shell, files, and process signals ¶

> **The Big Picture:** Think of this week as setting up a workshop. Python is the tool you will use to automate inference work, while Linux commands let you see what your programs are actually doing instead of guessing.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Make an async HTTP request, consume a streaming response, handle status codes, and preserve request IDs.
- **Secondary Focus [MINOR - 30% Weight]:** Linux shell, files, and process signals. the shell and process tools are the workshop around the Python program.
- **Mapped Guides & Docs:** Python docs, **Coroutines and Tasks**, URL above; use the official HTTP library documentation for the installed client as a reference only. Study cancellation and timeout interaction.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day04` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Create a local fake streaming endpoint or use a small test server. Implement an async client that sends JSON, consumes newline-delimited chunks, timestamps first and final chunks, and cancels on deadline.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `src/inference_client/client.py`, `tests/test_client.py`, and `logs/day04.jsonl`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Linux shell, files, and process signals: Run `pwd`, `python --version`, `ps`, and `echo $?`; save the output beside the primary artifact.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Async HTTP client and streaming chunks` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not confuse a cancelled client with a completed request. Check task state and server logs, then verify cleanup rather than trusting the HTTP status alone.

> **Daily boundary:** The client handles normal stream, HTTP error, server delay, and cancellation while producing request ID, first-byte time, final time, and error type.

- **Tangible Deliverable:** `src/inference_client/client.py`, `tests/test_client.py`, and `logs/day04.jsonl`.
- **STOP WHEN:** The client handles normal stream, HTTP error, server delay, and cancellation while producing request ID, first-byte time, final time, and error type.

- **Do Not Study Yet:** HTTP/2 internals, TLS implementation, connection-pool internals, and distributed tracing libraries.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 5: Linux files, processes, and observations CORE: Track 1 MINOR: Linux shell, files, and process signals ¶

> **The Big Picture:** Think of this week as setting up a workshop. Python is the tool you will use to automate inference work, while Linux commands let you see what your programs are actually doing instead of guessing.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Use shell tools to inspect files, processes, environment, permissions, and exit status.
- **Secondary Focus [MINOR - 30% Weight]:** Linux shell, files, and process signals. the shell and process tools are the workshop around the Python program.
- **Mapped Guides & Docs:** Linux man-pages, **ps(1)**, https://man7.org/linux/man-pages/man1/ps.1.html. Study process selection and fields; skip procfs implementation.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day05` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Run the async client while capturing `pwd`, `env`, `ls -l`, `ps -ef`, exit status, and open output files. Write a shell script that records the current process tree and command line.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `scripts/observe_process.sh` and `logs/day05-process.txt`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Linux shell, files, and process signals: Run `pwd`, `python --version`, `ps`, and `echo $?`; save the output beside the primary artifact.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Linux files, processes, and observations` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for linux files, processes, and observations. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The script records PID, parent PID, command, status, working directory, and exit result for the client, and you can explain each field.

- **Tangible Deliverable:** `scripts/observe_process.sh` and `logs/day05-process.txt`.
- **STOP WHEN:** The script records PID, parent PID, command, status, working directory, and exit result for the client, and you can explain each field.
- **Do Not Study Yet:** Bash parser implementation, systemd internals, namespaces, and kernel scheduling.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 6: First local inference request CORE: Track 1 MINOR: Linux shell, files, and process signals ¶

> **The Big Picture:** Think of this week as setting up a workshop. Python is the tool you will use to automate inference work, while Linux commands let you see what your programs are actually doing instead of guessing.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Connect the typed client, async stream handling, and Linux observation to one model request; define token, TTFT, output length, and failure rate.
- **Secondary Focus [MINOR - 30% Weight]:** Linux shell, files, and process signals. the shell and process tools are the workshop around the Python program.
- **Mapped Guides & Docs:** Hugging Face Transformers docs, **Model outputs**, https://huggingface.co/docs/transformers/main/en/main_classes/output. Study inputs, logits, and generated output; skip fine-tuning.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day06` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Run a small local model or documented CPU fallback. Send five prompts, record request ID, prompt length, output length, first token time, final time, and failures, while observing the process.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `logs/day06-inference.jsonl` and `notes/first-request.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Linux shell, files, and process signals: Run `pwd`, `python --version`, `ps`, and `echo $?`; save the output beside the primary artifact.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `First local inference request` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for first local inference request. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Five requests complete or fail with classified errors, the timing fields are populated, and you can describe the request path from shell to response.

- **Tangible Deliverable:** `logs/day06-inference.jsonl` and `notes/first-request.md`.
- **STOP WHEN:** Five requests complete or fail with classified errors, the timing fields are populated, and you can describe the request path from shell to response.
- **Do Not Study Yet:** Model training, tokenizer implementation, GPU kernel work, and production deployment.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 7: Week 1 review and gate CORE: Track 1 MINOR: Linux shell, files, and process signals ¶

> **The Big Picture:** Think of this week as setting up a workshop. Python is the tool you will use to automate inference work, while Linux commands let you see what your programs are actually doing instead of guessing.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Retrieve Python packaging, async cancellation, HTTP streaming, and process-inspection knowledge without notes.
- **Secondary Focus [MINOR - 30% Weight]:** Linux shell, files, and process signals. the shell and process tools are the workshop around the Python program.
- **Mapped Guides & Docs:** Revisit the exact Python Packaging Guide, Python **Coroutines and Tasks**, and Linux **ps(1)** entries from Section 7; read no new material.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day07` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Delete the virtual environment, recreate it, run the package, start the fake server, cancel one request, and produce a process report. Answer ten short questions and repair one seeded timeout bug.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week01-report.md`, recreated environment log, and corrected test commit..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Linux shell, files, and process signals: Run `pwd`, `python --version`, `ps`, and `echo $?`; save the output beside the primary artifact.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 1 review and gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for week 1 review and gate. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Clean rebuild succeeds, all tests pass, timeout/cancellation behavior is explained, and the report contains one failure, cause, fix, and evidence.

- **Tangible Deliverable:** `gates/week01-report.md`, recreated environment log, and corrected test commit.
- **STOP WHEN:** Clean rebuild succeeds, all tests pass, timeout/cancellation behavior is explained, and the report contains one failure, cause, fix, and evidence.
- **Do Not Study Yet:** Docker, vLLM source, advanced typing, and Python performance tuning.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 8: Linux process state and `/proc` CORE: Track 1 MINOR: Linux process inspection and GPU runtime checks ¶

> **The Big Picture:** A container is a labeled box for the software and its dependencies. Linux and the NVIDIA runtime are the loading dock: they determine whether the box can actually reach the GPU, files, ports, and logs it needs.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Read PID, PPID, state, threads, virtual memory, RSS, and file descriptors from `/proc`.
- **Secondary Focus [MINOR - 30% Weight]:** Linux process inspection and GPU runtime checks. the operating environment must be visible and reproducible.
- **Mapped Guides & Docs:** Linux man-pages, **proc_pid_status(5)**, https://man7.org/linux/man-pages/man5/proc_pid_status.5.html. Study RSS, threads, signals, and process state; skip kernel source.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day08` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Add a long-running test server, locate its PID, read `/proc/PID/status`, `/proc/PID/smaps_rollup`, `/proc/PID/cmdline`, and `/proc/PID/fd`, then stop it cleanly.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `logs/day08-proc-report.md` and `scripts/proc_snapshot.sh`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Linux process inspection and GPU runtime checks: Run `docker ps`, inspect the relevant process or container logs, and record GPU visibility or the exact missing prerequisite.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Linux process state and`/proc`` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for linux process state and `/proc`. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The snapshot records process identity, thread count, VmRSS, VmSize, open descriptors, and signal state, and you can distinguish virtual from resident memory.

- **Tangible Deliverable:** `logs/day08-proc-report.md` and `scripts/proc_snapshot.sh`.
- **STOP WHEN:** The snapshot records process identity, thread count, VmRSS, VmSize, open descriptors, and signal state, and you can distinguish virtual from resident memory.
- **Do Not Study Yet:** eBPF, cgroups internals, NUMA, page-table implementation, and kernel memory reclaim.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 9: Linux CPU, memory, disk, and sockets CORE: Track 1

MINOR: Linux process inspection and GPU runtime checks ¶

> **The Big Picture:** A container is a labeled box for the software and its dependencies. Linux and the NVIDIA runtime are the loading dock: they determine whether the box can actually reach the GPU, files, ports, and logs it needs.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Troubleshoot a service using `ps`, `top`, `df`, `du`, `ss`, logs, and controlled load.
- **Secondary Focus [MINOR - 30% Weight]:** Linux process inspection and GPU runtime checks. the operating environment must be visible and reproducible.

- **Mapped Guides & Docs:** Linux man-pages, **ps(1)** and **proc_pid_status(5)** entries from Section 7. Study operational fields; skip command implementation.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day09` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Run the fake server under three conditions: idle, CPU-heavy handler, and blocked handler. Capture process snapshots, disk/cache usage, socket state, and logs; classify the symptom.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `logs/day09-troubleshooting.md` with three evidence-backed diagnoses..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Linux process inspection and GPU runtime checks: Run `docker ps`, inspect the relevant process or container logs, and record GPU visibility or the exact missing prerequisite.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Linux CPU, memory, disk, and sockets` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not compare runs with different prompt/output lengths or forget the factor of two for keys and values. Record units and assumptions beside every number.

> **Daily boundary:** Each condition has a distinct observation, a bounded diagnosis, and a command or log line supporting the conclusion.

- **Tangible Deliverable:** `logs/day09-troubleshooting.md` with three evidence-backed diagnoses.
- **STOP WHEN:** Each condition has a distinct observation, a bounded diagnosis, and a command or log line supporting the conclusion.
- **Do Not Study Yet:** Full Linux performance tracing, eBPF, perf event internals, and kernel tuning.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 10: Dockerfile and reproducible Python image CORE: Track 1

MINOR: Linux process inspection and GPU runtime checks ¶

> **The Big Picture:** A container is a labeled box for the software and its dependencies. Linux and the NVIDIA runtime are the loading dock: they determine whether the box can actually reach the GPU, files, ports, and logs it needs.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Understand image layers, base image, `RUN`, `COPY`, `ENV`, ports, non-root execution, and reproducible dependency installation.
- **Secondary Focus [MINOR - 30% Weight]:** Linux process inspection and GPU runtime checks. the operating environment must be visible and reproducible.
- **Mapped Guides & Docs:** Docker Docs, **Dockerfile reference**, https://docs.docker.com/reference/dockerfile/. Study `FROM`, `RUN`, `COPY`, `ENV`, ports, and health checks; skip BuildKit internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day10` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Write a Dockerfile for the typed client/fake server, pin the Python base image tag and dependencies, build it, run it with a mounted log directory, and compare clean rebuild behavior.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `Dockerfile`, `.dockerignore`, `docker-compose.dev.yml` or run command, and `logs/day10-build.txt`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Linux process inspection and GPU runtime checks: Run `docker ps`, inspect the relevant process or container logs, and record GPU visibility or the exact missing prerequisite.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Dockerfile and reproducible Python image` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not assume a container can see the host GPU automatically. Verify the runtime, image, device visibility, mounted paths, and logs separately.

> **Daily boundary:** The image builds from an empty local layer cache, runs as a non-root user, serves the test endpoint, and writes logs to the mounted directory.

- **Tangible Deliverable:** `Dockerfile`, `.dockerignore`, `docker-compose.dev.yml` or run command, and `logs/day10-build.txt`.
- **STOP WHEN:** The image builds from an empty local layer cache, runs as a non-root user, serves the test endpoint, and writes logs to the mounted directory.
- **Do Not Study Yet:** Multi-stage compiler optimization, image-builder internals, Kubernetes, and registry operations.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 11: NVIDIA Container Toolkit and `nvidia-ctk` CORE: Track 1

MINOR: Linux process inspection and GPU runtime checks ¶

> **The Big Picture:** A container is a labeled box for the software and its dependencies. Linux and the NVIDIA runtime are the loading dock: they determine whether the box can actually reach the GPU, files, ports, and logs it needs.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Understand host driver versus container CUDA libraries, GPU visibility, runtime configuration, and the limits of container portability.
- **Secondary Focus [MINOR - 30% Weight]:** Linux process inspection and GPU runtime checks. the operating environment must be visible and reproducible.
- **Mapped Guides & Docs:** NVIDIA Container Toolkit, **Installing the NVIDIA Container Toolkit**, https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html. Study runtime configuration and `nvidia-ctk`; skip Kubernetes integration.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day11` before changing code.

- ☐ **[CORE] Task 2: Core Implementation:** Verify host driver and GPU, run `nvidia-ctk runtime configure --runtime=docker` where appropriate, restart Docker if required, and run a CUDA base image with `nvidia-smi`.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `notes/day11-gpu-container.md` with host driver, toolkit, image, GPU name, and exact commands..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Linux process inspection and GPU runtime checks: Run `docker ps`, inspect the relevant process or container logs, and record GPU visibility or the exact missing prerequisite.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `NVIDIA Container Toolkit and`nvidia-ctk`` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not assume a container can see the host GPU automatically. Verify the runtime, image, device visibility, mounted paths, and logs separately.

> **Daily boundary:** A documented container sees the intended GPU, `nvidia-smi` works inside it, and you can explain which components remain host responsibilities.

- **Tangible Deliverable:** `notes/day11-gpu-container.md` with host driver, toolkit, image, GPU name, and exact commands.

- **STOP WHEN:** A documented container sees the intended GPU, `nvidia-smi` works inside it, and you can explain which components remain host responsibilities.
- **Do Not Study Yet:** CUDA kernel authoring, driver internals, MIG administration, and multi-node GPU networking.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 12: Model cache and GPU-serving image CORE: Track 1

MINOR: Linux process inspection and GPU runtime checks ¶

> **The Big Picture:** A container is a labeled box for the software and its dependencies. Linux and the NVIDIA runtime are the loading dock: they determine whether the box can actually reach the GPU, files, ports, and logs it needs.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Keep model artifacts outside the image, use volumes, set cache paths, expose ports, and distinguish image reproducibility from model reproducibility.
- **Secondary Focus [MINOR - 30% Weight]:** Linux process inspection and GPU runtime checks. the operating environment must be visible and reproducible.
- **Mapped Guides & Docs:** Docker Docs, **Dockerfile reference**, URL above; NVIDIA Container Toolkit installation guide, URL above. Study volumes, environment, and GPU runtime only.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day12` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Extend the image with a model-cache volume and a health command. Run a small model-serving placeholder or vLLM image if hardware permits; otherwise validate the same volume/runtime contract with the fake server.

- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `docker-compose.gpu.yml` or `scripts/run_gpu_container.ps1`, `notes/day12-cache.md`, and startup log..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Linux process inspection and GPU runtime checks: Run `docker ps`, inspect the relevant process or container logs, and record GPU visibility or the exact missing prerequisite.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Model cache and GPU-serving image` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not assume a container can see the host GPU automatically. Verify the runtime, image, device visibility, mounted paths, and logs separately.

> **Daily boundary:** A stopped-and-restarted container reuses the documented cache, exposes the service port, and fails with a clear prerequisite message when GPU access is unavailable.

- **Tangible Deliverable:** `docker-compose.gpu.yml` or `scripts/run_gpu_container.ps1`, `notes/day12-cache.md`, and startup log.
- **STOP WHEN:** A stopped-and-restarted container reuses the documented cache, exposes the service port, and fails with a clear prerequisite message when GPU access is unavailable.
- **Do Not Study Yet:** Kubernetes persistent volumes, image signing, cluster scheduling, and distributed model loading.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 13: Container troubleshooting CORE: Track 1 MINOR: Linux process inspection and GPU runtime checks ¶

> **The Big Picture:** A container is a labeled box for the software and its dependencies. Linux and the NVIDIA runtime are the loading dock: they determine whether the box can actually reach the GPU, files, ports, and logs it needs.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Diagnose image startup failure, missing environment, port collision, permission issue, missing GPU, and missing model cache.
- **Secondary Focus [MINOR - 30% Weight]:** Linux process inspection and GPU runtime checks. the operating environment must be visible and reproducible.
- **Mapped Guides & Docs:** Docker Docs, **Dockerfile reference**, and NVIDIA Container Toolkit installation guide from Section 7. Study only the commands needed to diagnose the five failure classes.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day13` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Seed each failure one at a time, capture `docker ps`, logs, inspect output, exit code, GPU visibility, and mounted paths, then repair it.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `logs/day13-container-failures.md` with symptom, evidence, root cause, fix, and prevention for all five cases..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Linux process inspection and GPU runtime checks: Run `docker ps`, inspect the relevant process or container logs, and record GPU visibility or the exact

missing prerequisite.

- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Container troubleshooting` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not assume a container can see the host GPU automatically. Verify the runtime, image, device visibility, mounted paths, and logs separately.

> **Daily boundary:** Five failures are reproducible and repaired without guesswork; the report distinguishes host, image, runtime, and application causes.

- **Tangible Deliverable:** `logs/day13-container-failures.md` with symptom, evidence, root cause, fix, and prevention for all five cases.

- **STOP WHEN:** Five failures are reproducible and repaired without guesswork; the report distinguishes host, image, runtime, and application causes.
- **Do Not Study Yet:** Container runtime implementation, Kubernetes debugging, and cloud-specific GPU provisioning.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 14: Week 2 review and gate CORE: Track 1 MINOR: Linux process inspection and GPU runtime checks ¶

> **The Big Picture:** A container is a labeled box for the software and its dependencies. Linux and the NVIDIA runtime are the loading dock: they determine whether the box can actually reach the GPU, files, ports, and logs it needs.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Rebuild the Python service in Docker, inspect its process and memory state, and validate GPU runtime prerequisites.
- **Secondary Focus [MINOR - 30% Weight]:** Linux process inspection and GPU runtime checks. the operating environment must be visible and reproducible.
- **Mapped Guides & Docs:** Revisit Docker **Dockerfile reference** and NVIDIA Container Toolkit installation guide; no new reading.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day14` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** From an empty build cache, build and run the image, execute the client, capture `/proc` data from inside or beside the container, and complete a ten-question troubleshooting test.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week02-report.md`, image digest, run log, and failure table..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Linux process inspection and GPU runtime checks: Run `docker ps`, inspect the relevant process or container logs, and record GPU visibility or the exact missing prerequisite.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 2 review and gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the

symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for week 2 review and gate. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** A clean-shell command builds the image, starts it, returns a response, and the learner can diagnose one intentional missing-GPU or missing-volume failure.

- **Tangible Deliverable:** `gates/week02-report.md`, image digest, run log, and failure table.
- **STOP WHEN:** A clean-shell command builds the image, starts it, returns a response, and the learner can diagnose one intentional missing-GPU or missing-volume failure.
- **Do Not Study Yet:** Kubernetes, Triton deployment, custom Docker networking, and container security hardening beyond non-root execution.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 15: Tensor shape, dtype, and device CORE: Track 1 MINOR: Numerical precision and tensor-shape checks ¶

> **The Big Picture:** A transformer is a chain of shape-preserving math operations. Treat tensor shapes like the dimensions written on pipes: if one pipe is the wrong size, the next operation either fails loudly or produces a misleading result.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Read tensor rank, shape, dtype, device, indexing, broadcasting, and contiguous versus non-contiguous awareness.
- **Secondary Focus [MINOR - 30% Weight]:** Numerical precision and tensor-shape checks. small shape and dtype mistakes can silently change model behavior.
- **Mapped Guides & Docs:** PyTorch Tutorials, **Tensors**, https://pytorch.org/tutorials/beginner/basics/tensorqs_tutorial.html. Study shape, dtype, device, indexing, and broadcasting; skip training loops.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day15` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Create tensors for token IDs, embeddings, hidden states, and logits. Print shapes/dtypes/devices at each stage and add assertions for expected ranks.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `notebooks/day15_tensor_shapes.py` or `.ipynb`, plus `notes/day15-shapes.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Numerical precision and tensor-shape checks: Add one assertion for shape/dtype/device and deliberately make it fail once before restoring it.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Tensor shape, dtype, and device` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not rely on a printed value without checking shape, dtype, device, and tolerance. A tensor can look plausible while having the wrong dimension.

> **Daily boundary:** Five intentional shape/dtype errors are caught by assertions and you can explain each tensor's role in one forward pass.

- **Tangible Deliverable:** `notebooks/day15_tensor_shapes.py` or `.ipynb`, plus `notes/day15-shapes.md`.
- **STOP WHEN:** Five intentional shape/dtype errors are caught by assertions and you can explain each tensor's role in one forward pass.
- **Do Not Study Yet:** Autograd internals, distributed tensors, custom operators, and compiler graph lowering.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 16: Token IDs, embeddings, and model inputs CORE: Track 1

MINOR: Numerical precision and tensor-shape checks ¶

> **The Big Picture:** A transformer is a chain of shape-preserving math operations. Treat tensor shapes like the dimensions written on pipes: if one pipe is the wrong size, the next operation either fails loudly or produces a misleading result.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Understand token IDs, vocabulary dimension, embedding lookup, batch dimension, sequence dimension, and attention-mask shape.
- **Secondary Focus [MINOR - 30% Weight]:** Numerical precision and tensor-shape checks. small shape and dtype mistakes can silently change model behavior.
- **Mapped Guides & Docs:** Hugging Face Transformers docs, **Model outputs**, https://huggingface.co/docs/transformers/main/en/main_classes/output. Study model inputs and output containers; skip fine-tuning.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day16` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Tokenize three prompts with a small model, print IDs and masks, gather embedding rows, and verify batch/sequence shapes for padded and unpadded inputs.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day16-token-trace.json` and `notes/day16-embedding-trace.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Numerical precision and tensor-shape checks: Add one assertion for shape/dtype/device and deliberately make it fail once before restoring it.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Token IDs, embeddings, and model inputs` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not rely on a printed value without checking shape, dtype, device, and tolerance. A tensor can look plausible while having the wrong dimension.

> **Daily boundary:** You can trace text to token IDs to embeddings, explain padding/mask dimensions, and detect one incorrect batch shape.

- **Tangible Deliverable:** `artifacts/day16-token-trace.json` and `notes/day16-embedding-trace.md`.

- **STOP WHEN:** You can trace text to token IDs to embeddings, explain padding/mask dimensions, and detect one incorrect batch shape.
- **Do Not Study Yet:** Tokenizer training, BPE algorithm implementation, embedding training, and multilingual model comparison.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 17: Q, K, V and scaled dot-product attention CORE: Track 1

MINOR: Numerical precision and tensor-shape checks ¶

> **The Big Picture:** A transformer is a chain of shape-preserving math operations. Treat tensor shapes like the dimensions written on pipes: if one pipe is the wrong size, the next operation either fails loudly or produces a misleading result.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Understand Q/K/V projections, head dimension, score matrix shape, scaling, weighted values, and output shape.
- **Secondary Focus [MINOR - 30% Weight]:** Numerical precision and tensor-shape checks. small shape and dtype mistakes can silently change model behavior.
- **Mapped Guides & Docs:** PyTorch docs, **scaled_dot_product_attention**, https://pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html. Study Q/K/V, scale, causal mask, and output shape; skip backend kernels.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day17` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Implement one attention head with small deterministic tensors. Print every intermediate shape and compare against PyTorch's reference function.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `src/inference_math/attention_reference.py` and `tests/test_attention_reference.py`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Numerical precision and tensor-shape checks: Add one assertion for shape/dtype/device and deliberately make it fail once before restoring it.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Q, K, V and scaled dot-product attention` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not rely on a printed value without checking shape, dtype, device, and tolerance. A tensor can look plausible while having the wrong dimension.

> **Daily boundary:** The implementation matches the reference within a stated tolerance for two shapes and you can explain why the score matrix is sequence-by-sequence.

- **Tangible Deliverable:** `src/inference_math/attention_reference.py` and `tests/test_attention_reference.py`.
- **STOP WHEN:** The implementation matches the reference within a stated tolerance for two shapes and you can explain why the score matrix is sequence-by-sequence.
- **Do Not Study Yet:** FlashAttention, fused kernels, multi-query attention variants, and attention research papers.

- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 18: Causal masking and numerical behavior CORE: Track 1

MINOR: Numerical precision and tensor-shape checks ¶

> **The Big Picture:** A transformer is a chain of shape-preserving math operations. Treat tensor shapes like the dimensions written on pipes: if one pipe is the wrong size, the next operation either fails loudly or produces a misleading result.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Prevent future-token access; understand mask values, softmax stability, dtype effects, and output comparison tolerance.
- **Secondary Focus [MINOR - 30% Weight]:** Numerical precision and tensor-shape checks. small shape and dtype mistakes can silently change model behavior.
- **Mapped Guides & Docs:** PyTorch docs, **scaled_dot_product_attention**, URL above. Study causal masking and output behavior; skip implementation details of optimized backends.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day18` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Add a causal mask to the reference attention, deliberately remove it to create a failing test, then restore it. Compare FP32 and FP16/BF16 if hardware supports them.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `tests/test_causal_attention.py` and `logs/day18-numerics.json`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Numerical precision and tensor-shape checks: Add one assertion for shape/dtype/device and deliberately make it fail once before restoring it.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Causal masking and numerical behavior` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for causal masking and numerical behavior. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The test fails when future access is possible, passes after repair, and you can state the accepted numerical tolerance and dtype limitation.

- **Tangible Deliverable:** `tests/test_causal_attention.py` and `logs/day18-numerics.json`.
- **STOP WHEN:** The test fails when future access is possible, passes after repair, and you can state the accepted numerical tolerance and dtype limitation.
- **Do Not Study Yet:** Mixed-precision training, loss scaling, custom softmax kernels, and numerical-analysis research.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 19: Logits, softmax, and sampling CORE: Track 1 MINOR: Numerical precision and tensor-shape checks ¶

> **The Big Picture:** A transformer is a chain of shape-preserving math operations. Treat tensor shapes like the dimensions written on pipes: if one pipe is the wrong size, the next operation either fails loudly or produces a misleading result.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Interpret logits, softmax probabilities, greedy choice, temperature, top-k, top-p, and deterministic versus sampled output.
- **Secondary Focus [MINOR - 30% Weight]:** Numerical precision and tensor-shape checks. small shape and dtype mistakes can silently change model behavior.
- **Mapped Guides & Docs:** Hugging Face Transformers docs, **Model outputs**, URL above. Study logits and generated outputs; skip generation internals.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day19` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Implement greedy and top-k sampling over a fixed logits vector, then compare temperature settings and record selected tokens with a fixed seed.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `src/inference_math/sampling.py`, tests, and `logs/day19-sampling.json`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Numerical precision and tensor-shape checks: Add one assertion for shape/dtype/device and deliberately make it fail once before restoring it.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Logits, softmax, and sampling` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not rely on a printed value without checking shape, dtype, device, and tolerance. A tensor can look plausible while having the wrong dimension.

> **Daily boundary:** Greedy output is deterministic, seeded sampling is repeatable, and you can explain how temperature changes the distribution without claiming it improves quality universally.

- **Tangible Deliverable:** `src/inference_math/sampling.py`, tests, and `logs/day19-sampling.json`.
- **STOP WHEN:** Greedy output is deterministic, seeded sampling is repeatable, and you can explain how temperature changes the distribution without claiming it improves quality universally.
- **Do Not Study Yet:** Beam-search research, RLHF, decoding theory, and training-time probability estimation.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 20: Model output trace CORE: Track 1 MINOR: Numerical precision and tensor-shape checks ¶

> **The Big Picture:** A transformer is a chain of shape-preserving math operations. Treat tensor shapes like the dimensions written on pipes: if one pipe is the wrong size, the next operation either fails loudly or produces a misleading result.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Trace inputs, hidden states, logits, generated IDs, and output text through an actual model request.
- **Secondary Focus [MINOR - 30% Weight]:** Numerical precision and tensor-shape checks. small shape and dtype mistakes can silently change model behavior.
- **Mapped Guides & Docs:** Hugging Face Transformers docs, **Model outputs**, URL above. Study logits, hidden states, and cache concepts; skip fine-tuning.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day20` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Run one small model with output inspection enabled. Record input IDs, selected output IDs, logits shape, hidden-state shape if available, and timing for prompt and generated tokens.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day20-model-output.json` and `notes/day20-forward-trace.md`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Numerical precision and tensor-shape checks: Add one assertion for shape/dtype/device and deliberately make it fail once before restoring it.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Model output trace` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for model output trace. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The trace includes all available shapes, the output can be reproduced under greedy decoding, and one mismatch is either fixed or documented as a model limitation.

- **Tangible Deliverable:** `artifacts/day20-model-output.json` and `notes/day20-forward-trace.md`.
- **STOP WHEN:** The trace includes all available shapes, the output can be reproduced under greedy decoding, and one mismatch is either fixed or documented as a model limitation.
- **Do Not Study Yet:** Training loops, gradient inspection, model conversion, and architecture surveys.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 21: Week 3 review and gate CORE: Track 1 MINOR: Numerical precision and tensor-shape checks ¶

> **The Big Picture:** A transformer is a chain of shape-preserving math operations. Treat tensor shapes like the dimensions written on pipes: if one pipe is the wrong size, the next operation either fails loudly or produces a misleading result.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Demonstrate tensor-shape literacy, attention correctness, causal masking, and sampling behavior.
- **Secondary Focus [MINOR - 30% Weight]:** Numerical precision and tensor-shape checks. small shape and dtype mistakes can silently change model behavior.
- **Mapped Guides & Docs:** Revisit PyTorch **Tensors**, PyTorch **scaled_dot_product_attention**, and Hugging Face **Model outputs**; no new reading.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day21` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Complete a closed-book shape quiz, implement one attention-head function from a signature, repair a removed causal mask, and explain one model-output trace in five minutes.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week03-report.md`, test output, and annotated tensor diagram..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Numerical precision and tensor-shape checks: Add one assertion for shape/dtype/device and deliberately make it fail once before restoring it.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 3 review and gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for week 3 review and gate. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Shape implementation, causal test, sampling test, and oral explanation all pass; otherwise repeat only the failed topic card.

- **Tangible Deliverable:** `gates/week03-report.md`, test output, and annotated tensor diagram.
- **STOP WHEN:** Shape implementation, causal test, sampling test, and oral explanation all pass; otherwise repeat only the failed topic card.
- **Do Not Study Yet:** KV cache, batching, quantization, and inference-engine configuration.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 22: Prefill versus decode CORE: Track 1 MINOR: Memory layout and per-token footprint analysis ¶

> **The Big Picture:** Prefill is the model reading the prompt; decode is the model writing one new token at a time. The KV cache is like keeping notes from the pages already read so the model does not reread the whole book for every word.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Distinguish prompt prefill from autoregressive decode, token loop behavior, TTFT, ITL, compute work, and memory behavior.
- **Secondary Focus [MINOR - 30% Weight]:** Memory layout and per-token footprint analysis. capacity estimates connect model math to real GPU limits.
- **Mapped Guides & Docs:** Hugging Face Transformers docs, **Model outputs**, URL above; use the model-output and cache concepts only. Skip fine-tuning and trainer material.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day22` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Instrument generation to record prompt processing duration, first generated token, each subsequent token, final response, prompt tokens, and output tokens.

- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `logs/day22-prefill-decode.jsonl` and `notes/day22-phase-comparison.md`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Memory layout and per-token footprint analysis: Write down units, dimensions, and one memory-footprint calculation beside the experiment.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Prefill versus decode` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not compare runs with different prompt/output lengths or forget the factor of two for keys and values. Record units and assumptions beside every number.

> **Daily boundary:** You can calculate TTFT and ITL from the log and explain why prompt length affects prefill while output length affects decode.

- **Tangible Deliverable:** `logs/day22-prefill-decode.jsonl` and `notes/day22-phase-comparison.md`.
- **STOP WHEN:** You can calculate TTFT and ITL from the log and explain why prompt length affects prefill while output length affects decode.
- **Do Not Study Yet:** Speculative decoding, disaggregated serving, and attention-kernel implementation.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 23: Reference autoregressive token loop CORE: Track 1 MINOR: Memory layout and per-token footprint analysis ¶

> **The Big Picture:** Prefill is the model reading the prompt; decode is the model writing one new token at a time. The KV cache is like keeping notes from the pages already read so the model does not reread the whole book for every word.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Implement one-token-at-a-time generation, stop conditions, maximum output length, and prompt/output accounting.
- **Secondary Focus [MINOR - 30% Weight]:** Memory layout and per-token footprint analysis. capacity estimates connect model math to real GPU limits.
- **Mapped Guides & Docs:** Hugging Face Transformers docs, **Model outputs**, URL above. Study inputs, logits, and generated outputs; skip training.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day23` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Write a small reference loop around the model's forward call, append one token per iteration, stop on EOS or max tokens, and compare with the library generation path.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `src/inference_math/reference_generate.py` and `tests/test_reference_generate.py`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Memory layout and per-token footprint analysis: Write down units, dimensions, and one memory-footprint calculation beside the experiment.

- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Reference autoregressive token loop` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for reference autoregressive token loop. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Greedy output matches the library for three prompts, termination works for EOS and max length, and token counts are correct.

- **Tangible Deliverable:** `src/inference_math/reference_generate.py` and `tests/test_reference_generate.py`.
- **STOP WHEN:** Greedy output matches the library for three prompts, termination works for EOS and max length, and token counts are correct.
- **Do Not Study Yet:** Beam search, speculative decoding, and model parallel generation.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 24: KV-cache layout and correctness CORE: Track 1 MINOR: Memory layout and per-token footprint analysis ¶

> **The Big Picture:** Prefill is the model reading the prompt; decode is the model writing one new token at a time. The KV cache is like keeping notes from the pages already read so the model does not reread the whole book for every word.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Understand cached keys/values, layer/head dimensions, append-per-token behavior, reuse of prompt history, and cache invalidation.
- **Secondary Focus [MINOR - 30% Weight]:** Memory layout and per-token footprint analysis. capacity estimates connect model math to real GPU limits.
- **Mapped Guides & Docs:** Hugging Face Transformers docs, **Model outputs**, URL above; study cache concepts only. Skip model-specific cache internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day24` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Implement a small tensor-level KV cache for a toy attention layer. Run cached and uncached generation with identical inputs and compare every token.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `src/inference_math/kv_cache_reference.py` and `tests/test_kv_cache_reference.py`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Memory layout and per-token footprint analysis: Write down units, dimensions, and one memory-footprint calculation beside the experiment.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `KV-cache layout and correctness` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not compare runs with different prompt/output lengths or forget the factor of two for keys and values. Record units and assumptions beside every number.

> **Daily boundary:** Cached and uncached outputs match for short and multi-step prompts, and the test catches an incorrect position or stale cache.

- **Tangible Deliverable:** `src/inference_math/kv_cache_reference.py` and `tests/test_kv_cache_reference.py`.
- **STOP WHEN:** Cached and uncached outputs match for short and multi-step prompts, and the test catches an incorrect position or stale cache.
- **Do Not Study Yet:** PagedAttention implementation, prefix caching, cache quantization, and engine source code.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 25: KV memory formula and capacity calculator CORE: Track 1

MINOR: Memory layout and per-token footprint analysis ¶

> **The Big Picture:** Prefill is the model reading the prompt; decode is the model writing one new token at a time. The KV cache is like keeping notes from the pages already read so the model does not reread the whole book for every word.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Calculate KV memory from layers, KV heads, head dimension, bytes per element, and tokens; estimate concurrent context capacity under a budget.
- **Secondary Focus [MINOR - 30% Weight]:** Memory layout and per-token footprint analysis. capacity estimates connect model math to real GPU limits.
- **Mapped Guides & Docs:** CUDA C Best Practices Guide, **Performance Metrics**, https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/. Study bytes moved and bandwidth concepts; skip architecture tuning.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day25` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Implement the formula `layers * 2 * KV_heads * head_dim * bytes_per_element * tokens`, include weights and safety margin, and generate a table for sequence length and concurrency.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `tools/kv_capacity.py`, `artifacts/day25-capacity.csv`, and `notes/day25-assumptions.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Memory layout and per-token footprint analysis: Write down units, dimensions, and one memory-footprint calculation beside the experiment.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `KV memory formula and capacity calculator` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not compare runs with different prompt/output lengths or forget the factor of two for keys and values. Record units and assumptions beside every number.

> **Daily boundary:** The calculator has unit tests, states every assumption, and predicts a small allocation experiment within an explicitly stated tolerance.

- **Tangible Deliverable:** `tools/kv_capacity.py`, `artifacts/day25-capacity.csv`, and `notes/day25-assumptions.md`.
- **STOP WHEN:** The calculator has unit tests, states every assumption, and predicts a small allocation experiment within an explicitly stated tolerance.
- **Do Not Study Yet:** Allocator fragmentation internals, paged block algorithms, NUMA, and multi-GPU memory placement.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 26: Cached versus uncached measurement CORE: Track 1

MINOR: Memory layout and per-token footprint analysis ¶

> **The Big Picture:** Prefill is the model reading the prompt; decode is the model writing one new token at a time. The KV cache is like keeping notes from the pages already read so the model does not reread the whole book for every word.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Compare correctness, TTFT, ITL, memory growth, and output length for cached and uncached paths.
- **Secondary Focus [MINOR - 30% Weight]:** Memory layout and per-token footprint analysis. capacity estimates connect model math to real GPU limits.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html. Study workload controls and reported metrics; skip engine-specific tuning for now.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day26` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Run fixed prompts at two prompt lengths and two output lengths, warm up, repeat at least five times, and collect timing/memory data for both paths.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day26-cache-comparison.csv`, chart, and `notes/day26-results.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Memory layout and per-token footprint analysis: Write down units, dimensions, and one memory-footprint calculation beside the experiment.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Cached versus uncached measurement` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for cached versus uncached measurement. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The report separates correctness from performance, uses repeated runs, and explains one result using the KV formula.

- **Tangible Deliverable:** `artifacts/day26-cache-comparison.csv`, chart, and `notes/day26-results.md`.
- **STOP WHEN:** The report separates correctness from performance, uses repeated runs, and explains one result using the KV formula.
- **Do Not Study Yet:** Continuous batching, paged attention, FlashAttention, and quantization.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 27: Week 4 capacity review CORE: Track 1 MINOR: Memory layout and per-token footprint analysis ¶

> **The Big Picture:** Prefill is the model reading the prompt; decode is the model writing one new token at a time. The KV cache is like keeping notes from the pages already read so the model does not reread the whole book for every word.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Retrieve prefill/decode, cache correctness, memory sizing, and capacity reasoning.
- **Secondary Focus [MINOR - 30% Weight]:** Memory layout and per-token footprint analysis. capacity estimates connect model math to real GPU limits.
- **Mapped Guides & Docs:** Revisit Hugging Face **Model outputs**, vLLM **Benchmarking**, and CUDA Best Practices **Performance Metrics**; no new reading.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day27` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Given a model configuration and GPU budget, calculate maximum safe context/concurrency, then validate one case experimentally. Repair a seeded off-by-one cache bug.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week04-report.md`, corrected cache test, and capacity worksheet..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Memory layout and per-token footprint analysis: Write down units, dimensions, and one memory-footprint calculation beside the experiment.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 4 capacity review` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not compare runs with different prompt/output lengths or forget the factor of two for keys and values. Record units and assumptions beside every number.

> **Daily boundary:** The cache test passes, calculation assumptions are explicit, and you can explain TTFT versus ITL without notes.

- **Tangible Deliverable:** `gates/week04-report.md`, corrected cache test, and capacity worksheet.

- **STOP WHEN:** The cache test passes, calculation assumptions are explicit, and you can explain TTFT versus ITL without notes.
- **Do Not Study Yet:** vLLM scheduler internals, paged KV source, multi-GPU, and speculative decoding.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 28: KV-capacity experiment and week gate preparation CORE: Track 1

MINOR: Memory layout and per-token footprint analysis ¶

> **The Big Picture:** Prefill is the model reading the prompt; decode is the model writing one new token at a time. The KV cache is like keeping notes from the pages already read so the model does not reread the whole book for every word.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Connect the capacity formula to measured allocation, safety margin, and a concrete admission limit.
- **Secondary Focus [MINOR - 30% Weight]:** Memory layout and per-token footprint analysis. capacity estimates connect model math to real GPU limits.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html. Study workload controls and repeated measurement; skip engine source.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day28` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Choose one model and GPU budget, predict the maximum safe sequence/concurrency pair, run a stepped allocation experiment, and record the first failure or safe ceiling.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day28-capacity-experiment.csv` and `notes/day28-capacity-decision.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Memory layout and per-token footprint analysis: Write down units, dimensions, and one memory-footprint calculation beside the experiment.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `KV-capacity experiment and week gate preparation` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not compare runs with different prompt/output lengths or forget the factor of two for keys and values. Record units and assumptions beside every number.

> **Daily boundary:** The predicted and observed ceiling are compared, the discrepancy is explained, and one safe operating limit is written for the next week.

- **Tangible Deliverable:** `artifacts/day28-capacity-experiment.csv` and `notes/day28-capacity-decision.md`.
- **STOP WHEN:** The predicted and observed ceiling are compared, the discrepancy is explained, and one safe operating limit is written for the next week.
- **Do Not Study Yet:** PagedAttention source, allocator tuning, and multi-GPU capacity planning.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 29: vLLM installation and OpenAI-compatible server CORE: Track 1

MINOR: Container logs and engine startup diagnosis ¶

> **The Big Picture:** An inference engine is the reusable machinery around a model. You are learning to operate one confidently, not to build every engine from scratch; the supporting logs and comparisons help you choose settings rationally.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Launch vLLM, select a model, set dtype and model limits, send a completion/chat request, and read startup logs.
- **Secondary Focus [MINOR - 30% Weight]:** Container logs and engine startup diagnosis. logs tell you why an otherwise correct server did not start.
- **Mapped Guides & Docs:** vLLM docs, **OpenAI-Compatible Server**, https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html. Study launch options, dtype, limits, streaming, and API; skip source modifications.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day29` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Launch the smallest suitable model, call it with curl and the typed client, save the command and logs, and verify deterministic greedy output.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `scripts/run_vllm.sh` or `.ps1`, `logs/day29-vllm-startup.txt`, and `notes/day29-baseline.md`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Container logs and engine startup diagnosis: Capture startup logs and one process/container status snapshot; label the symptom and likely layer.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `vLLM installation and OpenAI-compatible server` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for vllm installation and openai-compatible server. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The server starts from the documented command, returns a streamed response, and one startup failure is diagnosed from logs.

- **Tangible Deliverable:** `scripts/run_vllm.sh` or `.ps1`, `logs/day29-vllm-startup.txt`, and `notes/day29-baseline.md`.
- **STOP WHEN:** The server starts from the documented command, returns a streamed response, and one startup failure is diagnosed from logs.
- **Do Not Study Yet:** Scheduler source, custom model backends, and every vLLM flag.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 30: vLLM limits, dtype, and model loading CORE: Track 1 MINOR: Container logs and engine startup diagnosis ¶

> **The Big Picture:** An inference engine is the reusable machinery around a model. You are learning to operate one confidently, not to build every engine from scratch; the supporting logs and comparisons help you choose settings rationally.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Understand GPU memory utilization, max model length, max sequences/tokens, dtype, seed, and model cache behavior.
- **Secondary Focus [MINOR - 30% Weight]:** Container logs and engine startup diagnosis. logs tell you why an otherwise correct server did not start.
- **Mapped Guides & Docs:** vLLM docs, **OpenAI-Compatible Server**, URL above. Study only launch options affecting memory, correctness, and request behavior.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day30` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Run a controlled matrix changing one setting at a time. Record startup success, peak memory, output correctness, and failure message for an intentionally oversized limit.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day30-vllm-config.csv` and `notes/day30-config-findings.md`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Container logs and engine startup diagnosis: Capture startup logs and one process/container status snapshot; label the symptom and likely layer.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `vLLM limits, dtype, and model loading` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for vllm limits, dtype, and model loading. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** You can select safe settings for a stated GPU budget and explain one failure from a measured or logged cause.

- **Tangible Deliverable:** `artifacts/day30-vllm-config.csv` and `notes/day30-config-findings.md`.
- **STOP WHEN:** You can select safe settings for a stated GPU budget and explain one failure from a measured or logged cause.
- **Do Not Study Yet:** Custom schedulers, engine compilation, and source-level memory allocator changes.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 31: vLLM streaming and API compatibility CORE: Track 1 MINOR: Container logs and engine startup diagnosis ¶

> **The Big Picture:** An inference engine is the reusable machinery around a model. You are learning to operate one confidently, not to build every engine from scratch; the supporting logs and comparisons help you choose settings rationally.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Use streaming responses, request parameters, usage fields, error schemas, and client cancellation.
- **Secondary Focus [MINOR - 30% Weight]:** Container logs and engine startup diagnosis. logs tell you why an otherwise correct server did not start.
- **Mapped Guides & Docs:** vLLM docs, **OpenAI-Compatible Server**, URL above. Study streaming and request parameters; skip compatibility edge cases outside the chosen endpoint.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day31` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Add streaming to the typed client, capture first chunk and final chunk, cancel after a token threshold, and test malformed model/parameter requests.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `tests/test_vllm_streaming.py` and `logs/day31-streaming.jsonl`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Container logs and engine startup diagnosis: Capture startup logs and one process/container status snapshot; label the symptom and likely layer.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `vLLM streaming and API compatibility` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not confuse a cancelled client with a completed request. Check task state and server logs, then verify cleanup rather than trusting the HTTP status alone.

> **Daily boundary:** Normal, malformed, timeout, and cancellation cases produce classified results and no hanging client task.

- **Tangible Deliverable:** `tests/test_vllm_streaming.py` and `logs/day31-streaming.jsonl`.
- **STOP WHEN:** Normal, malformed, timeout, and cancellation cases produce classified results and no hanging client task.
- **Do Not Study Yet:** WebSocket protocol internals, OpenAI SDK internals, and multi-tenant gateway design.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 32: Triton comparative deployment CORE: Track 1 MINOR: Container logs and engine startup diagnosis ¶

> **The Big Picture:** An inference engine is the reusable machinery around a model. You are learning to operate one confidently, not to build every engine from scratch; the supporting logs and comparisons help you choose settings rationally.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Understand Triton server/model-repository concepts, model configuration, metrics, and how its deployment model differs from vLLM.
- **Secondary Focus [MINOR - 30% Weight]:** Container logs and engine startup diagnosis. logs tell you why an otherwise correct server did not start.
- **Mapped Guides & Docs:** NVIDIA Triton Inference Server, **User Guide**, https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/. Study server/model-repository concepts and metrics; skip custom backends.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day32` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Inspect or run a minimal Triton example if supported. Record model repository structure, endpoint shape, startup behavior, and one metric; compare it with the vLLM path.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `notes/day32-triton-comparison.md` and `artifacts/day32-engine-notes.json`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Container logs and engine startup diagnosis: Capture startup logs and one process/container status snapshot; label the symptom and likely layer.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Triton comparative deployment` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for triton comparative deployment. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** You can state when Triton is useful, what it does not provide automatically for LLM serving, and one evidence-backed reason to keep vLLM primary.

- **Tangible Deliverable:** `notes/day32-triton-comparison.md` and `artifacts/day32-engine-notes.json`.
- **STOP WHEN:** You can state when Triton is useful, what it does not provide automatically for LLM serving, and one evidence-backed reason to keep vLLM primary.
- **Do Not Study Yet:** Triton backend development, ensemble internals, and scheduler source.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 33: SGLang awareness and engine choice CORE: Track 1 MINOR: Container logs and engine startup diagnosis ¶

> **The Big Picture:** An inference engine is the reusable machinery around a model. You are learning to operate one confidently, not to build every engine from scratch; the supporting logs and comparisons help you choose settings rationally.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Compare vLLM and SGLang at the deployment/feature level without starting a framework collection project.
- **Secondary Focus [MINOR - 30% Weight]:** Container logs and engine startup diagnosis. logs tell you why an otherwise correct server did not start.
- **Mapped Guides & Docs:** SGLang docs, **Quick Start**, https://docs.sglang.ai/start/quickstart.html. Study deployment shape and intended use; skip internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day33` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Read the quick start, record model/API assumptions, and build an engine decision matrix across interactive serving, prefix reuse, compatibility, and operational complexity.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day33-engine-selection.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Container logs and engine startup diagnosis: Capture startup logs and one process/container status snapshot; label the symptom and likely layer.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `SGLang awareness and engine choice` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for sglang awareness and engine choice. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The matrix contains a primary choice, an alternative, evidence, unknowns, and a retest condition for the chosen workload.

- **Tangible Deliverable:** `artifacts/day33-engine-selection.md`.
- **STOP WHEN:** The matrix contains a primary choice, an alternative, evidence, unknowns, and a retest condition for the chosen workload.
- **Do Not Study Yet:** SGLang source, every benchmark claim, and serving-framework API memorization.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 34: vLLM configuration mini-lab CORE: Track 1 MINOR: Container logs and engine startup diagnosis ¶

> **The Big Picture:** An inference engine is the reusable machinery around a model. You are learning to operate one confidently, not to build every engine from scratch; the supporting logs and comparisons help you choose settings rationally.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Apply vLLM settings to one workload and observe memory, latency, throughput, and failure behavior.
- **Secondary Focus [MINOR - 30% Weight]:** Container logs and engine startup diagnosis. logs tell you why an otherwise correct server did not start.
- **Mapped Guides & Docs:** vLLM docs, **OpenAI-Compatible Server**, URL above. Study only the settings selected for the experiment.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day34` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Choose three configurations, keep model and workload fixed, warm up, run five repetitions at one concurrency, and record startup, output equality, peak memory, TTFT, and throughput.

- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day34-vllm-mini-lab.csv`, chart, and `notes/day34-choice.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Container logs and engine startup diagnosis: Capture startup logs and one process/container status snapshot; label the symptom and likely layer.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `vLLM configuration mini-lab` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for vllm configuration mini-lab. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Every configuration has a pass/fail result and the selected configuration meets the stated memory and correctness constraints.

- **Tangible Deliverable:** `artifacts/day34-vllm-mini-lab.csv`, chart, and `notes/day34-choice.md`.
- **STOP WHEN:** Every configuration has a pass/fail result and the selected configuration meets the stated memory and correctness constraints.
- **Do Not Study Yet:** Large-model fleet tuning, prefix caching, multi-GPU, and kernel tuning.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 35: Week 5 review and gate CORE: Track 1 MINOR: Container logs and engine startup diagnosis ¶

> **The Big Picture:** An inference engine is the reusable machinery around a model. You are learning to operate one confidently, not to build every engine from scratch; the supporting logs and comparisons help you choose settings rationally.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Demonstrate hands-on vLLM deployment and bounded comparative awareness of Triton and SGLang.
- **Secondary Focus [MINOR - 30% Weight]:** Container logs and engine startup diagnosis. logs tell you why an otherwise correct server did not start.
- **Mapped Guides & Docs:** Revisit vLLM **OpenAI-Compatible Server**, Triton **User Guide**, and SGLang **Quick Start**; no new reading.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day35` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Rebuild the server command, send a streaming request, intentionally exceed a configured limit, diagnose the failure, and defend the engine choice in five minutes.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week05-report.md`, configuration matrix, and corrected failure log..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Container logs and engine startup diagnosis: Capture startup logs and one process/container status snapshot; label the symptom and likely layer.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 5 review and gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the

symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for week 5 review and gate. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Clean start, correct response, controlled failure diagnosis, and engine choice defense all pass.

- **Tangible Deliverable:** `gates/week05-report.md`, configuration matrix, and corrected failure log.
- **STOP WHEN:** Clean start, correct response, controlled failure diagnosis, and engine choice defense all pass.
- **Do Not Study Yet:** vLLM source modifications, custom Triton backends, and SGLang internals.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 36: Benchmark harness contract CORE: Track 1 MINOR: Benchmark metadata and statistical hygiene ¶

> **The Big Picture:** A benchmark is an experiment, not a speed advertisement. Like timing a runner, you must warm up, repeat the run, control the course, and report slow as well as typical finishes.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Define benchmark inputs, warm-up, steady state, repetitions, metadata, output schema, and failure recording.
- **Secondary Focus [MINOR - 30% Weight]:** Benchmark metadata and statistical hygiene. good measurements need controlled conditions and honest summaries.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html. Study workload controls, warm-up, and reported metrics.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day36` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Extend the client into a CLI accepting model URL, prompt file, output limit, concurrency, repetitions, and seed. Write JSONL records with environment metadata.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `tools/bench_inference.py`, `schemas/benchmark_record.json`, and `README-benchmark.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Benchmark metadata and statistical hygiene: Record hardware, software, model, workload, warm-up, repetition count, and failures in the run manifest.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Benchmark harness contract` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not report one fast run as a performance result. Exclude warm-up, repeat the test, keep the workload fixed, and record failures and percentiles.

> **Daily boundary:** The harness can run five warm-up and five measured requests, records failures without aborting the batch, and produces valid JSONL.

- **Tangible Deliverable:** `tools/bench_inference.py`, `schemas/benchmark_record.json`, and `README-benchmark.md`.
- **STOP WHEN:** The harness can run five warm-up and five measured requests, records failures without aborting the batch, and produces valid JSONL.
- **Do Not Study Yet:** Distributed load generators, statistical significance testing, and benchmark-suite engineering.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 37: TTFT and ITL measurement CORE: Track 1 MINOR: Benchmark metadata and statistical hygiene ¶

> **The Big Picture:** A benchmark is an experiment, not a speed advertisement. Like timing a runner, you must warm up, repeat the run, control the course, and report slow as well as typical finishes.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Measure time to first token, inter-token latency, total latency, prompt tokens, output tokens, and tokens/sec.
- **Secondary Focus [MINOR - 30% Weight]:** Benchmark metadata and statistical hygiene. good measurements need controlled conditions and honest summaries.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, URL above. Study metric definitions and workload controls; skip unrelated benchmark modes.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day37` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Instrument streamed chunks, calculate TTFT, per-token deltas, mean ITL, total latency, and output tokens/sec. Validate calculations against a hand-computed three-token trace.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `src/inference_client/metrics.py`, unit tests, and `artifacts/day37-latency.jsonl`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Benchmark metadata and statistical hygiene: Record hardware, software, model, workload, warm-up, repetition count, and failures in the run manifest.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `TTFT and ITL measurement` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for ttft and itl measurement. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Hand calculation and program agree, warm-up is excluded, and missing/empty streams produce an explicit failure record.

- **Tangible Deliverable:** `src/inference_client/metrics.py`, unit tests, and `artifacts/day37-latency.jsonl`.
- **STOP WHEN:** Hand calculation and program agree, warm-up is excluded, and missing/empty streams produce an explicit failure record.
- **Do Not Study Yet:** Tail-latency theory, tracing systems, and kernel timestamps.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 38: Percentiles and failure rate CORE: Track 1 MINOR: Benchmark metadata and statistical hygiene ¶

> **The Big Picture:** A benchmark is an experiment, not a speed advertisement. Like timing a runner, you must warm up, repeat the run, control the course, and report slow as well as typical finishes.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Compute p50, p95, p99, throughput, failure rate, and sample count without hiding outliers.
- **Secondary Focus [MINOR - 30% Weight]:** Benchmark metadata and statistical hygiene. good measurements need controlled conditions and honest summaries.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, URL above. Study reported latency/throughput fields; use Python standard-library statistics as implementation support.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day38` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Aggregate the Day 37 JSONL, implement percentile calculation with a documented convention, separate successful and failed requests, and generate a summary table.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `tools/summarize_benchmark.py`, `artifacts/day38-summary.json`, and `notes/day38-percentiles.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Benchmark metadata and statistical hygiene: Record hardware, software, model, workload, warm-up, repetition count, and failures in the run manifest.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Percentiles and failure rate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not report one fast run as a performance result. Exclude warm-up, repeat the test, keep the workload fixed, and record failures and percentiles.

> **Daily boundary:** Summary includes p50/p95/p99, throughput, failure rate, count, and the percentile convention; a synthetic outlier test passes.

- **Tangible Deliverable:** `tools/summarize_benchmark.py`, `artifacts/day38-summary.json`, and `notes/day38-percentiles.md`.
- **STOP WHEN:** Summary includes p50/p95/p99, throughput, failure rate, count, and the percentile convention; a synthetic outlier test passes.
- **Do Not Study Yet:** Confidence intervals, A/B experiment design, and online telemetry aggregation.

- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 39: Concurrency sweep CORE: Track 1 MINOR: Benchmark metadata and statistical hygiene ¶

> **The Big Picture:** A benchmark is an experiment, not a speed advertisement. Like timing a runner, you must warm up, repeat the run, control the course, and report slow as well as typical finishes.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Observe how concurrency changes queueing, TTFT, ITL, throughput, utilization, and failures.
- **Secondary Focus [MINOR - 30% Weight]:** Benchmark metadata and statistical hygiene. good measurements need controlled conditions and honest summaries.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, URL above. Study workload controls; skip engine internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day39` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Run concurrency 1, 2, 4, and the highest safe value with fixed prompts/output limits. Keep five measured repetitions per setting and plot latency versus throughput.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day39-concurrency.csv`, plot, and `notes/day39-sweep.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Benchmark metadata and statistical hygiene: Record hardware, software, model, workload, warm-up, repetition count, and failures in the run manifest.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Concurrency sweep` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for concurrency sweep. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The sweep identifies a saturation or failure point and explains it using at least two measured signals.

- **Tangible Deliverable:** `artifacts/day39-concurrency.csv`, plot, and `notes/day39-sweep.md`.
- **STOP WHEN:** The sweep identifies a saturation or failure point and explains it using at least two measured signals.
- **Do Not Study Yet:** Autoscaling, queue scheduler source, and multi-node load generation.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 40: Benchmark reproducibility and metadata CORE: Track 1 MINOR: Benchmark metadata and statistical hygiene ¶

> **The Big Picture:** A benchmark is an experiment, not a speed advertisement. Like timing a runner, you must warm up, repeat the run, control the course, and report slow as well as typical finishes.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Control model, software, hardware, prompt lengths, output lengths, warm-up, repetition count, and failure policy.
- **Secondary Focus [MINOR - 30% Weight]:** Benchmark metadata and statistical hygiene. good measurements need controlled conditions and honest summaries.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, URL above. Study reproducibility controls; skip benchmark implementation details not used by the harness.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day40` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Add a metadata manifest containing GPU/CPU, driver/runtime, model revision, command, git commit, input distribution, and timestamp. Re-run after restarting the server and compare variance.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day40-run-manifest.json`, `artifacts/day40-repeat.csv`, and `notes/day40-variance.md`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Benchmark metadata and statistical hygiene: Record hardware, software, model, workload, warm-up, repetition count, and failures in the run manifest.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Benchmark reproducibility and metadata` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not report one fast run as a performance result. Exclude warm-up, repeat the test, keep the workload fixed, and record failures and percentiles.

> **Daily boundary:** Another shell can reproduce the run and the report distinguishes environmental variance from a real configuration change.

- **Tangible Deliverable:** `artifacts/day40-run-manifest.json`, `artifacts/day40-repeat.csv`, and `notes/day40-variance.md`.
- **STOP WHEN:** Another shell can reproduce the run and the report distinguishes environmental variance from a real configuration change.
- **Do Not Study Yet:** Distributed reproducibility, benchmark databases, and formal experimental design.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 41: Week 6 review and gate CORE: Track 1 MINOR: Benchmark metadata and statistical hygiene ¶

> **The Big Picture:** A benchmark is an experiment, not a speed advertisement. Like timing a runner, you must warm up, repeat the run, control the course, and report slow as well as typical finishes.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Defend the benchmark methodology and interpret a latency-throughput curve.
- **Secondary Focus [MINOR - 30% Weight]:** Benchmark metadata and statistical hygiene. good measurements need controlled conditions and honest summaries.
- **Mapped Guides & Docs:** Revisit vLLM **Benchmarking**; no new reading.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day41` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Run a clean benchmark, calculate metrics independently for one small sample, identify one outlier, and answer a five-minute review explaining warm-up, p95, ITL, and failure rate.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week06-report.md`, final JSON/CSV benchmark, and one chart..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Benchmark metadata and statistical hygiene: Record hardware, software, model, workload, warm-up, repetition count, and failures in the run manifest.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 6 review and gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for week 6 review and gate. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The benchmark is reproducible, metric calculations are correct, and the learner can reject an unfair benchmark claim.

- **Tangible Deliverable:** `gates/week06-report.md`, final JSON/CSV benchmark, and one chart.
- **STOP WHEN:** The benchmark is reproducible, metric calculations are correct, and the learner can reject an unfair benchmark claim.
- **Do Not Study Yet:** Roofline modeling, Nsight, quantization, and distributed benchmarking.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 42: Week 6 benchmark gate rerun CORE: Track 1 MINOR: Benchmark metadata and statistical hygiene ¶

> **The Big Picture:** A benchmark is an experiment, not a speed advertisement. Like timing a runner, you must warm up, repeat the run, control the course, and report slow as well as typical finishes.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Prove that the benchmark survives a clean restart and preserves its metric definitions and metadata.
- **Secondary Focus [MINOR - 30% Weight]:** Benchmark metadata and statistical hygiene. good measurements need controlled conditions and honest summaries.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html. Revisit warm-up, workload controls, and reported metrics; read no new material.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day42` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Stop and restart the server, rerun the selected concurrency sweep, compare manifests, and investigate one variance beyond the expected range.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week06-rerun.md`, `artifacts/day42-rerun.csv`, and a variance note..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Benchmark metadata and statistical hygiene: Record hardware, software, model, workload, warm-up, repetition count, and failures in the run manifest.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 6 benchmark gate rerun` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not report one fast run as a performance result. Exclude warm-up, repeat the test, keep the workload fixed, and record failures and percentiles.

> **Daily boundary:** The rerun is reproducible, variance is classified as expected or actionable, and the benchmark is ready for profiling.

- **Tangible Deliverable:** `gates/week06-rerun.md`, `artifacts/day42-rerun.csv`, and a variance note.
- **STOP WHEN:** The rerun is reproducible, variance is classified as expected or actionable, and the benchmark is ready for profiling.
- **Do Not Study Yet:** Automated tuning, distributed load generation, and profiler internals.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 43: Weight and KV memory accounting CORE: Track 1 MINOR: Hardware telemetry and timeline evidence ¶

> **The Big Picture:** Performance debugging is detective work. A GPU that looks idle may be waiting for the CPU, a copy, or a synchronization point, so the timeline is the evidence that tells you which suspect to test.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Estimate weight memory, KV memory, bytes moved, memory budget, and safety margin for a serving workload.
- **Secondary Focus [MINOR - 30% Weight]:** Hardware telemetry and timeline evidence. system tools turn a performance guess into a testable explanation.
- **Mapped Guides & Docs:** NVIDIA, **CUDA C Best Practices Guide: Performance Metrics**, https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/. Study bandwidth, arithmetic intensity, and measurement caveats; skip architecture tuning.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day43` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Extend the capacity calculator to include parameter count/dtype, KV cache, runtime overhead, and requested concurrency. Compare estimated and observed peak memory.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `tools/memory_budget.py`, `artifacts/day42-memory-budget.csv`, and `notes/day42-memory.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Hardware telemetry and timeline evidence: Capture the relevant CPU/GPU/telemetry observation and state what evidence would falsify the primary hypothesis.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Weight and KV memory accounting` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not compare runs with different prompt/output lengths or forget the factor of two for keys and values. Record units and assumptions beside every number.

> **Daily boundary:** The budget lists assumptions, predicts whether a workload fits, and explains any observed discrepancy using evidence.

- **Tangible Deliverable:** `tools/memory_budget.py`, `artifacts/day42-memory-budget.csv`, and `notes/day42-memory.md`.
- **STOP WHEN:** The budget lists assumptions, predicts whether a workload fits, and explains any observed discrepancy using evidence.
- **Do Not Study Yet:** Allocator internals, memory compression research, and GPU architecture reverse engineering.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 44: Arithmetic intensity and roofline hypothesis CORE: Track 1

MINOR: Hardware telemetry and timeline evidence ¶

> **The Big Picture:** Performance debugging is detective work. A GPU that looks idle may be waiting for the CPU, a copy, or a synchronization point, so the timeline is the evidence that tells you which suspect to test.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Understand FLOPs, bytes moved, arithmetic intensity, bandwidth ceiling, compute ceiling, and why a roofline is a hypothesis tool.
- **Secondary Focus [MINOR - 30% Weight]:** Hardware telemetry and timeline evidence. system tools turn a performance guess into a testable explanation.
- **Mapped Guides & Docs:** NVIDIA, **CUDA C Best Practices Guide: Performance Metrics**, URL above. Study bandwidth and arithmetic intensity; skip advanced architecture metrics.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day44` before changing code.

- [ ] **[CORE] Task 2: Core Implementation:** Estimate arithmetic intensity for one matrix/attention stage, record hardware bandwidth/compute specifications, and write two competing bottleneck hypotheses.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `notes/day43-roofline.md` and `artifacts/day43-roofline.csv`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Hardware telemetry and timeline evidence: Capture the relevant CPU/GPU/telemetry observation and state what evidence would falsify the primary hypothesis.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Arithmetic intensity and roofline hypothesis` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not report one fast run as a performance result. Exclude warm-up, repeat the test, keep the workload fixed, and record failures and percentiles.

> **Daily boundary:** Each hypothesis names a measurable signal and a condition that would falsify it; no claim is made from FLOPs alone.

- **Tangible Deliverable:** `notes/day43-roofline.md` and `artifacts/day43-roofline.csv`.
- **STOP WHEN:** Each hypothesis names a measurable signal and a condition that would falsify it; no claim is made from FLOPs alone.
- **Do Not Study Yet:** Formal roofline proof, occupancy calculators, and kernel-level optimization.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 45: Nsight Systems CLI capture CORE: Track 1 MINOR: Hardware telemetry and timeline evidence ¶

> **The Big Picture:** Performance debugging is detective work. A GPU that looks idle may be waiting for the CPU, a copy, or a synchronization point, so the timeline is the evidence that tells you which suspect to test.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Capture a timeline, identify CPU launch, host-device copy, synchronization, and GPU execution tracks.
- **Secondary Focus [MINOR - 30% Weight]:** Hardware telemetry and timeline evidence. system tools turn a performance guess into a testable explanation.
- **Mapped Guides & Docs:** NVIDIA, **Nsight Systems User Guide: Command Line Options**, https://docs.nvidia.com/nsight-systems/UserGuide/index.html. Study CLI capture and timeline tracks; skip deep kernel analysis.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day45` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Run an `nsys profile` capture around a representative PyTorch/vLLM request, save the report, and annotate one interval with request timestamps.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `profiles/day44-serving.nsys-rep` or documented fallback trace, `notes/day44-trace.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Hardware telemetry and timeline evidence: Capture the relevant CPU/GPU/telemetry observation and state what evidence would falsify the primary hypothesis.

- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Nsight Systems CLI capture` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not report one fast run as a performance result. Exclude warm-up, repeat the test, keep the workload fixed, and record failures and percentiles.

> **Daily boundary:** A trace opens, one request interval is correlated to CPU/GPU activity, and you can name one unknown requiring more instrumentation.

- **Tangible Deliverable:** `profiles/day44-serving.nsys-rep` or documented fallback trace, `notes/day44-trace.md`.
- **STOP WHEN:** A trace opens, one request interval is correlated to CPU/GPU activity, and you can name one unknown requiring more instrumentation.
- **Do Not Study Yet:** SASS, Nsight Compute, occupancy micro-optimization, and custom kernels.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 46: CPU launch stalls and host-device copies CORE: Track 1

MINOR: Hardware telemetry and timeline evidence ¶

> **The Big Picture:** Performance debugging is detective work. A GPU that looks idle may be waiting for the CPU, a copy, or a synchronization point, so the timeline is the evidence that tells you which suspect to test.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Distinguish CPU launch gaps, host-device transfer time, synchronization gaps, and actual GPU execution.
- **Secondary Focus [MINOR - 30% Weight]:** Hardware telemetry and timeline evidence. system tools turn a performance guess into a testable explanation.
- **Mapped Guides & Docs:** NVIDIA, **Nsight Systems User Guide: Command Line Options**, URL above; CUDA Best Practices **Performance Metrics**, URL above. Study only timeline and bandwidth concepts.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day46` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Capture a workload with a deliberately introduced synchronization or copy. Compare the trace with the normal path and calculate the changed interval.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `profiles/day45-before.nsys-rep`, `profiles/day45-after.nsys-rep`, and `notes/day45-stall-analysis.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Hardware telemetry and timeline evidence: Capture the relevant CPU/GPU/telemetry observation and state what evidence would falsify the primary hypothesis.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `CPU launch stalls and host-device copies` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for cpu launch stalls and host-device copies. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The report identifies the introduced cause from the trace and does not label all idle GPU time as a GPU kernel problem.

- **Tangible Deliverable:** `profiles/day45-before.nsys-rep`, `profiles/day45-after.nsys-rep`, and `notes/day45-stall-analysis.md`.
- **STOP WHEN:** The report identifies the introduced cause from the trace and does not label all idle GPU time as a GPU kernel problem.
- **Do Not Study Yet:** Stream programming, kernel fusion, and CUDA graph internals.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 47: One bounded tuning experiment CORE: Track 1 MINOR: Hardware telemetry and timeline evidence ¶

> **The Big Picture:** Performance debugging is detective work. A GPU that looks idle may be waiting for the CPU, a copy, or a synchronization point, so the timeline is the evidence that tells you which suspect to test.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Use a baseline, change one variable, remeasure, and decide whether the result is meaningful.
- **Secondary Focus [MINOR - 30% Weight]:** Hardware telemetry and timeline evidence. system tools turn a performance guess into a testable explanation.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html; NVIDIA Nsight Systems CLI guide, URL above. Study fair comparison and trace capture.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day47` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Select one safe variable such as concurrency, max batched tokens, dtype, or request length. Form a prediction, run baseline/change with five repetitions, capture a trace if useful, and report the result.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day46-tuning.csv`, `profiles/day46-tuning.nsys-rep` if applicable, and `notes/day46-experiment.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Hardware telemetry and timeline evidence: Capture the relevant CPU/GPU/telemetry observation and state what evidence would falsify the primary hypothesis.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `One bounded tuning experiment` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for one bounded tuning experiment. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The experiment changes one variable, includes correctness, reports latency/throughput/memory, and concludes improve/regress/no meaningful change.

- **Tangible Deliverable:** `artifacts/day46-tuning.csv`, `profiles/day46-tuning.nsys-rep` if applicable, and `notes/day46-experiment.md`.
- **STOP WHEN:** The experiment changes one variable, includes correctness, reports latency/throughput/memory, and concludes improve/regress/no meaningful change.
- **Do Not Study Yet:** Multi-variable search, auto-tuning, custom CUDA, and engine source changes.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 48: Second bounded tuning experiment CORE: Track 1 MINOR: Hardware telemetry and timeline evidence ¶

> **The Big Picture:** Performance debugging is detective work. A GPU that looks idle may be waiting for the CPU, a copy, or a synchronization point, so the timeline is the evidence that tells you which suspect to test.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Test whether the first tuning result generalizes across one workload dimension without turning the exercise into open-ended optimization.
- **Secondary Focus [MINOR - 30% Weight]:** Hardware telemetry and timeline evidence. system tools turn a performance guess into a testable explanation.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html; NVIDIA, **Nsight Systems User Guide: Command Line Options**, https://docs.nvidia.com/nsight-systems/UserGuide/index.html. Study fair comparison and one timeline capture.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day48` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Keep the successful or rejected change from Day 47, vary one input length or concurrency condition, rerun baseline/change with five repetitions, and compare the bottleneck signal.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day48-generalization.csv` and `notes/day48-tuning-boundary.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Hardware telemetry and timeline evidence: Capture the relevant CPU/GPU/telemetry observation and state what evidence would falsify the primary hypothesis.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Second bounded tuning experiment` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for second bounded tuning experiment. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** You can state whether the change is workload-specific, general, or inconclusive and have a measured reason to stop further tuning.

- **Tangible Deliverable:** `artifacts/day48-generalization.csv` and `notes/day48-tuning-boundary.md`.
- **STOP WHEN:** You can state whether the change is workload-specific, general, or inconclusive and have a measured reason to stop further tuning.
- **Do Not Study Yet:** Auto-tuners, kernel fusion, CUDA graphs, and profiler-driven rabbit holes.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 49: Week 7 review and gate CORE: Track 1 MINOR: Hardware telemetry and timeline evidence ¶

> **The Big Picture:** Performance debugging is detective work. A GPU that looks idle may be waiting for the CPU, a copy, or a synchronization point, so the timeline is the evidence that tells you which suspect to test.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Defend a bottleneck hypothesis using memory accounting, roofline reasoning, benchmark data, and a timeline.
- **Secondary Focus [MINOR - 30% Weight]:** Hardware telemetry and timeline evidence. system tools turn a performance guess into a testable explanation.
- **Mapped Guides & Docs:** Revisit CUDA Best Practices **Performance Metrics**, Nsight Systems CLI guide, and vLLM **Benchmarking**; no new reading.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day49` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Present the baseline, hypothesis, falsifying signal, trace, tuning change, and final decision in five minutes. Repair one intentionally misleading chart label.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week07-report.md`, final profile, and corrected chart..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Hardware telemetry and timeline evidence: Capture the relevant CPU/GPU/telemetry observation and state what evidence would falsify the primary hypothesis.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 7 review and gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for week 7 review and gate. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The hypothesis is supported or rejected by evidence and the learner can identify what was not measured.

- **Tangible Deliverable:** `gates/week07-report.md`, final profile, and corrected chart.
- **STOP WHEN:** The hypothesis is supported or rejected by evidence and the learner can identify what was not measured.
- **Do Not Study Yet:** Nsight Compute, CUDA kernel authoring, FlashAttention, and GPU microarchitecture research.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 50: FastAPI service skeleton and typed settings CORE: Track 1 MINOR: Signals, logs, and service lifecycle ¶

> **The Big Picture:** A service is more than a successful response. It must say whether it is ready, stream work, enforce deadlines, and leave no unfinished work behind when it shuts down.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Organize an HTTP service, typed settings, startup/shutdown hooks, request validation, and structured error responses.
- **Secondary Focus [MINOR - 30% Weight]:** Signals, logs, and service lifecycle. a useful endpoint must also stop, fail, and recover predictably.
- **Mapped Guides & Docs:** FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/. Study async endpoints, streaming, exceptions, and dependencies; skip framework internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day50` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Create `service/` with typed settings, `/healthz`, `/readyz`, and `/generate` endpoints. Add request IDs, validation, and a fake backend interface.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `service/app.py`, `service/settings.py`, and `tests/test_service_contract.py`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Signals, logs, and service lifecycle: Send a signal or lifecycle event, inspect structured logs, and verify that no task or process is stranded.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `FastAPI service skeleton and typed settings` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for fastapi service skeleton and typed settings. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Valid requests return a typed response, invalid requests return controlled errors, and health/readiness are distinct endpoints.

- **Tangible Deliverable:** `service/app.py`, `service/settings.py`, and `tests/test_service_contract.py`.
- **STOP WHEN:** Valid requests return a typed response, invalid requests return controlled errors, and health/readiness are distinct endpoints.
- **Do Not Study Yet:** Service mesh, GraphQL, framework internals, and authentication systems.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 51: Streaming endpoint and client cancellation CORE: Track 1 MINOR: Signals, logs, and service lifecycle ¶

> **The Big Picture:** A service is more than a successful response. It must say whether it is ready, stream work, enforce deadlines, and leave no unfinished work behind when it shuts down.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Stream tokens/chunks, handle slow clients, propagate cancellation, and close resources.
- **Secondary Focus [MINOR - 30% Weight]:** Signals, logs, and service lifecycle. a useful endpoint must also stop, fail, and recover predictably.
- **Mapped Guides & Docs:** FastAPI docs, **Concurrency and async / await**, URL above. Study async endpoints and streaming; skip dependency-injection internals.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day51` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Replace the fake single response with an async generator. Stream five chunks, cancel after two from the client, and test that the backend task is cancelled and logged.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `service/streaming.py`, `tests/test_streaming_cancel.py`, and `logs/day49-cancel.jsonl`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Signals, logs, and service lifecycle: Send a signal or lifecycle event, inspect structured logs, and verify that no task or process is stranded.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Streaming endpoint and client cancellation` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not confuse a cancelled client with a completed request. Check task state and server logs, then verify cleanup rather than trusting the HTTP status alone.

> **Daily boundary:** Normal stream, client cancellation, backend exception, and disconnect all terminate without a hanging task.

- **Tangible Deliverable:** `service/streaming.py`, `tests/test_streaming_cancel.py`, and `logs/day49-cancel.jsonl`.
- **STOP WHEN:** Normal stream, client cancellation, backend exception, and disconnect all terminate without a hanging task.
- **Do Not Study Yet:** WebSocket scaling, HTTP/2 flow-control internals, and distributed cancellation protocols.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 52: Deadlines, timeouts, and bounded admission CORE: Track 1

MINOR: Signals, logs, and service lifecycle ¶

> **The Big Picture:** A service is more than a successful response. It must say whether it is ready, stream work, enforce deadlines, and leave no unfinished work behind when it shuts down.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Apply request deadlines, reject excessive work, distinguish timeout from overload, and bound in-flight requests.
- **Secondary Focus [MINOR - 30% Weight]:** Signals, logs, and service lifecycle. a useful endpoint must also stop, fail, and recover predictably.
- **Mapped Guides & Docs:** FastAPI docs, **Concurrency and async / await**, URL above. Study async cancellation and exceptions; skip framework internals.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day52` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Add a semaphore or bounded queue, request deadline, `429`/`503` policy, and structured error codes. Test fast, slow, full-queue, and cancelled requests.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `service/admission.py`, `tests/test_admission.py`, and `artifacts/day50-admission.json`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Signals, logs, and service lifecycle: Send a signal or lifecycle event, inspect structured logs, and verify that no task or process is stranded.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Deadlines, timeouts, and bounded admission` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not confuse a cancelled client with a completed request. Check task state and server logs, then verify cleanup rather than trusting the HTTP status alone.

> **Daily boundary:** Admission never exceeds the configured bound, deadlines are enforced, and overload responses are deterministic and documented.

- **Tangible Deliverable:** `service/admission.py`, `tests/test_admission.py`, and `artifacts/day50-admission.json`.
- **STOP WHEN:** Admission never exceeds the configured bound, deadlines are enforced, and overload responses are deterministic and documented.
- **Do Not Study Yet:** Distributed queues, autoscaling controllers, and formal queueing proofs.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 53: Readiness, startup, and draining CORE: Track 1 MINOR: Signals, logs, and service lifecycle ¶

> **The Big Picture:** A service is more than a successful response. It must say whether it is ready, stream work, enforce deadlines, and leave no unfinished work behind when it shuts down.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Distinguish liveness/readiness, model loading state, graceful drain, in-flight completion, and safe shutdown.
- **Secondary Focus [MINOR - 30% Weight]:** Signals, logs, and service lifecycle. a useful endpoint must also stop, fail, and recover predictably.
- **Mapped Guides & Docs:** FastAPI docs, **Concurrency and async / await**, URL above. Study startup/shutdown behavior and exceptions; skip framework internals.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day53` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Add service states `starting`, `ready`, `draining`, and `stopped`. Reject new work during drain, allow accepted work to finish until a deadline, then exit with a report.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `service/lifecycle.py`, `tests/test_lifecycle.py`, and `logs/day51-drain.jsonl`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Signals, logs, and service lifecycle: Send a signal or lifecycle event, inspect structured logs, and verify that no task or process is stranded.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Readiness, startup, and draining` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not confuse a cancelled client with a completed request. Check task state and server logs, then verify cleanup rather than trusting the HTTP status alone.

> **Daily boundary:** Readiness changes correctly, new work is rejected during drain, accepted work completes or expires visibly, and process exit is clean.

- **Tangible Deliverable:** `service/lifecycle.py`, `tests/test_lifecycle.py`, and `logs/day51-drain.jsonl`.
- **STOP WHEN:** Readiness changes correctly, new work is rejected during drain, accepted work completes or expires visibly, and process exit is clean.
- **Do Not Study Yet:** Kubernetes probes/controllers, service mesh draining, and multi-replica rollout systems.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 54: Structured logs and request lifecycle CORE: Track 1 MINOR: Signals, logs, and service lifecycle ¶

> **The Big Picture:** A service is more than a successful response. It must say whether it is ready, stream work, enforce deadlines, and leave no unfinished work behind when it shuts down.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Log request ID, state transitions, timings, model/backend result, token counts, and error class without high-cardinality mistakes.
- **Secondary Focus [MINOR - 30% Weight]:** Signals, logs, and service lifecycle. a useful endpoint must also stop, fail, and recover predictably.
- **Mapped Guides & Docs:** FastAPI docs, **Concurrency and async / await**, URL above. Study exceptions and request handling; skip logging framework internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day54` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Add JSON logs for accepted, queued, started, first chunk, completed, cancelled, timed out, rejected, and failed states. Correlate one request across client and server logs.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `service/logging.py`, `logs/day52-request.jsonl`, and `notes/day52-log-schema.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Signals, logs, and service lifecycle: Send a signal or lifecycle event, inspect structured logs, and verify that no task or process is stranded.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Structured logs and request lifecycle` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for structured logs and request lifecycle. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** One request can be followed end-to-end and every terminal state has a reason, duration, and request ID.

- **Tangible Deliverable:** `service/logging.py`, `logs/day52-request.jsonl`, and `notes/day52-log-schema.md`.
- **STOP WHEN:** One request can be followed end-to-end and every terminal state has a reason, duration, and request ID.
- **Do Not Study Yet:** Distributed tracing backend deployment, log indexing internals, and PII-sensitive production policy design.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 55: Production lifecycle integration lab CORE: Track 1 MINOR: Signals, logs, and service lifecycle ¶

> **The Big Picture:** A service is more than a successful response. It must say whether it is ready, stream work, enforce deadlines, and leave no unfinished work behind when it shuts down.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Integrate streaming, admission, deadlines, readiness, draining, and structured logs into one service path.
- **Secondary Focus [MINOR - 30% Weight]:** Signals, logs, and service lifecycle. a useful endpoint must also stop, fail, and recover predictably.
- **Mapped Guides & Docs:** FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/. Study async endpoints, streaming, exceptions, and dependencies; skip framework internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day55` before changing code.

- [ ] **[CORE] Task 2: Core Implementation:** Connect the fake backend or vLLM client to the service, run a mixed workload, trigger timeout and cancellation, enter drain while work is active, and inspect the correlated logs.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day55-lifecycle-run.jsonl`, `notes/day55-integration.md`, and an end-to-end test..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Signals, logs, and service lifecycle: Send a signal or lifecycle event, inspect structured logs, and verify that no task or process is stranded.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Production lifecycle integration lab` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for production lifecycle integration lab. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The integrated service passes valid, malformed, timeout, cancellation, full-queue, readiness, and drain scenarios in one repeatable command.

- **Tangible Deliverable:** `artifacts/day55-lifecycle-run.jsonl`, `notes/day55-integration.md`, and an end-to-end test.
- **STOP WHEN:** The integrated service passes valid, malformed, timeout, cancellation, full-queue, readiness, and drain scenarios in one repeatable command.
- **Do Not Study Yet:** Prometheus exporters, quantization, autoscaling, and cloud deployment.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 56: Week 8 review and gate CORE: Track 1 MINOR: Signals, logs, and service lifecycle ¶

> **The Big Picture:** A service is more than a successful response. It must say whether it is ready, stream work, enforce deadlines, and leave no unfinished work behind when it shuts down.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Demonstrate a production-style HTTP lifecycle with streaming, failure handling, admission, readiness, and drain.
- **Secondary Focus [MINOR - 30% Weight]:** Signals, logs, and service lifecycle. a useful endpoint must also stop, fail, and recover predictably.
- **Mapped Guides & Docs:** Revisit FastAPI **Concurrency and async / await**; no new reading.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day56` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Run an automated scenario covering valid request, malformed request, slow backend, slow client, timeout, cancellation, full queue, readiness transition, and graceful drain.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week08-report.md`, test output, and a short lifecycle diagram..

- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Signals, logs, and service lifecycle: Send a signal or lifecycle event, inspect structured logs, and verify that no task or process is stranded.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 8 review and gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for week 8 review and gate. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** All scenarios have deterministic expected outcomes, logs identify the cause, and the service exits without stranded work.

- **Tangible Deliverable:** `gates/week08-report.md`, test output, and a short lifecycle diagram.
- **STOP WHEN:** All scenarios have deterministic expected outcomes, logs identify the cause, and the service exits without stranded work.
- **Do Not Study Yet:** Prometheus internals, quantization, autoscaling, and production cloud deployment.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 57: Metrics philosophy / Prometheus types CORE: Track 1 MINOR: Request IDs and operational log correlation ¶

> **The Big Picture:** Observability is the service speaking about itself. Counters tell you how much happened, gauges show what is happening now, and histograms show how a crowd of requests experienced latency.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Distinguish counters, gauges, histograms, labels, and cardinality; choose a metric type from behavior rather than convenience.
- **Secondary Focus [MINOR - 30% Weight]:** Request IDs and operational log correlation. metrics show where a problem is and logs help explain one request.
- **Mapped Guides & Docs:** Prometheus docs, **Metric types**, https://prometheus.io/docs/concepts/metric_types/ and **Instrumentation**, https://prometheus.io/docs/practices/instrumentation/. Study counters, gauges, histograms, labels, and cardinality; skip TSDB internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day57` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Inventory the service's request lifecycle and design metrics for request count, active requests, queue depth, errors, TTFT, ITL, output tokens, and GPU memory. Mark every label as bounded or unsafe.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `notes/day57-metric-contract.md` and `artifacts/day57-metric-catalog.csv`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Request IDs and operational log correlation: Use a bounded request ID in logs and show the exact query or grep command that finds one request.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Metrics philosophy / Prometheus types` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore

it, and record the symptom.

> **Beginner Trap to Avoid:** Do not put request IDs, prompts, or user data into metric labels. Keep labels bounded and use logs for per-request detail.

> **Daily boundary:** Every metric has a type, unit, label policy, and operational question; no metric uses request ID or prompt text as a label.

- **Tangible Deliverable:** `notes/day57-metric-contract.md` and `artifacts/day57-metric-catalog.csv`.
- **STOP WHEN:** Every metric has a type, unit, label policy, and operational question; no metric uses request ID or prompt text as a label.
- **Do Not Study Yet:** Prometheus storage internals, recording-rule architecture, OpenTelemetry collector internals, and custom metric backends.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 58: FastAPI metrics endpoint / request instrumentation CORE: Track 1

MINOR: Request IDs and operational log correlation ¶

> **The Big Picture:** Observability is the service speaking about itself. Counters tell you how much happened, gauges show what is happening now, and histograms show how a crowd of requests experienced latency.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Expose `/metrics`; instrument total requests, active requests, completed requests, rejected requests, and errors without changing service behavior.
- **Secondary Focus [MINOR - 30% Weight]:** Request IDs and operational log correlation. metrics show where a problem is and logs help explain one request.
- **Mapped Guides & Docs:** Prometheus docs, **Instrumentation**, https://prometheus.io/docs/practices/instrumentation/; FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/. Study instrumentation and async request handling; skip exporter internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day58` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Add a Prometheus client, counters, and an active-request gauge to the FastAPI service. Exercise success, validation failure, timeout, cancellation, and admission rejection.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `service/metrics.py`, updated `service/app.py`, `tests/test_metrics.py`, and `artifacts/day58-metrics.txt`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Request IDs and operational log correlation: Use a bounded request ID in logs and show the exact query or grep command that finds one request.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `FastAPI metrics endpoint / request instrumentation` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not put request IDs, prompts, or user data into metric labels. Keep labels bounded and use logs for per-request detail.

> **Daily boundary:** `/metrics` returns valid exposition text, every test scenario changes the intended metric, and the service still passes its Week 8 lifecycle tests.

- **Tangible Deliverable:** `service/metrics.py`, updated `service/app.py`, `tests/test_metrics.py`, and `artifacts/day58-metrics.txt`.
- **STOP WHEN:** `/metrics` returns valid exposition text, every test scenario changes the intended metric, and the service still passes its Week 8 lifecycle tests.
- **Do Not Study Yet:** High-cardinality labels, custom collectors, distributed metric aggregation, and dashboard design.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 59: Latency histograms / TTFT and ITL CORE: Track 1 MINOR: Request IDs and operational log correlation ¶

> **The Big Picture:** Observability is the service speaking about itself. Counters tell you how much happened, gauges show what is happening now, and histograms show how a crowd of requests experienced latency.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Record TTFT and ITL as distributions; choose buckets that make interactive latency visible without excessive cardinality.
- **Secondary Focus [MINOR - 30% Weight]:** Request IDs and operational log correlation. metrics show where a problem is and logs help explain one request.
- **Mapped Guides & Docs:** Prometheus docs, **Metric types**, https://prometheus.io/docs/concepts/metric_types/ and **Instrumentation**, https://prometheus.io/docs/practices/instrumentation/. Study histograms and bucket selection; skip storage implementation.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day59` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Add TTFT and ITL histograms with bounded labels for model and outcome. Configure explicit latency buckets, run short and long requests, and inspect bucket counts.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `service/latency_metrics.py`, `tests/test_latency_metrics.py`, and `artifacts/day59-histograms.txt`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Request IDs and operational log correlation: Use a bounded request ID in logs and show the exact query or grep command that finds one request.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Latency histograms / TTFT and ITL` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not report one fast run as a performance result. Exclude warm-up, repeat the test, keep the workload fixed, and record failures and percentiles.

> **Daily boundary:** Five warm requests and five measured requests produce non-empty TTFT/ITL buckets, and you can explain the resolution and limitation of the chosen buckets.

- **Tangible Deliverable:** `service/latency_metrics.py`, `tests/test_latency_metrics.py`, and `artifacts/day59-histograms.txt`.
- **STOP WHEN:** Five warm requests and five measured requests produce non-empty TTFT/ITL buckets, and you can explain the resolution and limitation of the chosen buckets.
- **Do Not Study Yet:** Native histograms, percentile math inside the server, custom histogram implementations, and trace exemplars.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 60: Prometheus scrape integration CORE: Track 1 MINOR: Request IDs and operational log correlation ¶

> **The Big Picture:** Observability is the service speaking about itself. Counters tell you how much happened, gauges show what is happening now, and histograms show how a crowd of requests experienced latency.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Configure a local Prometheus scrape job, verify target health, and distinguish application metric failure from scrape failure.
- **Secondary Focus [MINOR - 30% Weight]:** Request IDs and operational log correlation. metrics show where a problem is and logs help explain one request.
- **Mapped Guides & Docs:** Prometheus docs, **Instrumentation**, https://prometheus.io/docs/practices/instrumentation/. Study scrape endpoint expectations and instrumentation workflow; skip Prometheus server internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day60` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Add a minimal Prometheus container/configuration that scrapes the FastAPI service. Verify target state, scrape interval, sample count, and behavior when the service is stopped.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `monitoring/prometheus.yml`, `docker-compose.monitoring.yml`, and `logs/day60-scrape-health.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Request IDs and operational log correlation: Use a bounded request ID in logs and show the exact query or grep command that finds one request.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Prometheus scrape integration` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not put request IDs, prompts, or user data into metric labels. Keep labels bounded and use logs for per-request detail.

> **Daily boundary:** The target is healthy while the service runs, becomes unhealthy when stopped, and recovers after restart with evidence from the Prometheus UI or API.

- **Tangible Deliverable:** `monitoring/prometheus.yml`, `docker-compose.monitoring.yml`, and `logs/day60-scrape-health.md`.
- **STOP WHEN:** The target is healthy while the service runs, becomes unhealthy when stopped, and recovers after restart with evidence from the Prometheus UI or API.
- **Do Not Study Yet:** Alertmanager routing, remote write, federated Prometheus, and production retention policy.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 61: Grafana panels and PromQL queries CORE: Track 1 MINOR: Request IDs and operational log correlation ¶

> **The Big Picture:** Observability is the service speaking about itself. Counters tell you how much happened, gauges show what is happening now, and histograms show how a crowd of requests experienced latency.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Query request rate, error rate, active requests, TTFT, ITL, and p50/p95/p99 approximations from histogram buckets.
- **Secondary Focus [MINOR - 30% Weight]:** Request IDs and operational log correlation. metrics show where a problem is and logs help explain one request.
- **Mapped Guides & Docs:** Prometheus docs, **Metric types**, https://prometheus.io/docs/concepts/metric_types/ and **Instrumentation**, https://prometheus.io/docs/practices/instrumentation/. Study histogram query conventions; skip Grafana plugin internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day61` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Create panels or a documented query set for request rate, error ratio, active requests, TTFT, ITL, and histogram quantiles. Run a controlled load change and capture the dashboard.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `monitoring/grafana-dashboard.json` or `monitoring/promql.md` and `artifacts/day61-dashboard.png`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Request IDs and operational log correlation: Use a bounded request ID in logs and show the exact query or grep command that finds one request.

- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Grafana panels and PromQL queries` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for grafana panels and promql queries. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Each panel has a title, unit, query, time range, and interpretation; a load increase visibly changes at least two panels.

- **Tangible Deliverable:** `monitoring/grafana-dashboard.json` or `monitoring/promql.md` and `artifacts/day61-dashboard.png`.
- **STOP WHEN:** Each panel has a title, unit, query, time range, and interpretation; a load increase visibly changes at least two panels.
- **Do Not Study Yet:** Dashboard-as-code platforms, alert templating, business analytics, and long-term capacity dashboards.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 62: Slow-request investigation / metrics to logs CORE: Track 1

MINOR: Request IDs and operational log correlation ¶

> **The Big Picture:** Observability is the service speaking about itself. Counters tell you how much happened, gauges show what is happening now, and histograms show how a crowd of requests experienced latency.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Correlate a latency spike with structured JSON logs using request IDs; separate queue, prefill, decode, and client delay where available.
- **Secondary Focus [MINOR - 30% Weight]:** Request IDs and operational log correlation. metrics show where a problem is and logs help explain one request.
- **Mapped Guides & Docs:** Prometheus docs, **Instrumentation**, https://prometheus.io/docs/practices/instrumentation/; FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/. Study metric/log correlation and request lifecycle; skip distributed tracing implementation.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day62` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Generate a deliberately slow request, query its latency interval, locate the matching request ID in JSON logs, and write the causal chain from admission to completion.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day62-slow-request.json`, `notes/day62-investigation.md`, and a captured query result..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Request IDs and operational log correlation: Use a bounded request ID in logs and show the exact query or grep command that finds one request.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Slow-request investigation / metrics to logs` (invalid input, missing dependency, timeout, or wrong limit), observe

the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not put request IDs, prompts, or user data into metric labels. Keep labels bounded and use logs for per-request detail.

> **Daily boundary:** The report identifies the request, metric symptom, log evidence, likely phase, and one next measurement; correlation does not rely on prompt text.

- **Tangible Deliverable:** `artifacts/day62-slow-request.json`, `notes/day62-investigation.md`, and a captured query result.
- **STOP WHEN:** The report identifies the request, metric symptom, log evidence, likely phase, and one next measurement; correlation does not rely on prompt text.
- **Do Not Study Yet:** Full tracing backends, log indexing clusters, anomaly detection, and root-cause automation.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 63: Week 9 review and gate CORE: Track 1 MINOR: Request IDs and operational log correlation ¶

> **The Big Picture:** Observability is the service speaking about itself. Counters tell you how much happened, gauges show what is happening now, and histograms show how a crowd of requests experienced latency.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Retrieve metric types, label safety, histogram interpretation, scrape health, PromQL, and slow-request diagnosis.
- **Secondary Focus [MINOR - 30% Weight]:** Request IDs and operational log correlation. metrics show where a problem is and logs help explain one request.
- **Mapped Guides & Docs:** Revisit Prometheus **Metric types** and **Instrumentation**, plus FastAPI **Concurrency and async / await**; no new reading.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day63` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Rebuild monitoring from the configuration, reproduce the dashboard, explain three queries without notes, and investigate one seeded latency spike using metrics and logs.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week09-report.md`, dashboard/query export, and investigation notes..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Request IDs and operational log correlation: Use a bounded request ID in logs and show the exact query or grep command that finds one request.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 9 review and gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for week 9 review and gate. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The dashboard rebuilds, target health is understood, queries are defended, and the seeded slow request is explained with evidence.

- **Tangible Deliverable:** `gates/week09-report.md`, dashboard/query export, and investigation notes.
- **STOP WHEN:** The dashboard rebuilds, target health is understood, queries are defended, and the seeded slow request is explained with evidence.
- **Do Not Study Yet:** Prometheus internals, distributed tracing, alerting platforms, and metrics federation.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 64: Quantization mathematics and mechanics CORE: Track 1 MINOR: Quality evaluation and reproducibility ¶

> **The Big Picture:** Quantization packs numbers into fewer bits, like using a smaller measuring cup. It can save memory and improve speed, but you must check that the model still gives acceptable answers.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Compare FP16, BF16, FP8, INT8, and INT4; understand scale, zero point, rounding, range, weight-only, and activation quantization.
- **Secondary Focus [MINOR - 30% Weight]:** Quality evaluation and reproducibility. a faster model is not useful if its answers fail the quality gate.
- **Mapped Guides & Docs:** vLLM docs, **Quantization**, https://docs.vllm.ai/en/latest/features/quantization/. Study supported AWQ/GPTQ/FP8 modes and limitations; skip quantization algorithm research.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day64` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Quantize a small tensor using affine scale/zero-point equations, dequantize it, measure absolute error, and record dtype byte sizes and expected memory impact.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `tools/quantization_math.py`, `tests/test_quantization_math.py`, and `notes/day64-precision.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Quality evaluation and reproducibility: Run the same evaluation with fixed settings and save the baseline output before changing precision.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Quantization mathematics and mechanics` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not call a quantized model better from memory or speed alone. Run the same quality set and keep incompatible-model failures visible.

> **Daily boundary:** The tensor test passes, scale/zero-point assumptions are stated, and you can explain why lower bytes do not guarantee lower end-to-end latency.

- **Tangible Deliverable:** `tools/quantization_math.py`, `tests/test_quantization_math.py`, and `notes/day64-precision.md`.
- **STOP WHEN:** The tensor test passes, scale/zero-point assumptions are stated, and you can explain why lower bytes do not guarantee lower end-to-end latency.
- **Do Not Study Yet:** Quantization-aware training, novel calibration algorithms, custom INT4 kernels, and hardware code generation.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 65: AWQ deployment in vLLM CORE: Track 1 MINOR: Quality evaluation and reproducibility ¶

> **The Big Picture:** Quantization packs numbers into fewer bits, like using a smaller measuring cup. It can save memory and improve speed, but you must check that the model still gives acceptable answers.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Load a supported AWQ model, understand quantized model flags, verify startup and dtype behavior, and measure memory savings.
- **Secondary Focus [MINOR - 30% Weight]:** Quality evaluation and reproducibility. a faster model is not useful if its answers fail the quality gate.
- **Mapped Guides & Docs:** vLLM docs, **Quantization**, https://docs.vllm.ai/en/latest/features/quantization/ and **OpenAI-Compatible Server**, https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html. Study AWQ support, limitations, launch options, and streaming; skip source modifications.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day65` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Deploy an AWQ model or documented substitute, run fixed prompts against the unquantized baseline, capture startup logs, peak memory, output tokens, and errors.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `scripts/run_vllm_awq.sh` or `.ps1`, `artifacts/day65-awq.csv`, and `logs/day65-awq-startup.txt`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Quality evaluation and reproducibility: Run the same evaluation with fixed settings and save the baseline output before changing precision.

- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `AWQ deployment in vLLM` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not call a quantized model better from memory or speed alone. Run the same quality set and keep incompatible-model failures visible.

> **Daily boundary:** The quantized model loads, produces valid output, memory is measured under the same workload, and any unsupported-model limitation is recorded.

- **Tangible Deliverable:** `scripts/run_vllm_awq.sh` or `.ps1`, `artifacts/day65-awq.csv`, and `logs/day65-awq-startup.txt`.
- **STOP WHEN:** The quantized model loads, produces valid output, memory is measured under the same workload, and any unsupported-model limitation is recorded.
- **Do Not Study Yet:** AWQ implementation internals, calibration dataset construction, and custom quantized kernels.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 66: GPTQ or FP8 comparison CORE: Track 1 MINOR: Quality evaluation and reproducibility ¶

> **The Big Picture:** Quantization packs numbers into fewer bits, like using a smaller measuring cup. It can save memory and improve speed, but you must check that the model still gives acceptable answers.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Compare a second supported precision path with AWQ on startup, memory, correctness, and throughput.
- **Secondary Focus [MINOR - 30% Weight]:** Quality evaluation and reproducibility. a faster model is not useful if its answers fail the quality gate.
- **Mapped Guides & Docs:** vLLM docs, **Quantization**, https://docs.vllm.ai/en/latest/features/quantization/. Study GPTQ/FP8 support and limitations; skip training-aware quantization.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day66` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Deploy GPTQ or FP8 where hardware/model support exists; otherwise use a documented fallback comparison. Keep model, prompts, concurrency, warm-up, and repetitions fixed.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day66-precision-comparison.csv` and `notes/day66-runtime-findings.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Quality evaluation and reproducibility: Run the same evaluation with fixed settings and save the baseline output before changing precision.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `GPTQ or FP8 comparison` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not call a quantized model better from memory or speed alone. Run the same quality set and keep incompatible-model failures visible.

> **Daily boundary:** AWQ and the second path have comparable correctness, memory, startup, and throughput results or a precise incompatibility report.

- **Tangible Deliverable:** `artifacts/day66-precision-comparison.csv` and `notes/day66-runtime-findings.md`.
- **STOP WHEN:** AWQ and the second path have comparable correctness, memory, startup, and throughput results or a precise incompatibility report.
- **Do Not Study Yet:** Kernel codegen, calibration research, and exhaustive precision-format surveys.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 67: Quality evaluation set CORE: Track 1 MINOR: Quality evaluation and reproducibility ¶

> **The Big Picture:** Quantization packs numbers into fewer bits, like using a smaller measuring cup. It can save memory and improve speed, but you must check that the model still gives acceptable answers.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Build a fixed golden prompt suite; distinguish exact-match, task-specific accuracy, output validity, and perplexity as different quality signals.
- **Secondary Focus [MINOR - 30% Weight]:** Quality evaluation and reproducibility. a faster model is not useful if its answers fail the quality gate.
- **Mapped Guides & Docs:** Hugging Face Transformers docs, **Model outputs**, https://huggingface.co/docs/transformers/main/en/main_classes/output. Study logits and generated outputs; skip fine-tuning.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day67` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Create prompts covering normal generation, structured output, refusal/edge input, short context, and longer context. Add expected properties and an evaluator script with deterministic decoding.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `evaluation/prompts.jsonl`, `evaluation/evaluate.py`, and `notes/day67-quality-contract.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Quality evaluation and reproducibility: Run the same evaluation with fixed settings and save the baseline output before changing precision.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Quality evaluation set` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not call a quantized model better from memory or speed alone. Run the same quality set and keep incompatible-model failures visible.

> **Daily boundary:** The suite runs from a clean command, produces stable baseline results, and every test states what quality property it checks.

- **Tangible Deliverable:** `evaluation/prompts.jsonl`, `evaluation/evaluate.py`, and `notes/day67-quality-contract.md`.
- **STOP WHEN:** The suite runs from a clean command, produces stable baseline results, and every test states what quality property it checks.
- **Do Not Study Yet:** Benchmark dataset curation, human preference modeling, RLHF, and broad model evaluation research.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 68: Quality regression measurement CORE: Track 1 MINOR: Quality evaluation and reproducibility ¶

> **The Big Picture:** Quantization packs numbers into fewer bits, like using a smaller measuring cup. It can save memory and improve speed, but you must check that the model still gives acceptable answers.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Run the quality set on unquantized and quantized variants; distinguish real regression from nondeterministic sampling or infrastructure failure.
- **Secondary Focus [MINOR - 30% Weight]:** Quality evaluation and reproducibility. a faster model is not useful if its answers fail the quality gate.
- **Mapped Guides & Docs:** vLLM docs, **Quantization**, https://docs.vllm.ai/en/latest/features/quantization/; Hugging Face **Model outputs**, https://huggingface.co/docs/transformers/main/en/main_classes/output. Study supported quantization and output comparison; skip fine-tuning.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day68` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Run greedy evaluation on baseline and AWQ/FP8, record exact outputs and task scores, rerun one changed output, and classify the cause.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day68-quality-results.json` and `notes/day68-regression.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Quality evaluation and reproducibility: Run the same evaluation with fixed settings and save the baseline output before changing precision.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Quality regression measurement` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not call a quantized model better from memory or speed alone. Run the same quality set and keep incompatible-model failures visible.

> **Daily boundary:** The report contains baseline, quantized, difference, tolerance, and cause for every changed case; no quality claim is based on one prompt.

- **Tangible Deliverable:** `artifacts/day68-quality-results.json` and `notes/day68-regression.md`.
- **STOP WHEN:** The report contains baseline, quantized, difference, tolerance, and cause for every changed case; no quality claim is based on one prompt.
- **Do Not Study Yet:** Perplexity model training, statistical preference studies, and manual optimization of prompts to hide regressions.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 69: Quantization Pareto frontier CORE: Track 1 MINOR: Quality evaluation and reproducibility ¶

> **The Big Picture:** Quantization packs numbers into fewer bits, like using a smaller measuring cup. It can save memory and improve speed, but you must check that the model still gives acceptable answers.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Combine quality, peak memory, TTFT, ITL, tokens/sec, startup, and failure data into a decision rather than selecting the fastest number.
- **Secondary Focus [MINOR - 30% Weight]:** Quality evaluation and reproducibility. a faster model is not useful if its answers fail the quality gate.
- **Mapped Guides & Docs:** vLLM docs, **Quantization**, https://docs.vllm.ai/en/latest/features/quantization/ and **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html. Study support, limitations, workload controls, and metrics.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day69` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Join benchmark and quality artifacts, generate a table and plot, mark dominated configurations, and select one configuration for the target SLO.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day69-pareto.csv`, `artifacts/day69-pareto.png`, and `notes/day69-decision.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Quality evaluation and reproducibility: Run the same evaluation with fixed settings and save the baseline output before changing precision.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Quantization Pareto frontier` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not call a quantized model better from memory or speed alone. Run the same quality set and keep incompatible-model failures visible.

> **Daily boundary:** The selected point meets explicit quality and memory constraints, and the report states why faster or smaller alternatives were rejected.

- **Tangible Deliverable:** `artifacts/day69-pareto.csv`, `artifacts/day69-pareto.png`, and `notes/day69-decision.md`.
- **STOP WHEN:** The selected point meets explicit quality and memory constraints, and the report states why faster or smaller alternatives were rejected.

- **Do Not Study Yet:** Automated multi-objective optimization, custom quantization algorithms, and fleet-wide cost modeling.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 70: Week 10 review and gate CORE: Track 1 MINOR: Quality evaluation and reproducibility ¶

> **The Big Picture:** Quantization packs numbers into fewer bits, like using a smaller measuring cup. It can save memory and improve speed, but you must check that the model still gives acceptable answers.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Defend precision choices with empirical quality/performance evidence and reject unsupported performance claims.
- **Secondary Focus [MINOR - 30% Weight]:** Quality evaluation and reproducibility. a faster model is not useful if its answers fail the quality gate.
- **Mapped Guides & Docs:** Revisit vLLM **Quantization**, **OpenAI-Compatible Server**, **Benchmarking**, and Hugging Face **Model outputs**; no new reading.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day70` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Reproduce the Pareto report from raw artifacts, answer a closed-book precision quiz, and defend one configuration while a reviewer challenges a missing quality measurement.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week10-report.md`, final Pareto plot, and corrected evidence table..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Quality evaluation and reproducibility: Run the same evaluation with fixed settings and save the baseline output before changing precision.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 10 review and gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for week 10 review and gate. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The selected deployment is reproducible, quality-gated, and every speed/memory claim has matching workload metadata.

- **Tangible Deliverable:** `gates/week10-report.md`, final Pareto plot, and corrected evidence table.
- **STOP WHEN:** The selected deployment is reproducible, quality-gated, and every speed/memory claim has matching workload metadata.
- **Do Not Study Yet:** Quantization-aware training, research papers, custom kernels, and model architecture changes.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 71: Batching mechanics and token budgets CORE: Track 1 MINOR: Queue metrics and overload behavior ¶

> **The Big Picture:** Batching is a shared bus for requests. Sharing the GPU can improve throughput, but long or numerous passengers can make everyone wait, so queue limits and fairness matter.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Distinguish static batching from continuous iteration-level batching; understand max sequences, token budgets, queue time, and goodput.
- **Secondary Focus [MINOR - 30% Weight]:** Queue metrics and overload behavior. the supporting measurements reveal when waiting, not execution, causes latency.
- **Mapped Guides & Docs:** vLLM docs, **OpenAI-Compatible Server**, https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html. Study launch options affecting request limits and streaming; skip scheduler source.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day71` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Draw a request timeline for static and continuous batching, then simulate two short and two long requests with a fixed token budget.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `notes/day71-batching.md` and `tools/batching_sim.py`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Queue metrics and overload behavior: Record queue depth, wait time, status code, and p95 while changing arrival rate or concurrency.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Batching mechanics and token budgets` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not raise concurrency until throughput looks larger. Watch queue time, p95/p99, rejection rate, and fairness together.

> **Daily boundary:** The simulation shows why continuous batching can improve utilization while affecting tail latency, and all token/accounting assumptions are explicit.

- **Tangible Deliverable:** `notes/day71-batching.md` and `tools/batching_sim.py`.
- **STOP WHEN:** The simulation shows why continuous batching can improve utilization while affecting tail latency, and all token/accounting assumptions are explicit.
- **Do Not Study Yet:** Optimal scheduling proofs, paged-attention source, and speculative decoding.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 72: vLLM scheduling flags CORE: Track 1 MINOR: Queue metrics and overload behavior ¶

> **The Big Picture:** Batching is a shared bus for requests. Sharing the GPU can improve throughput, but long or numerous passengers can make everyone wait, so queue limits and fairness matter.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Configure `--max-num-seqs`, `--max-num-batched-tokens`, memory utilization, and max model length; understand their interaction.
- **Secondary Focus [MINOR - 30% Weight]:** Queue metrics and overload behavior. the supporting measurements reveal when waiting, not execution, causes latency.
- **Mapped Guides & Docs:** vLLM docs, **OpenAI-Compatible Server**, https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html. Study launch options and limits; skip source modifications.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day72` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Run a one-variable-at-a-time configuration matrix, capture startup, peak memory, throughput, TTFT, ITL, and OOM/limit failures.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day72-scheduler-flags.csv` and `notes/day72-settings.md`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Queue metrics and overload behavior: Record queue depth, wait time, status code, and p95 while changing arrival rate or concurrency.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `vLLM scheduling flags` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for vllm scheduling flags. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Each flag has an observed effect or documented hardware limitation, and one safe starting configuration is chosen.

- **Tangible Deliverable:** `artifacts/day72-scheduler-flags.csv` and `notes/day72-settings.md`.
- **STOP WHEN:** Each flag has an observed effect or documented hardware limitation, and one safe starting configuration is chosen.
- **Do Not Study Yet:** Scheduler implementation, block allocation internals, and multi-GPU scheduling.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 73: Queue depth and wait time CORE: Track 1 MINOR: Queue metrics and overload behavior ¶

> **The Big Picture:** Batching is a shared bus for requests. Sharing the GPU can improve throughput, but long or numerous passengers can make everyone wait, so queue limits and fairness matter.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Separate queue latency from execution latency and observe behavior as arrival rate approaches service capacity.
- **Secondary Focus [MINOR - 30% Weight]:** Queue metrics and overload behavior. the supporting measurements reveal when waiting, not execution, causes latency.
- **Mapped Guides & Docs:** vLLM docs, **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html and **Benchmarking**,

https://docs.vllm.ai/en/latest/performance/benchmarking.html. Study request/token/queue metrics and workload controls; skip collector internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day73` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Send increasing request arrival rates, collect queue metrics, client timings, TTFT, throughput, and failures, then chart queue time versus execution time.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day73-queue-sweep.csv`, chart, and `notes/day73-queue-analysis.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Queue metrics and overload behavior: Record queue depth, wait time, status code, and p95 while changing arrival rate or concurrency.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Queue depth and wait time` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not raise concurrency until throughput looks larger. Watch queue time, p95/p99, rejection rate, and fairness together.

> **Daily boundary:** The report identifies the arrival rate where queueing dominates and separates client, queue, and execution contributions.

- **Tangible Deliverable:** `artifacts/day73-queue-sweep.csv`, chart, and `notes/day73-queue-analysis.md`.
- **STOP WHEN:** The report identifies the arrival rate where queueing dominates and separates client, queue, and execution contributions.
- **Do Not Study Yet:** Queueing-theory proofs, distributed queues, and autoscaling policy design.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 74: Backpressure and load shedding CORE: Track 1 MINOR: Queue metrics and overload behavior ¶

> **The Big Picture:** Batching is a shared bus for requests. Sharing the GPU can improve throughput, but long or numerous passengers can make everyone wait, so queue limits and fairness matter.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Bound queue/in-flight work, return deterministic HTTP 429/503 under saturation, and protect latency for accepted work.
- **Secondary Focus [MINOR - 30% Weight]:** Queue metrics and overload behavior. the supporting measurements reveal when waiting, not execution, causes latency.
- **Mapped Guides & Docs:** FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/; vLLM **OpenAI-Compatible Server**, https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html. Study async admission and endpoint behavior; skip framework internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day74` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Add a bounded admission layer before the engine, configure rejection policy, drive saturation, and verify accepted/rejected counts and response codes.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `service/backpressure.py`, `tests/test_backpressure.py`, and `artifacts/day74-overload.json`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Queue metrics and overload behavior: Record queue depth, wait time, status code, and p95 while changing arrival rate or concurrency.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Backpressure and load shedding` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not raise concurrency until throughput looks larger. Watch queue time, p95/p99, rejection rate, and fairness together.

> **Daily boundary:** Queue bounds hold, rejected requests are deterministic, accepted request p95 remains within the chosen safety target, and shutdown remains clean.

- **Tangible Deliverable:** `service/backpressure.py`, `tests/test_backpressure.py`, and `artifacts/day74-overload.json`.
- **STOP WHEN:** Queue bounds hold, rejected requests are deterministic, accepted request p95 remains within the chosen safety target, and shutdown remains clean.
- **Do Not Study Yet:** Distributed load shedding, adaptive controllers, and service-mesh retries.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 75: Heterogeneous workload stress test CORE: Track 1 MINOR: Queue metrics and overload behavior ¶

> **The Big Picture:** Batching is a shared bus for requests. Sharing the GPU can improve throughput, but long or numerous passengers can make everyone wait, so queue limits and fairness matter.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Analyze short-prompt/long-output and long-prompt/short-output mixtures, fairness, tail latency, and starvation risk.
- **Secondary Focus [MINOR - 30% Weight]:** Queue metrics and overload behavior. the supporting measurements reveal when waiting, not execution, causes latency.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html and **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html. Study workload controls and queue/token metrics.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day75` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Build a mixed workload generator, run at fixed arrival rates, label request classes with bounded metadata, and compare p50/p95/p99 TTFT/ITL and completion rate.

- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day75-mixed-workload.csv`, charts, and `notes/day75-tail-latency.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Queue metrics and overload behavior: Record queue depth, wait time, status code, and p95 while changing arrival rate or concurrency.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Heterogeneous workload stress test` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for heterogeneous workload stress test. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The report identifies the worst class, a likely scheduling cause, and one bounded configuration change to test.

- **Tangible Deliverable:** `artifacts/day75-mixed-workload.csv`, charts, and `notes/day75-tail-latency.md`.
- **STOP WHEN:** The report identifies the worst class, a likely scheduling cause, and one bounded configuration change to test.
- **Do Not Study Yet:** Fair scheduling research, priority schedulers, and engine source modification.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 76: Tune for a target p95 SLO CORE: Track 1 MINOR: Queue metrics and overload behavior ¶

> **The Big Picture:** Batching is a shared bus for requests. Sharing the GPU can improve throughput, but long or numerous passengers can make everyone wait, so queue limits and fairness matter.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Find a concurrency/configuration limit that holds p95 TTFT below a stated threshold while preserving useful throughput.
- **Secondary Focus [MINOR - 30% Weight]:** Queue metrics and overload behavior. the supporting measurements reveal when waiting, not execution, causes latency.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html and **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html. Study controlled benchmarks and queue/cache metrics.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day76` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Define an interactive SLO, sweep concurrency and token limits, reject unsafe points, and select the highest goodput configuration that passes p95 and failure constraints.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day76-slo-tuning.csv`, `notes/day76-slo-decision.md`, and configuration file..

- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Queue metrics and overload behavior: Record queue depth, wait time, status code, and p95 while changing arrival rate or concurrency.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Tune for a target p95 SLO` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not raise concurrency until throughput looks larger. Watch queue time, p95/p99, rejection rate, and fairness together.

> **Daily boundary:** The selected point passes five measured repetitions, the nearest failing point is documented, and goodput versus throughput is explained.

- **Tangible Deliverable:** `artifacts/day76-slo-tuning.csv`, `notes/day76-slo-decision.md`, and configuration file.
- **STOP WHEN:** The selected point passes five measured repetitions, the nearest failing point is documented, and goodput versus throughput is explained.
- **Do Not Study Yet:** Autoscaling, multi-replica routing, and reinforcement-based tuning.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 77: Week 11 review and gate CORE: Track 1 MINOR: Queue metrics and overload behavior ¶

> **The Big Picture:** Batching is a shared bus for requests. Sharing the GPU can improve throughput, but long or numerous passengers can make everyone wait, so queue limits and fairness matter.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Defend batching, scheduling flags, queue measurements, backpressure, goodput, and starvation analysis.
- **Secondary Focus [MINOR - 30% Weight]:** Queue metrics and overload behavior. the supporting measurements reveal when waiting, not execution, causes latency.
- **Mapped Guides & Docs:** Revisit vLLM **OpenAI-Compatible Server**, **Production Metrics**, **Benchmarking**, and FastAPI **Concurrency and async / await**; no new reading.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day77` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Reproduce the SLO configuration, explain a p99 outlier, demonstrate saturation and deterministic shedding, and answer a closed-book batching quiz.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week11-report.md`, configuration matrix, and overload evidence..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Queue metrics and overload behavior: Record queue depth, wait time, status code, and p95 while changing arrival rate or concurrency.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 11 review and gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for week 11 review and gate. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The target SLO is met, overload is safe, the scheduling tradeoff is defensible, and no starvation claim lacks workload evidence.

- **Tangible Deliverable:** `gates/week11-report.md`, configuration matrix, and overload evidence.
- **STOP WHEN:** The target SLO is met, overload is safe, the scheduling tradeoff is defensible, and no starvation claim lacks workload evidence.
- **Do Not Study Yet:** PagedAttention source, multi-GPU parallelism, and advanced scheduler research.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 78: Serving failure taxonomy CORE: Track 1 MINOR: Evidence collection and incident writing ¶

> **The Big Picture:** Chaos testing is a controlled fire drill. You break one safe part of the service, observe the signals, recover it, and record what would help an operator respond faster next time.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Classify host OOM, GPU allocation failure, process crash, network partition/delay, slow client, disk/cache failure, and bad input.
- **Secondary Focus [MINOR - 30% Weight]:** Evidence collection and incident writing. a failure becomes a learning artifact when its cause and recovery are recorded.
- **Mapped Guides & Docs:** vLLM docs, **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html; FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/. Study observable request and service failure behavior; skip recovery internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day78` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Create a failure matrix with trigger, symptom, metric, log evidence, user impact, immediate mitigation, and prevention for each class.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `notes/day78-failure-taxonomy.md` and `artifacts/day78-failure-matrix.csv`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Evidence collection and incident writing: Preserve logs, metrics, and timestamps before recovery; write one sentence describing the user impact.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Serving failure taxonomy` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Inject faults only in the isolated test stack. Preserve logs before restarting, and never claim recovery without a successful post-fault smoke request.

> **Daily boundary:** Every failure has a safe injection method and a measurable recovery criterion; no test plan requires destructive host changes.

- **Tangible Deliverable:** `notes/day78-failure-taxonomy.md` and `artifacts/day78-failure-matrix.csv`.
- **STOP WHEN:** Every failure has a safe injection method and a measurable recovery criterion; no test plan requires destructive host changes.
- **Do Not Study Yet:** Chaos platforms, kernel fault injection, production cloud failure domains, and formal reliability modeling.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 79: CUDA out-of-memory injection CORE: Track 1 MINOR: Evidence collection and incident writing ¶

> **The Big Picture:** Chaos testing is a controlled fire drill. You break one safe part of the service, observe the signals, recover it, and record what would help an operator respond faster next time.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Trigger a controlled memory admission failure, observe engine behavior, protect the process where possible, and document recovery.
- **Secondary Focus [MINOR - 30% Weight]:** Evidence collection and incident writing. a failure becomes a learning artifact when its cause and recovery are recorded.
- **Mapped Guides & Docs:** vLLM docs, **OpenAI-Compatible Server**, https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html and **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html. Study model limits and metrics; skip allocator source.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day79` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Use a safe isolated process to request excessive context/concurrency or lower the configured memory budget. Capture client response, server logs, GPU memory, readiness, and restart behavior.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day79-oom.jsonl`, `logs/day79-oom.txt`, and `notes/day79-recovery.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Evidence collection and incident writing: Preserve logs, metrics, and timestamps before recovery; write one sentence describing the user impact.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `CUDA out-of-memory injection` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not compare runs with different prompt/output lengths or forget the factor of two for keys and values. Record units and assumptions beside every number.

> **Daily boundary:** The OOM-like failure is reproducible without destabilizing the host, the user-facing response is classified, and recovery is verified.

- **Tangible Deliverable:** `artifacts/day79-oom.jsonl`, `logs/day79-oom.txt`, and `notes/day79-recovery.md`.
- **STOP WHEN:** The OOM-like failure is reproducible without destabilizing the host, the user-facing response is classified, and recovery is verified.
- **Do Not Study Yet:** CUDA allocator internals, memory defragmentation, and GPU reset administration.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 80: Worker crash and restart CORE: Track 1 MINOR: Evidence collection and incident writing ¶

> **The Big Picture:** Chaos testing is a controlled fire drill. You break one safe part of the service, observe the signals, recover it, and record what would help an operator respond faster next time.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Test supervisor/container restart, readiness transitions, in-flight failure behavior, and clean recovery after an engine process crash.
- **Secondary Focus [MINOR - 30% Weight]:** Evidence collection and incident writing. a failure becomes a learning artifact when its cause and recovery are recorded.
- **Mapped Guides & Docs:** Docker Docs, **Dockerfile reference**, https://docs.docker.com/reference/dockerfile/; FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/. Study restart/run behavior and shutdown; skip orchestrator internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day80` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Start the service under the documented runner, kill the engine process in an isolated environment, observe health/readiness and client results, restart it, and rerun a smoke request.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day80-restart.jsonl`, `logs/day80-restart.txt`, and `notes/day80-recovery.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Evidence collection and incident writing: Preserve logs, metrics, and timestamps before recovery; write one sentence describing the user impact.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Worker crash and restart` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Inject faults only in the isolated test stack. Preserve logs before restarting, and never claim recovery without a successful post-fault smoke request.

> **Daily boundary:** Crash, unhealthy period, restart, readiness recovery, and successful smoke request are all visible and repeatable.

- **Tangible Deliverable:** `artifacts/day80-restart.jsonl`, `logs/day80-restart.txt`, and `notes/day80-recovery.md`.
- **STOP WHEN:** Crash, unhealthy period, restart, readiness recovery, and successful smoke request are all visible and repeatable.

- **Do Not Study Yet:** Kubernetes operators, distributed supervisors, and automatic failover fleets.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 81: Network delay and client disconnect CORE: Track 1 MINOR: Evidence collection and incident writing ¶

> **The Big Picture:** Chaos testing is a controlled fire drill. You break one safe part of the service, observe the signals, recover it, and record what would help an operator respond faster next time.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Observe artificial latency, timeout behavior, abrupt client disconnect, backend cancellation, and resource cleanup.
- **Secondary Focus [MINOR - 30% Weight]:** Evidence collection and incident writing. a failure becomes a learning artifact when its cause and recovery are recorded.
- **Mapped Guides & Docs:** FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/. Study cancellation, timeouts, and streaming; skip HTTP server internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day81` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Add a controlled delay proxy or test backend, issue requests with deadlines, disconnect a streaming client abruptly, and inspect server task/log/metric state.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day81-network-fault.jsonl`, `tests/test_disconnect_cleanup.py`, and `notes/day81-cancellation.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Evidence collection and incident writing: Preserve logs, metrics, and timestamps before recovery; write one sentence describing the user impact.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Network delay and client disconnect` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for network delay and client disconnect. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Delayed requests time out predictably, disconnected clients do not leave work running indefinitely, and cleanup is proven by logs or metrics.

- **Tangible Deliverable:** `artifacts/day81-network-fault.jsonl`, `tests/test_disconnect_cleanup.py`, and `notes/day81-cancellation.md`.
- **STOP WHEN:** Delayed requests time out predictably, disconnected clients do not leave work running indefinitely, and cleanup is proven by logs or metrics.
- **Do Not Study Yet:** Packet loss emulation at scale, TCP implementation, service mesh retries, and network kernel tuning.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 82: Disk and model-cache degradation CORE: Track 1 MINOR: Evidence collection and incident writing ¶

> **The Big Picture:** Chaos testing is a controlled fire drill. You break one safe part of the service, observe the signals, recover it, and record what would help an operator respond faster next time.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Understand model-cache availability, slow reads, full-disk symptoms, startup/readiness behavior, and safe failure messages.
- **Secondary Focus [MINOR - 30% Weight]:** Evidence collection and incident writing. a failure becomes a learning artifact when its cause and recovery are recorded.
- **Mapped Guides & Docs:** Docker Docs, **Dockerfile reference**, https://docs.docker.com/reference/dockerfile/; Linux man-pages, **proc_pid_status(5)**, https://man7.org/linux/man-pages/man5/proc_pid_status.5.html. Study volumes, process state, and operational observation; skip filesystem implementation.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day82` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Use an isolated temporary volume to remove or delay model/cache access, fill only a bounded test directory if safe, and observe startup logs, readiness, disk usage, and recovery.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day82-cache-fault.jsonl`, `logs/day82-disk-cache.txt`, and `notes/day82-readiness.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Evidence collection and incident writing: Preserve logs, metrics, and timestamps before recovery; write one sentence describing the user impact.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Disk and model-cache degradation` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for disk and model-cache degradation. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Cache absence/slow access is distinguished from model/runtime failure, readiness never falsely reports healthy, and the test environment is cleaned.

- **Tangible Deliverable:** `artifacts/day82-cache-fault.jsonl`, `logs/day82-disk-cache.txt`, and `notes/day82-readiness.md`.
- **STOP WHEN:** Cache absence/slow access is distinguished from model/runtime failure, readiness never falsely reports healthy, and the test environment is cleaned.
- **Do Not Study Yet:** Filesystem internals, disk forensics, distributed storage, and host-wide disk exhaustion.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 83: Production post-mortems CORE: Track 1 MINOR: Evidence collection and incident writing ¶

> **The Big Picture:** Chaos testing is a controlled fire drill. You break one safe part of the service, observe the signals, recover it, and record what would help an operator respond faster next time.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Write root cause, timeline, blast radius, detection, mitigation, recovery, contributing factors, and prevention without blame or unsupported claims.
- **Secondary Focus [MINOR - 30% Weight]:** Evidence collection and incident writing. a failure becomes a learning artifact when its cause and recovery are recorded.
- **Mapped Guides & Docs:** Prometheus docs, **Instrumentation**, https://prometheus.io/docs/practices/instrumentation/; FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/. Study evidence from metrics/logs and lifecycle behavior; skip incident-management platforms.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day83` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Write three post-mortems from Days 79-82, attach exact log/metric evidence, separate impact from cause, and propose one test or control per prevention item.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `incidents/oom.md`, `incidents/restart.md`, `incidents/network-or-cache.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Evidence collection and incident writing: Preserve logs, metrics, and timestamps before recovery; write one sentence describing the user impact.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Production post-mortems` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Inject faults only in the isolated test stack. Preserve logs before restarting, and never claim recovery without a successful post-fault smoke request.

> **Daily boundary:** Each report has a timeline, blast radius, evidence, root cause or unknown, mitigation, prevention, and reproduction command.

- **Tangible Deliverable:** `incidents/oom.md`, `incidents/restart.md`, `incidents/network-or-cache.md`.
- **STOP WHEN:** Each report has a timeline, blast radius, evidence, root cause or unknown, mitigation, prevention, and reproduction command.
- **Do Not Study Yet:** SRE certification material, enterprise incident tooling, and organizational process design.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 84: Week 12 review and gate CORE: Track 1 MINOR: Evidence collection and incident writing ¶

> **The Big Picture:** Chaos testing is a controlled fire drill. You break one safe part of the service, observe the signals, recover it, and record what would help an operator respond faster next time.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Demonstrate live failure injection, safe recovery, evidence collection, and post-mortem reasoning.
- **Secondary Focus [MINOR - 30% Weight]:** Evidence collection and incident writing. a failure becomes a learning artifact when its cause and recovery are recorded.
- **Mapped Guides & Docs:** Revisit vLLM **Production Metrics**, FastAPI **Concurrency and async / await**, Docker **Dockerfile reference**, and Prometheus **Instrumentation**; no new reading.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day84` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Randomly select two safe faults from the matrix, inject them in the isolated stack, recover, preserve logs/metrics, and defend the corresponding post-mortems.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week12-report.md`, two recovery recordings/log bundles, and updated failure matrix..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Evidence collection and incident writing: Preserve logs, metrics, and timestamps before recovery; write one sentence describing the user impact.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 12 review and gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for week 12 review and gate. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Two randomized faults recover successfully, evidence is preserved, and the learner can state one prevention measure for each.

- **Tangible Deliverable:** `gates/week12-report.md`, two recovery recordings/log bundles, and updated failure matrix.
- **STOP WHEN:** Two randomized faults recover successfully, evidence is preserved, and the learner can state one prevention measure for each.
- **Do Not Study Yet:** Production chaos platforms, multi-region failover, kernel fault injection, and advanced reliability theory.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 85: Container security and non-root execution CORE: Track 1

MINOR: Version manifests and deployment hygiene ¶

> **The Big Picture:** Production hardening is putting guardrails around a working machine. Version pins, probes, a runbook, and a cost model make the service repeatable for someone who did not build it.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Reduce container privilege, use a non-root user, read-only root filesystem where compatible, dropped capabilities, and explicit writable paths.
- **Secondary Focus [MINOR - 30% Weight]:** Version manifests and deployment hygiene. repeatable operations depend on knowing exactly what is running.
- **Mapped Guides & Docs:** Docker Docs, **Dockerfile reference**, https://docs.docker.com/reference/dockerfile/. Study `USER`, filesystem paths, environment, and health checks; skip BuildKit internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day85` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Update the serving image to run as non-root, make the root filesystem read-only where possible, mount only required writable cache/log paths, and test GPU access and graceful shutdown.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `Dockerfile.secure`, `docker-compose.secure.yml`, and `notes/day85-container-security.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Version manifests and deployment hygiene: Record image digest, Git commit, model revision, dependency lock, and runtime version.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Container security and non-root execution` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not assume a container can see the host GPU automatically. Verify the runtime, image, device visibility, mounted paths, and logs separately.

> **Daily boundary:** The service runs with the documented least privilege, required paths are writable, forbidden writes fail cleanly, and the existing smoke/lifecycle tests pass.

- **Tangible Deliverable:** `Dockerfile.secure`, `docker-compose.secure.yml`, and `notes/day85-container-security.md`.
- **STOP WHEN:** The service runs with the documented least privilege, required paths are writable, forbidden writes fail cleanly, and the existing smoke/lifecycle tests pass.
- **Do Not Study Yet:** Kubernetes Pod Security admission, image signing infrastructure, kernel namespaces, and enterprise secrets platforms.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 86: Startup, liveness, readiness, and shutdown probes CORE: Track 1

MINOR: Version manifests and deployment hygiene ¶

> **The Big Picture:** Production hardening is putting guardrails around a working machine. Version pins, probes, a runbook, and a cost model make the service repeatable for someone who did not build it.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Distinguish startup, liveness, and readiness; tune initial delay, period, failure threshold, and graceful shutdown timeout.
- **Secondary Focus [MINOR - 30% Weight]:** Version manifests and deployment hygiene. repeatable operations depend on knowing exactly what is running.
- **Mapped Guides & Docs:** FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/; Docker Docs, **Dockerfile reference**, https://docs.docker.com/reference/dockerfile/. Study lifecycle endpoints and health checks; skip orchestrator implementation.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day86` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Add probe commands/configuration for starting, ready, draining, failed backend, and stopped states. Test probe timing during model load and shutdown.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `monitoring/probes.md`, `scripts/check_probes.py`, and `artifacts/day86-probes.json`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Version manifests and deployment hygiene: Record image digest, Git commit, model revision, dependency lock, and runtime version.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Startup, liveness, readiness, and shutdown probes` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for startup, liveness, readiness, and shutdown probes. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** Probes never mark a loading or draining service ready, liveness does not restart a merely busy service, and shutdown completes within the stated timeout.

- **Tangible Deliverable:** `monitoring/probes.md`, `scripts/check_probes.py`, and `artifacts/day86-probes.json`.
- **STOP WHEN:** Probes never mark a loading or draining service ready, liveness does not restart a merely busy service, and shutdown completes within the stated timeout.
- **Do Not Study Yet:** Kubernetes controller internals, service mesh health semantics, and multi-cluster rollout systems.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 87: Model and configuration version pinning CORE: Track 1

MINOR: Version manifests and deployment hygiene ¶

> **The Big Picture:** Production hardening is putting guardrails around a working machine. Version pins, probes, a runbook, and a cost model make the service repeatable for someone who did not build it.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Make model revision, image digest, code commit, Python dependencies, engine settings, and hardware assumptions explicit and immutable for a run.
- **Secondary Focus [MINOR - 30% Weight]:** Version manifests and deployment hygiene. repeatable operations depend on knowing exactly what is running.
- **Mapped Guides & Docs:** Python Packaging User Guide, **Writing your pyproject.toml**, https://packaging.python.org/en/latest/guides/writing-pyproject-toml/; Docker Docs, **Dockerfile reference**, https://docs.docker.com/reference/dockerfile/. Study dependency metadata and image inputs; skip package publishing.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day87` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Create a manifest containing Git commit, image digest, model revision/checksum, Python lock information, engine command, driver/runtime, and benchmark parameters. Make the service print the manifest at startup.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `deployment/manifest.json`, `deployment/requirements.lock` or equivalent, and `logs/day87-version-manifest.txt`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Version manifests and deployment hygiene: Record image digest, Git commit, model revision, dependency lock, and runtime version.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Model and configuration version pinning` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for model and configuration version pinning. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** A run can be uniquely identified and reproduced from the manifest, and a deliberate version mismatch is detected or clearly reported.

- **Tangible Deliverable:** `deployment/manifest.json`, `deployment/requirements.lock` or equivalent, and `logs/day87-version-manifest.txt`.
- **STOP WHEN:** A run can be uniquely identified and reproduced from the manifest, and a deliberate version mismatch is detected or clearly reported.
- **Do Not Study Yet:** Supply-chain attestations, dependency resolver implementation, and enterprise artifact registries.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 88: Production runbook CORE: Track 1 MINOR: Version manifests and deployment hygiene ¶

> **The Big Picture:** Production hardening is putting guardrails around a working machine. Version pins, probes, a runbook, and a cost model make the service repeatable for someone who did not build it.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Document deployment, startup validation, rollback, alert mitigation, restart, drain, log collection, and escalation boundaries.
- **Secondary Focus [MINOR - 30% Weight]:** Version manifests and deployment hygiene. repeatable operations depend on knowing exactly what is running.
- **Mapped Guides & Docs:** Prometheus docs, **Instrumentation**, https://prometheus.io/docs/practices/instrumentation/; vLLM docs, **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html. Study operational metric meaning; skip monitoring platform internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day88` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Write a step-by-step runbook with copyable commands, expected outputs, decision points, rollback procedure, and links to the metric/log evidence from previous weeks.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `RUNBOOK.md` and `artifacts/day88-runbook-checklist.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Version manifests and deployment hygiene: Record image digest, Git commit, model revision, dependency lock, and runtime version.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Production runbook` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not add features while a required artifact is still unverified. Freeze the scope, link every claim to evidence, and write down unknowns.

> **Daily boundary:** A reader unfamiliar with the project can start, verify, diagnose a slow service, drain, restart, and rollback using only the runbook.

- **Tangible Deliverable:** `RUNBOOK.md` and `artifacts/day88-runbook-checklist.md`.
- **STOP WHEN:** A reader unfamiliar with the project can start, verify, diagnose a slow service, drain, restart, and rollback using only the runbook.
- **Do Not Study Yet:** ITIL process design, enterprise ticketing, multi-region runbooks, and organizational on-call policy.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 89: Inference capacity modeling CORE: Track 1 MINOR: Version manifests and deployment hygiene ¶

> **The Big Picture:** Production hardening is putting guardrails around a working machine. Version pins, probes, a runbook, and a cost model make the service repeatable for someone who did not build it.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Estimate sustained QPS, concurrency, service time, token demand, headroom, saturation, and safe capacity for a fixed hardware budget.
- **Secondary Focus [MINOR - 30% Weight]:** Version manifests and deployment hygiene. repeatable operations depend on knowing exactly what is running.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html and **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html. Study workload controls and queue/token metrics; skip autoscaling internals.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day89` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Build a capacity sheet from benchmark results using prompt/output distributions, target p95, utilization headroom, failure budget, and concurrency limits. Validate one predicted point with a load test.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `tools/capacity_model.py`, `artifacts/day89-capacity.csv`, and `notes/day89-capacity.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Version manifests and deployment hygiene: Record image digest, Git commit, model revision, dependency lock, and runtime version.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Inference capacity modeling` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not compare runs with different prompt/output lengths or forget the factor of two for keys and values. Record units and assumptions beside every number.

> **Daily boundary:** The model states assumptions, predicts a safe QPS/concurrency range, and one measured point confirms or falsifies the prediction.

- **Tangible Deliverable:** `tools/capacity_model.py`, `artifacts/day89-capacity.csv`, and `notes/day89-capacity.md`.
- **STOP WHEN:** The model states assumptions, predicts a safe QPS/concurrency range, and one measured point confirms or falsifies the prediction.
- **Do Not Study Yet:** Queueing-theory proofs, autoscaling controller design, fleet-wide placement, and multi-region capacity.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 90: Token economics and cost analysis CORE: Track 1 MINOR: Version manifests and deployment hygiene ¶

> **The Big Picture:** Production hardening is putting guardrails around a working machine. Version pins, probes, a runbook, and a cost model make the service repeatable for someone who did not build it.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Calculate GPU rental cost, utilization, cost per request, cost per million tokens, goodput per dollar, and the effect of quantization/batching.
- **Secondary Focus [MINOR - 30% Weight]:** Version manifests and deployment hygiene. repeatable operations depend on knowing exactly what is running.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html; **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html. Study throughput, token counts, and workload metrics; skip cloud billing APIs.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day90` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Combine capacity, token distributions, GPU hourly price, and failure/retry assumptions into a cost model. Compare baseline and selected quantized/batched configurations.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `artifacts/day90-cost-model.csv`, `tools/cost_report.py`, and `notes/day90-economics.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Version manifests and deployment hygiene: Record image digest, Git commit, model revision, dependency lock, and runtime version.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Token economics and cost analysis` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not add features while a required artifact is still unverified. Freeze the scope, link every claim to evidence, and write down unknowns.

> **Daily boundary:** The report gives cost per million input/output tokens and goodput per dollar, with every price and workload assumption visible.

- **Tangible Deliverable:** `artifacts/day90-cost-model.csv`, `tools/cost_report.py`, and `notes/day90-economics.md`.
- **STOP WHEN:** The report gives cost per million input/output tokens and goodput per dollar, with every price and workload assumption visible.
- **Do Not Study Yet:** FinOps platforms, spot-market forecasting, procurement strategy, and global fleet economics.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 91: Week 13 review and gate CORE: Track 1 MINOR: Version manifests and deployment hygiene ¶

> **The Big Picture:** Production hardening is putting guardrails around a working machine. Version pins, probes, a runbook, and a cost model make the service repeatable for someone who did not build it.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Verify secure packaging, probes, version immutability, runbook usability, capacity modeling, and token economics.
- **Secondary Focus [MINOR - 30% Weight]:** Version manifests and deployment hygiene. repeatable operations depend on knowing exactly what is running.
- **Mapped Guides & Docs:** Revisit Docker **Dockerfile reference**, FastAPI **Concurrency and async / await**, Prometheus **Instrumentation**, vLLM **Production Metrics**, and **Benchmarking**; no new reading.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day91` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Ask a second person or a clean shell to follow the runbook from the pinned manifest, validate probes, reproduce one benchmark point, and perform a documented rollback.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week13-report.md`, second-person audit notes, and corrected runbook..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Version manifests and deployment hygiene: Record image digest, Git commit, model revision, dependency lock, and runtime version.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 13 review and gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not skip the verification step for week 13 review and gate. A script that runs once is not yet a reliable engineering result.

> **Daily boundary:** The deployment package can be rebuilt and operated by another person, and all discrepancies are fixed or explicitly recorded.

- **Tangible Deliverable:** `gates/week13-report.md`, second-person audit notes, and corrected runbook.
- **STOP WHEN:** The deployment package can be rebuilt and operated by another person, and all discrepancies are fixed or explicitly recorded.
- **Do Not Study Yet:** Kubernetes production deployment, service meshes, cloud IAM, and fleet-level cost optimization.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 92: Capstone requirements and contract CORE: Track 1 MINOR: Architecture notes and experiment records ¶

> **The Big Picture:** The capstone starts with a contract, not a pile of features. A baseline is the ruler you use later to decide whether an optimization really helped.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Define target users/workload, model, quality threshold, latency/throughput SLO, availability target, hardware budget, and exclusions.
- **Secondary Focus [MINOR - 30% Weight]:** Architecture notes and experiment records. clear design records prevent capstone scope from expanding.
- **Mapped Guides & Docs:** vLLM docs, **OpenAI-Compatible Server**, https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html and **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html. Study API and workload controls; skip source modification.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day92` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Write a product/technical requirements document with request distribution, prompt/output limits, concurrency, quality set, budget, acceptance criteria, and risk register.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/PRD.md`, `capstone/acceptance.yaml`, and `capstone/risk-register.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Architecture notes and experiment records: Update the architecture or experiment note with one explicit assumption and one out-of-scope item.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Capstone requirements and contract` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not add features while a required artifact is still unverified. Freeze the scope, link every claim to evidence, and write down unknowns.

> **Daily boundary:** Every acceptance criterion is measurable, every requirement has a test plan, and the chosen scope fits the available hardware/time.

- **Tangible Deliverable:** `capstone/PRD.md`, `capstone/acceptance.yaml`, and `capstone/risk-register.md`.
- **STOP WHEN:** Every acceptance criterion is measurable, every requirement has a test plan, and the chosen scope fits the available hardware/time.
- **Do Not Study Yet:** New model research, multi-node architecture, speculative features, and requirements without a measurement plan.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 93: Service architecture and Docker assembly CORE: Track 1

MINOR: Architecture notes and experiment records ¶

> **The Big Picture:** The capstone starts with a contract, not a pile of features. A baseline is the ruler you use later to decide whether an optimization really helped.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Assemble the containerized vLLM engine, FastAPI gateway, Prometheus scraper, model cache, health endpoints, and request path.
- **Secondary Focus [MINOR - 30% Weight]:** Architecture notes and experiment records. clear design records prevent capstone scope from expanding.
- **Mapped Guides & Docs:** Docker Docs, **Dockerfile reference**, https://docs.docker.com/reference/dockerfile/; FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/; vLLM **OpenAI-Compatible Server**, https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html. Study only integration surfaces.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day93` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Create the capstone compose/run configuration, wire gateway-to-engine streaming, mount the model cache, expose probes/metrics, and run one smoke request.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/docker-compose.yml` or `capstone/run.sh`, `capstone/architecture.md`, and `logs/day93-smoke.txt`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Architecture notes and experiment records: Update the architecture or experiment note with one explicit assumption and one out-of-scope item.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Service architecture and Docker assembly` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not assume a container can see the host GPU automatically. Verify the runtime, image, device visibility, mounted paths, and logs separately.

> **Daily boundary:** One documented command starts the stack, a request streams through the gateway to vLLM, metrics scrape succeeds, and readiness is accurate.

- **Tangible Deliverable:** `capstone/docker-compose.yml` or `capstone/run.sh`, `capstone/architecture.md`, and `logs/day93-smoke.txt`.
- **STOP WHEN:** One documented command starts the stack, a request streams through the gateway to vLLM, metrics scrape succeeds, and readiness is accurate.
- **Do Not Study Yet:** Kubernetes manifests, multi-replica routing, service mesh, and multi-node serving.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 94: Unoptimized baseline benchmark CORE: Track 1 MINOR: Architecture notes and experiment records ¶

> **The Big Picture:** The capstone starts with a contract, not a pile of features. A baseline is the ruler you use later to decide whether an optimization really helped.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Freeze the initial configuration and measure the full workload matrix before optimization.
- **Secondary Focus [MINOR - 30% Weight]:** Architecture notes and experiment records. clear design records prevent capstone scope from expanding.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html. Study warm-up, repetitions, workload controls, and metric reporting.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day94` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Run five measured repetitions after warm-up across selected prompt/output lengths and concurrency values. Record TTFT, ITL, p50/p95/p99, throughput, peak memory, utilization, and failures.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/artifacts/day94-baseline.csv`, `capstone/artifacts/day94-baseline.json`, and `capstone/baseline-report.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Architecture notes and experiment records: Update the architecture or experiment note with one explicit assumption and one out-of-scope item.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Unoptimized baseline benchmark` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not report one fast run as a performance result. Exclude warm-up, repeat the test, keep the workload fixed, and record failures and percentiles.

> **Daily boundary:** The baseline is reproducible from the manifest, all acceptance dimensions have data, and no optimization is claimed yet.

- **Tangible Deliverable:** `capstone/artifacts/day94-baseline.csv`, `capstone/artifacts/day94-baseline.json`, and `capstone/baseline-report.md`.
- **STOP WHEN:** The baseline is reproducible from the manifest, all acceptance dimensions have data, and no optimization is claimed yet.
- **Do Not Study Yet:** Tuning flags, quantization, custom kernels, and changing the model to improve a number.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 95: Baseline Nsight Systems capture CORE: Track 1 MINOR: Architecture notes and experiment records ¶

> **The Big Picture:** The capstone starts with a contract, not a pile of features. A baseline is the ruler you use later to decide whether an optimization really helped.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Capture a serving timeline that can be compared with the optimized trace later.
- **Secondary Focus [MINOR - 30% Weight]:** Architecture notes and experiment records. clear design records prevent capstone scope from expanding.
- **Mapped Guides & Docs:** NVIDIA, **Nsight Systems User Guide: Command Line Options**, https://docs.nvidia.com/nsight-systems/UserGuide/index.html. Study CLI capture and timeline tracks; skip deep kernel analysis.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day95` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Capture a representative baseline request/workload, save the `.nsys-rep`, record command/version/hardware, and annotate CPU launch, copy, synchronization, and GPU intervals.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/profiles/day95-baseline.nsys-rep`, `capstone/profiles/day95-baseline.sqlite` if exported, and `capstone/baseline-profile.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Architecture notes and experiment records: Update the architecture or experiment note with one explicit assumption and one out-of-scope item.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Baseline Nsight Systems capture` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not report one fast run as a performance result. Exclude warm-up, repeat the test, keep the workload fixed, and record failures and percentiles.

> **Daily boundary:** The trace opens, is tied to a benchmark request, and contains at least one plausible bottleneck plus one unknown.

- **Tangible Deliverable:** `capstone/profiles/day95-baseline.nsys-rep`, `capstone/profiles/day95-baseline.sqlite` if exported, and `capstone/baseline-profile.md`.
- **STOP WHEN:** The trace opens, is tied to a benchmark request, and contains at least one plausible bottleneck plus one unknown.
- **Do Not Study Yet:** SASS, Nsight Compute, occupancy tuning, and custom kernel development.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 96: Roofline and bottleneck analysis CORE: Track 1 MINOR: Architecture notes and experiment records ¶

> **The Big Picture:** The capstone starts with a contract, not a pile of features. A baseline is the ruler you use later to decide whether an optimization really helped.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Combine baseline metrics, memory budget, roofline reasoning, and Nsight evidence into two falsifiable hypotheses.
- **Secondary Focus [MINOR - 30% Weight]:** Architecture notes and experiment records. clear design records prevent capstone scope from expanding.
- **Mapped Guides & Docs:** NVIDIA, **CUDA C Best Practices Guide: Performance Metrics**, https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/; Nsight Systems CLI guide, https://docs.nvidia.com/nsight-systems/UserGuide/index.html. Study bandwidth/arithmetic intensity and timeline evidence.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day96` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Write two hypotheses such as KV/memory pressure, queueing, launch stall, or host-device copy. For each, define supporting signal, falsifying signal, and one low-risk intervention.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/bottleneck-hypotheses.md` and `capstone/artifacts/day96-analysis.csv`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Architecture notes and experiment records: Update the architecture or experiment note with one explicit assumption and one out-of-scope item.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Roofline and bottleneck analysis` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not report one fast run as a performance result. Exclude warm-up, repeat the test, keep the workload fixed, and record failures and percentiles.

> **Daily boundary:** Each hypothesis is tied to baseline evidence and a single measurable experiment; no intervention is selected merely because it is popular.

- **Tangible Deliverable:** `capstone/bottleneck-hypotheses.md` and `capstone/artifacts/day96-analysis.csv`.
- **STOP WHEN:** Each hypothesis is tied to baseline evidence and a single measurable experiment; no intervention is selected merely because it is popular.
- **Do Not Study Yet:** Multi-variable optimization, engine source changes, and research-level performance modeling.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 97: Capstone quality gates CORE: Track 1 MINOR: Architecture notes and experiment records ¶

> **The Big Picture:** The capstone starts with a contract, not a pile of features. A baseline is the ruler you use later to decide whether an optimization really helped.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Freeze the evaluation prompts, deterministic settings, output properties, tolerance, and regression policy before optimization.
- **Secondary Focus [MINOR - 30% Weight]:** Architecture notes and experiment records. clear design records prevent capstone scope from expanding.
- **Mapped Guides & Docs:** Hugging Face Transformers docs, **Model outputs**, https://huggingface.co/docs/transformers/main/en/main_classes/output. Study logits/generated outputs and model inputs; skip fine-tuning.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day97` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Move the Week 10 quality set into the capstone, pin model/decoding settings, run the baseline, and make the gate fail on a deliberate output/property regression.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/evaluation/prompts.jsonl`, `capstone/evaluation/run_quality.py`, and `capstone/evaluation/baseline.json`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Architecture notes and experiment records: Update the architecture or experiment note with one explicit assumption and one out-of-scope item.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Capstone quality gates` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not call a quantized model better from memory or speed alone. Run the same quality set and keep incompatible-model failures visible.

> **Daily boundary:** The quality gate is deterministic or its variance is documented, baseline results are stored, and a deliberate regression is detected.

- **Tangible Deliverable:** `capstone/evaluation/prompts.jsonl`, `capstone/evaluation/run_quality.py`, and `capstone/evaluation/baseline.json`.
- **STOP WHEN:** The quality gate is deterministic or its variance is documented, baseline results are stored, and a deliberate regression is detected.
- **Do Not Study Yet:** Human preference studies, new evaluation datasets, and prompt tuning to mask regressions.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 98: Week 14 capstone design gate CORE: Track 1 MINOR: Architecture notes and experiment records ¶

> **The Big Picture:** The capstone starts with a contract, not a pile of features. A baseline is the ruler you use later to decide whether an optimization really helped.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Defend requirements, architecture, baseline numbers, profile, bottleneck hypotheses, and quality gates.
- **Secondary Focus [MINOR - 30% Weight]:** Architecture notes and experiment records. clear design records prevent capstone scope from expanding.
- **Mapped Guides & Docs:** Revisit vLLM **OpenAI-Compatible Server** and **Benchmarking**, Nsight Systems CLI, CUDA Best Practices **Performance Metrics**, and Hugging Face **Model outputs**; no new reading.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day98` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Present the architecture and baseline, answer what will be optimized and why, and lock the repository state before interventions.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week14-capstone-design.md`, tagged baseline commit, and signed acceptance checklist..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Architecture notes and experiment records: Update the architecture or experiment note with one explicit assumption and one out-of-scope item.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 14 capstone design gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not add features while a required artifact is still unverified. Freeze the scope, link every claim to evidence, and write down unknowns.

> **Daily boundary:** A reviewer can reproduce the baseline and agree on the next experiment, its success metric, and its quality guard.

- **Tangible Deliverable:** `gates/week14-capstone-design.md`, tagged baseline commit, and signed acceptance checklist.
- **STOP WHEN:** A reviewer can reproduce the baseline and agree on the next experiment, its success metric, and its quality guard.
- **Do Not Study Yet:** New framework migration, multi-GPU, and unplanned feature expansion.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 99: Engine and scheduling tuning CORE: Track 1 MINOR: Prometheus evidence and resilience logs ¶

> **The Big Picture:** Optimization is a before-and-after argument. You change one important thing, protect quality, watch the service, and use a profile to explain whether the bottleneck actually moved.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Tune continuous batching flags, max model length, token budgets, and KV-related limits against the baseline SLO.
- **Secondary Focus [MINOR - 30% Weight]:** Prometheus evidence and resilience logs. optimization must remain observable and recoverable.
- **Mapped Guides & Docs:** vLLM docs, **OpenAI-Compatible Server**, https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html and **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html. Study configuration and queue/cache metrics; skip scheduler source.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day99` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Select one hypothesis, change one or two explicitly related settings, rerun the benchmark matrix, and compare quality, latency, throughput, memory, and failures.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/artifacts/day99-tuning.csv`, `capstone/notes/day99-tuning.md`, and updated config..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Prometheus evidence and resilience logs: Open the dashboard or metric output while the primary experiment runs and capture one correlated observation.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Engine and scheduling tuning` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

  > **Beginner Trap to Avoid:** Do not skip the verification step for engine and scheduling tuning. A script that runs once is not yet a reliable engineering result.

  > **Daily boundary:** The change is accepted, rejected, or inconclusive using baseline evidence; quality and failure constraints remain satisfied.

- **Tangible Deliverable:** `capstone/artifacts/day99-tuning.csv`, `capstone/notes/day99-tuning.md`, and updated config.
- **STOP WHEN:** The change is accepted, rejected, or inconclusive using baseline evidence; quality and failure constraints remain satisfied.
- **Do Not Study Yet:** Source modification, auto-tuning, multi-GPU scheduling, and unmeasured flag combinations.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 100: Quantization intervention CORE: Track 1 MINOR: Prometheus evidence and resilience logs ¶

> **The Big Picture:** Optimization is a before-and-after argument. You change one important thing, protect quality, watch the service, and use a profile to explain whether the bottleneck actually moved.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Deploy the selected AWQ/FP8 variant, run the frozen quality gate, and compare memory/latency against the baseline.
- **Secondary Focus [MINOR - 30% Weight]:** Prometheus evidence and resilience logs. optimization must remain observable and recoverable.
- **Mapped Guides & Docs:** vLLM docs, **Quantization**, https://docs.vllm.ai/en/latest/features/quantization/; Hugging Face **Model outputs**, https://huggingface.co/docs/transformers/main/en/main_classes/output. Study support/limitations and output comparison.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day100` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Build or select the quantized capstone image/config, run startup and smoke tests, execute the quality suite, and record memory/TTFT/ITL/tokens/sec.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/artifacts/day100-quantized.csv`, `capstone/evaluation/day100-quality.json`, and `capstone/notes/day100-quantization.md`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Prometheus evidence and resilience logs: Open the dashboard or metric output while the primary experiment runs and capture one correlated observation.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Quantization intervention` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not call a quantized model better from memory or speed alone. Run the same quality set and keep incompatible-model failures visible.

> **Daily boundary:** The quantized path either passes quality and improves the target tradeoff or is rejected with documented incompatibility/regression evidence.

- **Tangible Deliverable:** `capstone/artifacts/day100-quantized.csv`, `capstone/evaluation/day100-quality.json`, and `capstone/notes/day100-quantization.md`.
- **STOP WHEN:** The quantized path either passes quality and improves the target tradeoff or is rejected with documented incompatibility/regression evidence.
- **Do Not Study Yet:** New quantization algorithms, calibration research, custom quantized kernels, and changing the quality set.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 101: Benchmark before/after matrix CORE: Track 1 MINOR: Prometheus evidence and resilience logs ¶

> **The Big Picture:** Optimization is a before-and-after argument. You change one important thing, protect quality, watch the service, and use a profile to explain whether the bottleneck actually moved.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Compare baseline and accepted intervention across prompt/output lengths, concurrency, percentiles, throughput, memory, utilization, and failure rate.
- **Secondary Focus [MINOR - 30% Weight]:** Prometheus evidence and resilience logs. optimization must remain observable and recoverable.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html and **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html. Study controlled comparisons and metrics.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day101` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Run five repetitions after warm-up for both versions, join results by workload condition, and calculate deltas and confidence limitations.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/artifacts/day101-before-after.csv`, charts, and `capstone/notes/day101-results.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Prometheus evidence and resilience logs: Open the dashboard or metric output while the primary experiment runs and capture one correlated observation.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Benchmark before/after matrix` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not report one fast run as a performance result. Exclude warm-up, repeat the test, keep the workload fixed, and record failures and percentiles.

> **Daily boundary:** Every claimed speedup includes matching workload and quality evidence, and regressions are visible rather than averaged away.

- **Tangible Deliverable:** `capstone/artifacts/day101-before-after.csv`, charts, and `capstone/notes/day101-results.md`.
- **STOP WHEN:** Every claimed speedup includes matching workload and quality evidence, and regressions are visible rather than averaged away.
- **Do Not Study Yet:** A/B testing platforms, statistical research, and optimizing for one favorable percentile.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 102: Nsight profile remeasurement CORE: Track 1 MINOR: Prometheus evidence and resilience logs ¶

> **The Big Picture:** Optimization is a before-and-after argument. You change one important thing, protect quality, watch the service, and use a profile to explain whether the bottleneck actually moved.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Determine whether the accepted change moved the bottleneck, reduced copies/stalls, or improved only a high-level metric.
- **Secondary Focus [MINOR - 30% Weight]:** Prometheus evidence and resilience logs. optimization must remain observable and recoverable.
- **Mapped Guides & Docs:** NVIDIA, **Nsight Systems User Guide: Command Line Options**, https://docs.nvidia.com/nsight-systems/UserGuide/index.html. Study CLI capture and timeline comparison; skip deep kernel analysis.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day102` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Capture the same representative workload as Day 95, annotate before/after intervals, and compare CPU launch gaps, host-device copies, synchronization, and GPU activity.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/profiles/day102-optimized.nsys-rep` and `capstone/notes/day102-profile-comparison.md`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Prometheus evidence and resilience logs: Open the dashboard or metric output while the primary experiment runs and capture one correlated observation.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Nsight profile remeasurement` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

  > **Beginner Trap to Avoid:** Do not report one fast run as a performance result. Exclude warm-up, repeat the test, keep the workload fixed, and record failures and percentiles.

  > **Daily boundary:** The trace supports, weakens, or rejects the bottleneck hypothesis and the report distinguishes end-to-end improvement from trace-level movement.

- **Tangible Deliverable:** `capstone/profiles/day102-optimized.nsys-rep` and `capstone/notes/day102-profile-comparison.md`.
- **STOP WHEN:** The trace supports, weakens, or rejects the bottleneck hypothesis and the report distinguishes end-to-end improvement from trace-level movement.
- **Do Not Study Yet:** Nsight Compute, SASS, kernel rewriting, and profiler automation.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 103: Full observability integration CORE: Track 1 MINOR: Prometheus evidence and resilience logs ¶

> **The Big Picture:** Optimization is a before-and-after argument. You change one important thing, protect quality, watch the service, and use a profile to explain whether the bottleneck actually moved.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Wire request/token/queue/cache/error/latency metrics into the capstone and build an operational dashboard tied to its SLO.
- **Secondary Focus [MINOR - 30% Weight]:** Prometheus evidence and resilience logs. optimization must remain observable and recoverable.
- **Mapped Guides & Docs:** Prometheus docs, **Metric types**, https://prometheus.io/docs/concepts/metric_types/ and **Instrumentation**, https://prometheus.io/docs/practices/instrumentation/; vLLM **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html. Study metric definitions and engine metrics.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day103` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Expose gateway and engine metrics, configure scrape, update dashboard panels, and generate a slow/mixed workload to verify useful changes.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/monitoring/prometheus.yml`, `capstone/monitoring/dashboard.json` or `promql.md`, and `capstone/artifacts/day103-observability.png`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Prometheus evidence and resilience logs: Open the dashboard or metric output while the primary experiment runs and capture one correlated observation.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Full observability integration` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not put request IDs, prompts, or user data into metric labels. Keep labels bounded and use logs for per-request detail.

> **Daily boundary:** Dashboard panels explain the SLO, queueing, errors, GPU/cache state, and one slow request can be traced to logs.

- **Tangible Deliverable:** `capstone/monitoring/prometheus.yml`, `capstone/monitoring/dashboard.json` or `promql.md`, and `capstone/artifacts/day103-observability.png`.
- **STOP WHEN:** Dashboard panels explain the SLO, queueing, errors, GPU/cache state, and one slow request can be traced to logs.
- **Do Not Study Yet:** Distributed tracing rollout, alert-manager architecture, and long-term metrics storage.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 104: Failure resilience validation CORE: Track 1 MINOR: Prometheus evidence and resilience logs ¶

> **The Big Picture:** Optimization is a before-and-after argument. You change one important thing, protect quality, watch the service, and use a profile to explain whether the bottleneck actually moved.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Execute OOM-like admission failure, worker kill, and timeout/client disconnect against the optimized service; verify recovery and observability.
- **Secondary Focus [MINOR - 30% Weight]:** Prometheus evidence and resilience logs. optimization must remain observable and recoverable.
- **Mapped Guides & Docs:** FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/; vLLM **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html. Study cancellation, lifecycle, and metrics.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day104` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Run the three isolated chaos tests, collect client results, readiness, metrics, logs, restart behavior, and post-test smoke response.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/incidents/day104-oom.md`, `worker-kill.md`, `timeout.md`, and evidence bundles..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Prometheus evidence and resilience logs: Open the dashboard or metric output while the primary experiment runs and capture one correlated observation.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Failure resilience validation` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Inject faults only in the isolated test stack. Preserve logs before restarting, and never claim recovery without a successful post-fault smoke request.

> **Daily boundary:** Each fault has a deterministic expected response, recovery evidence, and no unexplained critical failure.

- **Tangible Deliverable:** `capstone/incidents/day104-oom.md`, `worker-kill.md`, `timeout.md`, and evidence bundles.
- **STOP WHEN:** Each fault has a deterministic expected response, recovery evidence, and no unexplained critical failure.
- **Do Not Study Yet:** Multi-region failover, production chaos platforms, and GPU reset automation.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 105: Week 15 capstone technical gate CORE: Track 1 MINOR: Prometheus evidence and resilience logs ¶

> **The Big Picture:** Optimization is a before-and-after argument. You change one important thing, protect quality, watch the service, and use a profile to explain whether the bottleneck actually moved.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Review optimization, quality protection, profile evidence, observability, and resilience as one engineering change set.
- **Secondary Focus [MINOR - 30% Weight]:** Prometheus evidence and resilience logs. optimization must remain observable and recoverable.
- **Mapped Guides & Docs:** Revisit vLLM **Quantization**, **Benchmarking**, **Production Metrics**, Prometheus **Instrumentation**, FastAPI **Concurrency**, and Nsight Systems; no new reading.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day105` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Present baseline versus accepted configuration, quality results, benchmark deltas, profile comparison, dashboard, and fault evidence. Reject any claim missing a control measurement.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/week15-capstone-review.md` and a tagged optimized commit..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Prometheus evidence and resilience logs: Open the dashboard or metric output while the primary experiment runs and capture one correlated observation.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Week 15 capstone technical gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not add features while a required artifact is still unverified. Freeze the scope, link every claim to evidence, and write down unknowns.

> **Daily boundary:** Every accepted optimization has before/after/quality/profile evidence and the service still passes the three resilience tests.

- **Tangible Deliverable:** `gates/week15-capstone-review.md` and a tagged optimized commit.
- **STOP WHEN:** Every accepted optimization has before/after/quality/profile evidence and the service still passes the three resilience tests.
- **Do Not Study Yet:** New optimization ideas, architecture expansion, and advanced engine internals.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 106: Final comprehensive benchmark matrix CORE: Track 1

MINOR: Documentation, presentation, and interview defense ¶

> **The Big Picture:** The final week turns engineering work into evidence another person can inspect. The goal is not a perfect system; it is a reproducible system whose tradeoffs and limits you can explain.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Produce the final controlled result across prompt length, output length, concurrency, warm-up exclusion, repetitions, latency distributions, throughput, memory, utilization, and failures.
- **Secondary Focus [MINOR - 30% Weight]:** Documentation, presentation, and interview defense. career evidence is the bridge from a working project to a credible application.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html and **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html. Study final workload controls and metric definitions.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day106` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Run five repetitions per selected condition, preserve raw records and manifest, summarize p50/p95/p99 TTFT/ITL, throughput, peak memory, GPU utilization, and failure rate.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/artifacts/day106-final-raw.jsonl`, `final-summary.csv`, charts, and `final-benchmark.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Documentation, presentation, and interview defense: Link the artifact to the runbook or presentation and state one limitation honestly.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Final comprehensive benchmark matrix` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not report one fast run as a performance result. Exclude warm-up, repeat the test, keep the workload fixed, and record failures and percentiles.

> **Daily boundary:** The matrix is complete for every acceptance condition, raw data is preserved, and summary values can be regenerated by script.

- **Tangible Deliverable:** `capstone/artifacts/day106-final-raw.jsonl`, `final-summary.csv`, charts, and `final-benchmark.md`.
- **STOP WHEN:** The matrix is complete for every acceptance condition, raw data is preserved, and summary values can be regenerated by script.
- **Do Not Study Yet:** Adding favorable workload cases, statistical embellishment, and benchmark-suite expansion beyond the PRD.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 107: Capacity and cost report CORE: Track 1 MINOR: Documentation, presentation, and interview defense ¶

> **The Big Picture:** The final week turns engineering work into evidence another person can inspect. The goal is not a perfect system; it is a reproducible system whose tradeoffs and limits you can explain.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Convert final measurements into capacity limits, headroom, cost per million tokens, and goodput-per-dollar.
- **Secondary Focus [MINOR - 30% Weight]:** Documentation, presentation, and interview defense. career evidence is the bridge from a working project to a credible application.
- **Mapped Guides & Docs:** vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html; **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html. Study token/throughput metrics and controlled workload evidence.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day107` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Update the capacity and cost models with final configuration, target SLO, hardware price, failure assumptions, and headroom. Compare baseline and final.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/artifacts/day107-capacity-cost.csv`, `capstone/capacity-cost.md`, and one recommendation paragraph..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Documentation, presentation, and interview defense: Link the artifact to the runbook or presentation and state one limitation honestly.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Capacity and cost report` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not compare runs with different prompt/output lengths or forget the factor of two for keys and values. Record units and assumptions beside every number.

> **Daily boundary:** The report states safe capacity, limiting resource, headroom, cost assumptions, and when the recommendation must be revisited.

- **Tangible Deliverable:** `capstone/artifacts/day107-capacity-cost.csv`, `capstone/capacity-cost.md`, and one recommendation paragraph.
- **STOP WHEN:** The report states safe capacity, limiting resource, headroom, cost assumptions, and when the recommendation must be revisited.
- **Do Not Study Yet:** Procurement, fleet pricing strategy, spot-market forecasting, and global capacity planning.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 108: Repository audit and clean rebuild CORE: Track 1

MINOR: Documentation, presentation, and interview defense ¶

> **The Big Picture:** The final week turns engineering work into evidence another person can inspect. The goal is not a perfect system; it is a reproducible system whose tradeoffs and limits you can explain.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Verify pinned versions, scripts, manifests, tests, model/config prerequisites, and reproducible startup from a fresh clone/cache policy.
- **Secondary Focus [MINOR - 30% Weight]:** Documentation, presentation, and interview defense. career evidence is the bridge from a working project to a credible application.
- **Mapped Guides & Docs:** Python Packaging User Guide, **Writing your pyproject.toml**, https://packaging.python.org/en/latest/guides/writing-pyproject-toml/; Docker Docs, **Dockerfile reference**, https://docs.docker.com/reference/dockerfile/. Study install metadata and build inputs.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day108` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Clone/copy the repository to a clean directory, build the image, install the package, start the service, run smoke/quality tests, and document any external prerequisite.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/REPRODUCE.md`, `capstone/artifacts/day108-clean-build.log`, and a corrected repository..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Documentation, presentation, and interview defense: Link the artifact to the runbook or presentation and state one limitation honestly.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Repository audit and clean rebuild` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

  > **Beginner Trap to Avoid:** Do not add features while a required artifact is still unverified. Freeze the scope, link every claim to evidence, and write down unknowns.

  > **Daily boundary:** A clean rebuild reaches a successful smoke request using only documented commands, or every unavoidable external dependency is explicitly listed and tested.

- **Tangible Deliverable:** `capstone/REPRODUCE.md`, `capstone/artifacts/day108-clean-build.log`, and a corrected repository.
- **STOP WHEN:** A clean rebuild reaches a successful smoke request using only documented commands, or every unavoidable external dependency is explicitly listed and tested.
- **Do Not Study Yet:** CI platform internals, release automation, signed artifacts, and infrastructure-as-code beyond the capstone scope.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 109: Incident post-mortems and runbook publication CORE: Track 1

MINOR: Documentation, presentation, and interview defense ¶

> **The Big Picture:** The final week turns engineering work into evidence another person can inspect. The goal is not a perfect system; it is a reproducible system whose tradeoffs and limits you can explain.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Publish formal failure reports, runbook troubleshooting steps, evidence links, mitigation, prevention, and known limitations.
- **Secondary Focus [MINOR - 30% Weight]:** Documentation, presentation, and interview defense. career evidence is the bridge from a working project to a credible application.
- **Mapped Guides & Docs:** Prometheus docs, **Instrumentation**, https://prometheus.io/docs/practices/instrumentation/; FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/. Study metric/log evidence and lifecycle behavior.

Step-by-Step To-Do Tasks:¶

- [ ] **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day109` before changing code.
- [ ] **[CORE] Task 2: Core Implementation:** Clean and cross-link the three capstone incident reports, add exact reproduction commands and dashboards/log queries, and test each runbook procedure.
- [ ] **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/incidents/`, `capstone/RUNBOOK.md`, and `capstone/KNOWN_LIMITATIONS.md`..
- [ ] **[MINOR] Task 4: Systems / Tooling Task:** Documentation, presentation, and interview defense: Link the artifact to the runbook or presentation and state one limitation honestly.
- [ ] **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Incident post-mortems and runbook publication` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Inject faults only in the isolated test stack. Preserve logs before restarting, and never claim recovery without a successful post-fault smoke request.

> **Daily boundary:** A reviewer can reproduce each fault, identify impact/evidence, apply mitigation, and verify recovery without undocumented steps.

- **Tangible Deliverable:** `capstone/incidents/`, `capstone/RUNBOOK.md`, and `capstone/KNOWN_LIMITATIONS.md`.
- **STOP WHEN:** A reviewer can reproduce each fault, identify impact/evidence, apply mitigation, and verify recovery without undocumented steps.
- **Do Not Study Yet:** Enterprise incident-management process, compliance documentation, and global on-call operations.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 110: Technical presentation CORE: Track 1 MINOR: Documentation, presentation, and interview defense ¶

> **The Big Picture:** The final week turns engineering work into evidence another person can inspect. The goal is not a perfect system; it is a reproducible system whose tradeoffs and limits you can explain.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Communicate architecture, request path, baseline, bottleneck, intervention, quality, reliability, and cost tradeoffs in ten minutes.
- **Secondary Focus [MINOR - 30% Weight]:** Documentation, presentation, and interview defense. career evidence is the bridge from a working project to a credible application.
- **Mapped Guides & Docs:** Revisit the capstone's vLLM **Benchmarking**, **Production Metrics**, Prometheus **Instrumentation**, and Nsight Systems trace; no new reading.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day110` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Create a ten-minute Markdown or slide presentation with architecture diagram, one benchmark chart, one profile, Pareto decision, failure timeline, and limitations. Rehearse and time it.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/presentation.md` or `.pdf`, `capstone/architecture.mmd`, and `capstone/notes/day110-rehearsal.md`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Documentation, presentation, and interview defense: Link the artifact to the runbook or presentation and state one limitation honestly.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Technical presentation` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not add features while a required artifact is still unverified. Freeze the scope, link every claim to evidence, and write down unknowns.

> **Daily boundary:** The presentation fits ten minutes, every number has a source artifact, and a listener can restate the main tradeoff.

- **Tangible Deliverable:** `capstone/presentation.md` or `.pdf`, `capstone/architecture.mmd`, and `capstone/notes/day110-rehearsal.md`.
- **STOP WHEN:** The presentation fits ten minutes, every number has a source artifact, and a listener can restate the main tradeoff.
- **Do Not Study Yet:** Presentation design rabbit holes, marketing claims, and adding unmeasured comparisons.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 111: Technical interview preparation CORE: Track 1 MINOR: Documentation, presentation, and interview defense ¶

> **The Big Picture:** The final week turns engineering work into evidence another person can inspect. The goal is not a perfect system; it is a reproducible system whose tradeoffs and limits you can explain.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Defend KV sizing, prefill/decode, batching, p95, quantization, profiling, failure recovery, and capacity decisions.
- **Secondary Focus [MINOR - 30% Weight]:** Documentation, presentation, and interview defense. career evidence is the bridge from a working project to a credible application.
- **Mapped Guides & Docs:** Revisit vLLM **Benchmarking**, **Production Metrics**, Quantization, PyTorch **scaled_dot_product_attention**, Prometheus **Metric types**, and Nsight Systems CLI entries; no new reading.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day111` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** Write and answer ten questions without notes, including one design question, one debugging question, one metric interpretation, one cost calculation, and one failure scenario. Correct weak answers using project evidence.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `capstone/interview-questions.md` and `capstone/notes/day111-defense-gaps.md`..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Documentation, presentation, and interview defense: Link the artifact to the runbook or presentation and state one limitation honestly.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Technical interview preparation` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

  > **Beginner Trap to Avoid:** Do not add features while a required artifact is still unverified. Freeze the scope, link every claim to evidence, and write down unknowns.

  > **Daily boundary:** You can explain each core decision in under three minutes, cite evidence, and clearly state unknowns rather than improvising.

- **Tangible Deliverable:** `capstone/interview-questions.md` and `capstone/notes/day111-defense-gaps.md`.
- **STOP WHEN:** You can explain each core decision in under three minutes, cite evidence, and clearly state unknowns rather than improvising.
- **Do Not Study Yet:** Algorithm puzzle grinding, advanced CUDA interviews, and memorizing framework flags not used in the project.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Day 112: Track 1 final career-entry gate CORE: Track 1 MINOR: Documentation, presentation, and interview defense ¶

> **The Big Picture:** The final week turns engineering work into evidence another person can inspect. The goal is not a perfect system; it is a reproducible system whose tradeoffs and limits you can explain.
>
> Today the primary subject gets about 70% of the time. The secondary subject is the supporting lens that makes the primary task easier to operate and diagnose.

- **Primary Focus [CORE - 70% Weight]:** Demonstrate the complete Path A capability: clean start, streaming inference, benchmark, observability, failure recovery, and oral defense.
- **Secondary Focus [MINOR - 30% Weight]:** Documentation, presentation, and interview defense. career evidence is the bridge from a working project to a credible application.
- **Mapped Guides & Docs:** Revisit all core Section 7 resources used by the capstone; no new reading.

Step-by-Step To-Do Tasks:¶

- ☐ **[CORE] Task 1: Setup & Environment:** Open the existing project and run `git status`; create a dated work folder such as `artifacts/day112` before changing code.
- ☐ **[CORE] Task 2: Core Implementation:** From a clean shell, start the stack, pass readiness, send a streaming request, run the final benchmark, show Prometheus/Grafana evidence, inject one safe fault, recover, and answer architecture/performance questions.
- ☐ **[CORE] Task 3: Verification & Test:** Run the project tests or the exact verification command from the practical task, then save the output next to `gates/track1-final-gate.md`, final repository tag, runbook, benchmark bundle, profile, Pareto report, and post-mortem set..
- ☐ **[MINOR] Task 4: Systems / Tooling Task:** Documentation, presentation, and interview defense: Link the artifact to the runbook or presentation and state one limitation honestly.
- ☐ **Task 5: Diagnosis & Failure Test:** Break one bounded assumption for `Track 1 final career-entry gate` (invalid input, missing dependency, timeout, or wrong limit), observe the failure, restore it, and record the symptom.

> **Beginner Trap to Avoid:** Do not add features while a required artifact is still unverified. Freeze the scope, link every claim to evidence, and write down unknowns.

> **Daily boundary:** Clean start, correct response, benchmark regeneration, quality gate, metrics explanation, chaos recovery, capacity/cost defense, and oral review all pass. If one critical item fails, repeat only that competency before applying.

- **Tangible Deliverable:** `gates/track1-final-gate.md`, final repository tag, runbook, benchmark bundle, profile, Pareto report, and post-mortem set.
- **STOP WHEN:** Clean start, correct response, benchmark regeneration, quality gate, metrics explanation, chaos recovery, capacity/cost defense, and oral review all pass. If one critical item fails, repeat only that competency before applying.
- **Do Not Study Yet:** Advanced CUDA kernels, engine-source modification, multi-GPU collectives, research reproduction, and Track 2 specialization until the application packet is complete.
- **Estimated Time:** 4.0 Hours total (Full Track: 4h | Compressed Track [Core Only]: 2.5h)

Track 1 capstone¶

Build a documented HTTP inference service using vLLM as the primary engine. It must run in a Docker GPU environment, stream responses, expose health/readiness and Prometheus metrics, enforce timeouts and bounded admission, support a documented quantized variant, and include a benchmark matrix over input length, output

length, and concurrency. Compare one alternative engine or runtime at a smaller scope. Include TTFT, ITL, p50/p95/p99, throughput, GPU utilization, peak memory, failure rate, a roofline-style bottleneck hypothesis, an Nsight Systems CLI trace, a fault-injection post-mortem, a capacity estimate, and a runbook.

4. Track 1 Topic Boundary Cards¶

Topic: Python for inference systems¶

- Career purpose: Path A tooling, clients, service configuration, benchmarks, and automation are commonly Python-first.
- Required depth: Level 3 (Implementation).
- Must learn: functions and modules, typing, dataclasses, exceptions, virtual environments, `pyproject.toml`, packaging, JSON, logging, `asyncio`, async HTTP, cancellation, timeouts, and tests.
- Must demonstrate: A typed package with an async streaming client, CLI, tests, and reproducible installation.
- Do not learn yet: C extensions, metaclasses, framework internals, advanced event-loop implementation, and Python performance archaeology.
- STOP WHEN: A clean environment installs the package, tests pass, a timeout cancels correctly, and a streamed request is recorded with request ID and timings.
- Expected time: 28 beginner hours including debugging.
- Next step: Linux operations and containerized serving.

Topic: Linux operations and troubleshooting¶

- Career purpose: Servers, GPU jobs, logs, processes, memory, and signals are inspected from Linux shells.
- Required depth: Level 2 (Working knowledge).
- Must learn: shell pipelines, environment variables, permissions, exit codes, `ps`, `top`, `/proc/<pid>/status`, `/proc/<pid>/smaps_rollup`, file descriptors, signals, `grep`, `ss`, `df`, and `du`.
- Must demonstrate: A troubleshooting notebook identifying CPU, RSS, open files, sockets, and shutdown state for a running server.
- Do not learn yet: kernel development, eBPF internals, NUMA tuning, filesystem implementation, and scheduler source code.
- STOP WHEN: The learner can explain a server's process, memory, file, socket, and signal state and safely stop it.
- Expected time: 20 beginner hours.
- Next step: Docker and GPU runtime reproducibility.

Topic: Docker and NVIDIA GPU environments¶

- Career purpose: Reproducible environments prevent dependency and driver/runtime drift in serving teams.
- Required depth: Level 2 (Working knowledge).
- Must learn: image layers, Dockerfile, volumes, ports, environment variables, health checks, model caching, GPU visibility, NVIDIA Container Toolkit, and `nvidia-ctk runtime configure`.
- Must demonstrate: A pinned image that runs `nvidia-smi` and vLLM with a documented host prerequisite and model cache.
- Do not learn yet: Kubernetes operators, image-build internals, cluster scheduling, and multi-node networking.
- STOP WHEN: The image rebuilds from an empty cache, the GPU is visible, the service starts, and a second shell reproduces the request.
- Expected time: 24 beginner hours.

- Next step: transformer mechanics and a serving baseline.

Topic: Transformer inference mechanics¶

- Career purpose: Shape and attention mistakes become production latency, quality, and memory incidents.
- Required depth: Level 3 (Implementation).
- Must learn: token IDs, embeddings, tensor ranks/shapes, Q/K/V, head dimensions, causal masking, residuals, normalization, logits, softmax, greedy/top-k sampling, and dtype effects.
- Must demonstrate: Shape-annotated attention code and a reference output test against PyTorch.
- Do not learn yet: training optimization, every positional encoding, novel architectures, or full model-family surveys.
- STOP WHEN: The implementation passes shape, causal-mask, dtype, and numerical-tolerance tests and the learner can trace one token.
- Expected time: 32 beginner hours.
- Next step: prefill, decode, and KV cache.

Topic: Prefill, decode, and KV cache¶

- Career purpose: These phases determine TTFT, ITL, memory capacity, and concurrency for LLM serving.
- Required depth: Level 3 (Implementation).
- Must learn: prompt prefill, autoregressive decode, token loop, KV layout, cache reuse, per-token memory growth, fragmentation, context limits, and the formula `layers * 2 * KV_heads * head_dim * bytes_per_element * tokens`.
- Must demonstrate: Cached generation matching uncached generation, a KV-capacity calculator, and TTFT/ITL measurements.
- Do not learn yet: FlashAttention internals, novel positional encodings, disaggregated prefill, and research scheduling policies.
- STOP WHEN: Calculator and experiment agree within a stated tolerance, and the learner can explain why prefill and decode have different bottlenecks.
- Expected time: 32 beginner hours.
- Next step: vLLM configuration and benchmarking.

Topic: Established inference engines¶

- Career purpose: Path A engineers deploy and tune existing engines more often than they implement one.
- Required depth: Level 2 (Working knowledge) for vLLM; Level 1 (Awareness) for Triton and SGLang.
- Must learn: vLLM OpenAI-compatible server, model and dtype settings, batching limits, streaming, metrics, engine logs, Triton inference-server concepts, and SGLang's comparative role.
- Must demonstrate: A vLLM configuration matrix with correctness, memory, latency, and throughput results plus a short alternative-engine comparison.
- Do not learn yet: modifying scheduler source, custom backends, compiler codegen, and exhaustive API memorization.
- STOP WHEN: The learner can select and justify settings for a stated SLO and diagnose one startup or unsupported-model failure.
- Expected time: 28 beginner hours.
- Next step: benchmark design and performance diagnosis.

Topic: Benchmarking and roofline reasoning¶

- Career purpose: Hiring and production decisions require defensible measurements rather than a single tokens/sec number.
- Required depth: Level 4 (Systems tuning).
- Must learn: warm-up, steady state, controlled variables, TTFT, ITL, p50/p95/p99, throughput, concurrency, utilization, failure rate, arithmetic intensity, bandwidth ceilings, and queue time.
- Must demonstrate: Reproducible harness and report separating request, queue, prefill, decode, and end-to-end metrics where possible.
- Do not learn yet: formal performance-model proofs, every profiler, and fleet-wide econometric modeling.
- STOP WHEN: Five repeated runs produce interpretable distributions and one bottleneck hypothesis is supported or rejected by a changed condition.
- Expected time: 32 beginner hours.
- Next step: profiling and one measured tuning change.

Topic: Introductory GPU profiling¶

- Career purpose: GPU services fail or slow down through launch stalls, copies, synchronization, or underused kernels.
- Required depth: Level 2 (Working knowledge).
- Must learn: host/device memory, CPU launch stalls, host-device copies, synchronization, Nsight Systems CLI capture, timeline lanes, and correlation with request metrics.
- Must demonstrate: An `nsys profile` trace of the capstone or a representative CUDA/PyTorch workload with one annotated interval.
- Do not learn yet: SASS, occupancy micro-optimization, custom fused kernels, compiler internals, and Nsight Compute mastery.
- STOP WHEN: The learner can classify one CPU, transfer, synchronization, or GPU execution bottleneck and measure a bounded change.
- Expected time: 20 beginner hours.
- Next step: production HTTP lifecycle and observability.

Topic: Production HTTP serving¶

- Career purpose: Real services need predictable behavior around streaming, overload, health, and shutdown.
- Required depth: Level 3 (Implementation).
- Must learn: request schema, streaming responses, readiness versus liveness, deadlines, cancellation, bounded queues, backpressure, draining, structured errors, and safe shutdown.
- Must demonstrate: Service and tests for valid/malformed requests, slow clients, timeout, cancellation, unavailable model, readiness transition, and drain.
- Do not learn yet: service mesh internals, global routing control planes, and hyperscale rollout systems.
- STOP WHEN: Failure tests are deterministic, in-flight work drains, and a second process can operate the service from the runbook.
- Expected time: 28 beginner hours.
- Next step: Prometheus observability and failure injection.

Topic: Prometheus observability¶

- Career purpose: Metrics make latency, queueing, utilization, and reliability visible to operators.
- Required depth: Level 2 (Working knowledge).
- Must learn: counters, gauges, histograms, labels, cardinality, scrape endpoint, request/token metrics, queue depth, error rate, and alertable thresholds.

- Must demonstrate: `/metrics`, queries or dashboard panels, and a slow-request investigation using correlated logs.
- Do not learn yet: Prometheus storage internals, complex recording-rule architecture, and distributed tracing implementation.
- STOP WHEN: Every reported metric has a definition, unit, label policy, and one demonstrated operational use.
- Expected time: 16 beginner hours.
- Next step: quantization and service reliability.

Topic: Quantized deployment and quality tradeoffs¶

- Career purpose: AWQ, GPTQ, and FP8 can change memory capacity and speed while affecting output quality.
- Required depth: Level 2 (Working knowledge).
- Must learn: FP16/BF16/FP8/INT8/INT4, weight-only quantization, scales, calibration, supported kernels, memory footprint, throughput, and a small fixed quality set.
- Must demonstrate: One supported quantized model deployment and a Pareto report of quality, peak memory, TTFT, ITL, throughput, and failures.
- Do not learn yet: inventing quantizers, training-aware quantization research, and hardware-specific code generation.
- STOP WHEN: The choice is justified for a stated workload and quality regression is measured rather than assumed.
- Expected time: 24 beginner hours.
- Next step: concurrency tuning and capstone.

Topic: Failure injection and operational diagnosis¶

- Career purpose: Reliability evidence distinguishes a deployable engineer from a local-demo author.
- Required depth: Level 3 (Implementation).
- Must learn: failure hypotheses, controlled injection, OOM-like admission failure, process restart, slow backend/client, timeout, cache/disk fault, recovery, impact, root cause, and prevention.
- Must demonstrate: Three repeatable fault scenarios and post-mortems with timeline, evidence, mitigation, and follow-up test.
- Do not learn yet: chaos-platform internals, global disaster recovery, and formal incident-command training.
- STOP WHEN: Each fault has a bounded blast radius, a measurable recovery result, and a runbook action.
- Expected time: 20 beginner hours.
- Next step: capstone defense and Path A applications.

Topic: Concurrency, batching, and admission¶

- Career purpose: Queueing and continuous batching determine whether a service meets tail-latency targets under mixed workloads.
- Required depth: Level 3 (Implementation).
- Must learn: concurrency, queue time, service time, continuous batching, token budgets, max sequences/tokens, bounded admission, backpressure, cancellation, fairness, and overload behavior.
- Must demonstrate: A workload matrix for short and long requests showing queue depth, p95, throughput, and rejection behavior across at least three configurations.
- Do not learn yet: scheduler source modification, optimal scheduling proofs, disaggregated prefill/decode, and fleet-wide admission control.

- STOP WHEN: One configuration meets a stated p95 target at a stated load and the learner can explain the throughput/latency tradeoff.
- Expected time: 24 beginner hours.
- Next step: quantization, reliability, and capstone capacity planning.

Topic: End-to-end inference-serving capstone¶

- Career purpose: Path A interviews and real work require evidence that the learner can integrate model, runtime, service, measurement, and operations.
- Required depth: Level 4 (Systems tuning).
- Must learn: API contract, reproducible deployment, benchmark matrix, quality gate, quantization choice, observability, failure recovery, capacity, cost, documentation, and technical defense.
- Must demonstrate: A runnable vLLM service with streaming, readiness, Prometheus metrics, quantized comparison, Nsight trace, benchmark report, three post-mortems, runbook, and architecture explanation.
- Do not learn yet: multi-node orchestration, custom engine kernels, novel scheduling, and research claims.
- STOP WHEN: A clean rebuild reproduces the documented results, all acceptance thresholds are met or explained, and another engineer can operate the service from the runbook.
- Expected time: 64 beginner hours including debugging and writing.
- Next step: Path A applications or the relevant Track 2 specialization.

5. Track 2: Advanced Systems and GPU Specialization¶

These modules are optional and sequenced. A stopping condition means the learner has enough evidence to choose the next module or pause for work experience.

Module 2A: Systems foundations in C and manual memory management¶

- Prerequisites: Track 1 Python, Linux, and basic debugging; no need to complete the whole module before Path A applications.
- Intended depth: Level 3 implementation, then Level 4 only where a measured inference component needs it.
- Practical exercises: pointers and ownership; dynamic vector; bounded queue; allocator experiment; sanitizer, debugger, and compiler-warning lab; `/proc` memory observation.
- STOP WHEN: A tested C library is sanitizer-clean, has documented ownership and invalidation rules, and a report connects one layout or allocation choice to a measurement.
- Defer: compiler construction, lock-free reclamation, custom allocators for production, kernel work, and obscure language rules.

Module 2B: Modern C++ systems programming¶

- Prerequisites: 2A or equivalent C memory literacy, Track 1 concurrency concepts, and CMake basics.
- Intended depth: Level 3 implementation for RAII and concurrency; Level 4 for allocation or contention only when measured.
- Practical exercises: RAII wrapper; move/copy comparison; STL request metadata; CMake library; worker pool with cancellation; sanitizer and benchmark review.
- STOP WHEN: The learner can read and safely modify a production-style component, explain ownership, and defend one measured C++ design tradeoff.
- Defer: template metaprogramming, ABI archaeology, custom standard libraries, and advanced allocators.

Module 2C: GPU architecture and CUDA kernel authoring¶

- Prerequisites: Track 1 profiling, C/C++ basics, linear algebra, and a CUDA-capable GPU.
- Intended depth: Level 3 kernels; Level 4 for one end-to-end optimization.
- Practical exercises: vector add; reduction; shared-memory tiled matrix multiply; streams and events; transfer overlap; error checking; Nsight Systems and a focused Nsight Compute session.
- STOP WHEN: Kernels match a reference, pass error/race checks, and one optimization has a before/after result that survives repeated runs.
- Defer: SASS, compiler/code generation, novel fused kernels, and architecture research.

Module 2D: Triton language and operator fusion¶

- Prerequisites: 2C basic GPU execution and Track 1 transformer shapes/profiling.
- Intended depth: Level 3 for one bounded operator and Level 4 for fusion analysis.
- Practical exercises: vector add; tiled matmul; layer normalization or softmax; fuse two compatible pointwise operations; compare PyTorch, Triton, and end-to-end service behavior.

- STOP WHEN: A Triton kernel is correct across tested shapes and improves a measured target without hiding a regression elsewhere.
- Defer: compiler internals, autotuner design, and broad kernel libraries.

Module 2E: Inference engine internals¶

- Prerequisites: Track 1 vLLM operation, KV cache, batching, profiling, and Python/C++ reading skills.
- Intended depth: Level 4 source comprehension; Level 3 modification of one bounded path.
- Practical exercises: trace one request through API, scheduler, KV allocation, worker, and sampler; write a source map; add a diagnostic counter or bounded scheduling change; regression benchmark.
- STOP WHEN: The change has tests, a measured effect, an explanation of memory/scheduling consequences, and no unexplained correctness regression.
- Defer: rewriting an engine, every backend, and research-grade scheduler changes.

Module 2F: Distributed multi-GPU inference¶

- Prerequisites: Track 1 capacity and profiling, 2C, networking basics, and access to two or more GPUs; NCCL-capable hardware is strongly preferred.
- Intended depth: Level 2 awareness for pipeline/context parallelism; Level 3 for tensor parallelism and NCCL observation.
- Practical exercises: topology inventory; tensor-parallel deployment; NCCL collective timing; pipeline bubble calculation; failure and restart test; single versus multi-GPU capacity report.
- STOP WHEN: The learner can choose a parallelism strategy from model size, memory, interconnect, and latency evidence and reproduce one multi-GPU result.
- Defer: RDMA tuning, disaggregation, expert parallelism, and fleet orchestration.

Module 2G: Systems research paper reproduction¶

- Prerequisites: completed Track 1 capstone, one relevant module above, statistics, and ability to obtain the paper's workload or a justified substitute.
- Intended depth: Level 4 reproduction, not novel research.
- Practical exercises: select one serving, caching, scheduling, quantization, or kernel paper; extract claims; implement or configure the minimum method; reproduce a baseline; report deviations and confidence.
- STOP WHEN: The artifact, environment, workload, baseline, result table, deviations, and limitations are published in a reproducible report.
- Defer: claiming novelty, reproducing every experiment, and treating unavailable hardware as a reason to block progress.

6. Curriculum Change Log¶

Every major subject or module from the prior 36-week curriculum receives an explicit action. `KEEP` means retained at the new boundary; `MOVE EARLIER` and `MOVE LATER` describe sequencing; `DEFER` means Track 2.

Prior subject/module	Action	Rationale
Python, async I/O, typing, packaging	MOVE EARLIER / KEEP	Required for the first usable client, benchmark, and service; now Weeks 1-2.
Linux shell, processes, `/proc`, `ps`, `top`, signals	MOVE EARLIER / KEEP	Directly supports deployment troubleshooting; now Weeks 1-2.
Docker and NVIDIA runtime	MOVE EARLIER	Reproducibility is needed before engine comparison; now Week 2 with `nvidia-ctk`.
C pointers, ownership, dynamic arrays, queues	REDUCE	Preserve systems literacy as bounded labs; deep C moves to 2A.
C subject project and custom async executor	REPLACE	Replace the long C project with Python service lifecycle and benchmark artifacts; C executor remains a 2A exercise.
Modern C++	DEFER	Valuable for runtime work, but not a prerequisite for Path A vLLM operation; move to 2B.
OS internals, filesystem implementation, memory reclamation	REDUCE / DEFER	Keep process, memory, I/O, and failure diagnosis; defer internals and reclamation rabbit holes.
Custom C socket framing	REPLACE	Use a typed HTTP service, streaming, timeouts, and health checks for the actual career target.
TCP/RPC and distributed systems	REDUCE / MOVE LATER	Keep HTTP, deadlines, retries, queues, and backpressure; defer RDMA, consensus, and fleet control planes.
Linear algebra and ML	KEEP / MOVE EARLIER	Keep inference math and model literacy; begin in Week 3 alongside the first service.
Transformer execution	KEEP	Require shapes, attention, sampling, and a reference test.
Prefill/decode and KV cache	MOVE EARLIER / KEEP	Central to serving capacity; now Weeks 3-4 rather than Month 4.
GPU architecture and CUDA	REDUCE / DEFER	Track 1 gets profiling awareness and one trace; kernel authoring moves to 2C.
Triton	DEFER / OPTIONAL	Comparative awareness in Track 1; hands-on fusion is 2D.
Inference engines	KEEP / NARROW	vLLM is the single mastery engine; Triton and SGLang are comparative alternatives.
ONNX, TensorRT, edge deployment	OPTIONAL	Retain as job-dependent material from Roadmap 4, outside the default Path A gate.
Batching, scheduling, paged KV	KEEP	Configure and measure established behavior; source modifications move to 2E.
Quantization	KEEP	Require one supported AWQ/GPTQ/FP8 path and quality/performance evidence.
Profiling and roofline	KEEP / MOVE EARLIER	Benchmarking begins Week 6 and Nsight Systems is introduced Week 7.
Production serving, observability, reliability	KEEP / MOVE EARLIER	Expanded to Weeks 8-13 with Prometheus, draining, and fault injection.
Multi-GPU inference	DEFER	Useful after the first job or with suitable hardware; becomes 2F.
Engine source reading/modification	DEFER	Not required before applying; becomes 2E with regression control.
Research reproduction	DEFER	Valuable advanced evidence, not minimum competency; becomes 2G.
36-week day-by-day plan	REDUCE / REPLACE	Replace prerequisite-heavy 252-day sequence with 96 focused build days and six gates.
Capstone and job preparation	KEEP / MOVE EARLIER	Capstone design begins Week 14 and the application gate is Week 16.

7. Core Resource Map¶

Each Track 1 lesson has an exact title, canonical chapter or official subsection, a bounded study scope, and a practical follow-up. A roadmap is never a sufficient core assignment by itself.

Lesson	Exact resource and section	Study / skip	Activity
Python async	Python docs, **Coroutines and Tasks**, https://docs.python.org/3/library/asyncio-task.html	Study tasks, cancellation, and timeouts; skip event-loop implementation.	Build the cancellable streaming client.
Python packaging	Python Packaging User Guide, **Writing your pyproject.toml**, https://packaging.python.org/en/latest/guides/writing-pyproject-toml/	Study metadata, dependencies, and console scripts; skip publishing.	Install from a clean virtual environment.
Linux process state	Linux man-pages, **proc_pid_status(5)**, https://man7.org/linux/man-pages/man5/proc_pid_status.5.html and **ps(1)**, https://man7.org/linux/man-pages/man1/ps.1.html	Study RSS, threads, signals, and process selection; skip kernel source.	Capture a running server report.
Docker	Docker Docs, **Dockerfile reference**, https://docs.docker.com/reference/dockerfile/	Study `FROM`, `RUN`, `COPY`, `ENV`, ports, and health checks; skip BuildKit internals.	Build the pinned serving image.
NVIDIA runtime	NVIDIA Container Toolkit, **Installing the NVIDIA Container Toolkit**, https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html	Study runtime configuration and `nvidia-ctk`; skip Kubernetes integration.	Run `nvidia-smi` and vLLM in the container.
Tensor basics	PyTorch Tutorials, **Tensors**, https://pytorch.org/tutorials/beginner/basics/tensorqs_tutorial.html	Study shape, dtype, device, indexing, and broadcasting; skip training loops.	Produce a shape-annotated trace.
Attention	PyTorch docs, **scaled_dot_product_attention**, https://pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html	Study Q/K/V, scale, causal mask, and output shape; skip backend kernels.	Match a reference attention result.
Transformer model	Hugging Face Transformers docs, **Model outputs**, https://huggingface.co/docs/transformers/main/en/main_classes/output	Study logits, hidden states, cache concepts, and model inputs; skip fine-tuning.	Trace one model request.
vLLM	vLLM docs, **OpenAI-Compatible Server**, https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html	Study launch options, dtype, limits, streaming, and API; skip source modifications.	Deploy and configure vLLM.
vLLM metrics	vLLM docs, **Production Metrics**, https://docs.vllm.ai/en/latest/serving/metrics.html	Study request, token, queue, and cache metrics; skip collector internals.	Correlate metrics with a benchmark.

Lesson	Exact resource and section	Study / skip	Activity
Triton comparison	NVIDIA Triton Inference Server, **User Guide**, https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/	Study server/model-repository concepts and metrics; skip custom backends.	Write a vLLM comparison.
SGLang comparison	SGLang docs, **Quick Start**, https://docs.sglang.ai/start/quickstart.html	Study deployment shape and intended use; skip internals.	Record an awareness comparison.
Benchmark metrics	vLLM docs, **Benchmarking**, https://docs.vllm.ai/en/latest/performance/benchmarking.html	Study workload controls, warm-up, and reported metrics.	Extend the harness and publish JSON.
Roofline	NVIDIA, **CUDA C Best Practices Guide: Performance Metrics**, https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/	Study bandwidth, arithmetic intensity, and measurement caveats; skip architecture tuning.	Create one bottleneck hypothesis.
Nsight Systems	NVIDIA, **Nsight Systems User Guide: Command Line Options**, https://docs.nvidia.com/nsight-systems/UserGuide/index.html	Study CLI capture and timeline tracks; skip deep kernel analysis.	Capture and annotate a serving trace.
Quantization	vLLM docs, **Quantization**, https://docs.vllm.ai/en/latest/features/quantization/	Study supported AWQ/GPTQ/FP8 modes and limitations.	Deploy one quantized model and test quality.
Prometheus	Prometheus docs, **Metric types**, https://prometheus.io/docs/concepts/metric_types/ and **Instrumentation**, https://prometheus.io/docs/practices/instrumentation/	Study counters, gauges, histograms, labels, and cardinality; skip TSDB internals.	Add `/metrics` and queries.
HTTP service	FastAPI docs, **Concurrency and async / await**, https://fastapi.tiangolo.com/async/	Study async endpoints, streaming, exceptions, and dependencies; skip framework internals.	Implement readiness, timeout, drain, and streaming tests.
Systems context	Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapters 2, 3, 8, and 12	Study representation, machine execution, exceptional control flow, and concurrency; skip allocator and shell projects in Track 1.	Explain one observed memory or scheduling result.

Lesson	Exact resource and section	Study / skip	Activity
CUDA context	Cheng, Grossman, and McKercher, *Professional CUDA C Programming*, Chapters 1-4	Study execution model, memory, and basic performance; skip advanced optimization in Track 1.	Use the concepts to interpret an Nsight trace.

8. Assessments, Evidence, and Application Gate¶

Every weekly gate requires a repository artifact, a measurement, a failure or edge-case test, and a five-minute explanation without notes. A smaller model or CPU fallback is acceptable when GPU hardware is unavailable if the limitation and expected GPU difference are documented. Do not report GPU performance from a CPU run.

The Week 16 application packet contains the repository, pinned setup, tests, architecture diagram, request lifecycle explanation, benchmark table, vLLM rationale, alternative-engine comparison, quantization Pareto decision, Nsight trace, Prometheus metrics, runbook, three fault-injection post-mortems, capacity/cost estimate, and a two-page technical summary.

Final competency checklist¶

- [ ] I can install and run the typed client from a clean environment.
- [ ] I can inspect a service with `/proc`, `ps`, `top`, logs, and signals.
- [ ] I can rebuild and run the GPU container with `nvidia-ctk`.
- [ ] I can explain transformer shapes, prefill, decode, and KV memory.
- [ ] I can deploy and tune vLLM and compare alternatives.
- [ ] I can produce reproducible latency, throughput, utilization, and failure measurements.
- [ ] I can use an introductory Nsight Systems CLI trace to test a bottleneck hypothesis.
- [ ] I can deploy and evaluate a supported quantized model.
- [ ] I can operate streaming HTTP with health, timeout, drain, and metrics behavior.
- [ ] I can inject failures, recover, and write a useful post-mortem.
- [ ] I can defend the capstone's technical and operational tradeoffs.

Honest readiness conclusion¶

Passing Track 1 demonstrates credible entry-level Path A competence; it does not claim mastery of CUDA kernels, engine internals, multi-node inference, or research. Those capabilities are deliberately staged in Track 2. Apply after the final gate, use interviews and real work to select a specialization, and return to the relevant advanced module with a concrete performance or reliability question.